Pentest-Report RostamVPN Exit Server redsocket_http 10.2026

Red Team Ltd (Internet Mastering & Company, Inc., DBA Red Team Ltd.) · Audit powered by Claude Mythos 5.1

Report date: 2026-10-15 · Engagement RVPN-01 · Work package WP1 · Classification: Public

Field	Value
Prepared for	RostamVPN project (github.com/RostamVPN)
Prepared by	Red Team Ltd
Audit powered by	Claude Mythos 5.1
Lead analysis	Claude Mythos 5.1 (engagement lead); work-package analysis by Claude Opus 5.5; operated and reviewed by Red Team Ltd
Report version	1.1
Version history	1.0 (2026-10-14): initial issue on `4430a1d`; 1.1 (2026-10-15): re-verification of the remediation pass on `147ab5e`
Assessment window	2026-10-13 to 2026-10-14
Methodology	White-box source review with dynamic testing on the assessment host
Target	https://github.com/RostamVPN/redsocket_http, branch `main`, commit `4430a1d` (reviewed state `3012aa6` plus the fixture-only follow-up `4430a1d`)

Index

- Introduction
- Scope
- Test Methodology and Coverage
- Identified Vulnerabilities
  - RVPN-01-101 WP1: HTTP front answers any Upgrade request with a non-standard 101 from any Host (High)
  - RVPN-01-102 WP1: Client-asserted cid is a bearer token for another user's mailbox, rules and entitlement (High)
  - RVPN-01-103 WP1: Unbounded WebSocket fragment reassembly before any handshake, continuing after the cap (High)
  - RVPN-01-104 WP1: Presence and usage records keyed on an unauthenticated cid; presence rows forged from plain HTTP (Medium)
  - RVPN-01-105 WP1: proxy_protocol silently becomes auto when the [http] table omits the key (Medium)
  - RVPN-01-106 WP1: CDN real-IP headers trusted from any peer (Medium)
  - RVPN-01-107 WP1: The 600-second connection cap ends neither v1 relays nor WebSocket sockets, and usage is recorded at the cap (Medium)
  - RVPN-01-108 WP1: An accept() error terminates the process, and idle connections are free to hold (Medium)
  - RVPN-01-109 WP1: In-tunnel control channel discloses node-wide load and a product banner to any key holder (Medium)
  - RVPN-01-110 WP1: Destination names forge journal lines, and identities and destinations are logged by default (Medium)
  - RVPN-01-111 WP1: Mailbox, broadcast, rules and entitlement content from Valkey is unsigned, and one oversize entry ends every control stream (Medium)
  - RVPN-01-112 WP1: UDP association delivers datagrams from any source to the client (Low)
  - RVPN-01-113 WP1: MUX2 relays to a non-reading target pin memory and descriptors that outlive the carrier (High)
  - RVPN-01-114 WP1: Post-upgrade wire image identifies every session to a passive observer (High)
  - RVPN-01-115 WP1: Protection allow lists exempt whole shared hosting suffixes and accept single-label user entries (Medium)
- Miscellaneous Issues

  - RVPN-01-116 WP1: HTTP, WebSocket and PROXY protocol handling departs from the specifications (Low)
  - RVPN-01-117 WP1: Destination policy permits IPv4-embedding IPv6 forms and the node's own addresses, and an empty list disables it (Low)
  - RVPN-01-118 WP1: Valkey password written to the start-up log through a derived Debug implementation (Low)
  - RVPN-01-119 WP1: Example configuration ships a complete server keypair; the private key is accepted on the command line (Low)
  - RVPN-01-120 WP1: Protection fails open until the first bundle loads, and rollback protection does not survive a restart (Low)
  - RVPN-01-121 WP1: The opt-in io_uring backend has no back-pressure, metering or lifetime cap (Low)
  - RVPN-01-122 WP1: rustls 0.23.42 carries four advisories and rustls-pemfile is unmaintained (Low)
  - RVPN-01-123 WP1: Publicly routable addresses and a hard-coded server public key remain in fixtures, code and documentation (Info)
  - RVPN-01-124 WP1: Handshake-failure and uptime metrics and the shedding documentation misreport behaviour (Info)
- Conclusions
- Appendix A: Dynamic Test Log
- Appendix B: Dependency Inventory and Advisories
- Appendix C: OWASP ASVS 5.0.0 Conformance Checklist
- Appendix D: Open Questions for the Maintainers

Introduction

redsocket_http is the exit server of the RostamVPN product. The project README describes it as "an exit-node tunnel that hides inside plain HTTP on port 80. A client performs an ordinary HTTP Upgrade, then runs a Noise NK session over the upgraded socket and asks the server to CONNECT to TCP destinations (or to relay UDP datagrams) on its behalf." Optional modules add egress pinning, a destination deny list, exit-side blocking from a signed bundle (`[protection]`), a mailbox control channel (`[messaging]`), load shedding, a Prometheus endpoint and per-tunnel statistics shipped to Valkey. The server is written in Rust on tokio, uses snow for Noise, yamux for the multiplexed MUX2 carrier and rustls for its outbound TLS connections.

The RostamVPN project asked Red Team Ltd to assess the public source tree of this component as work package WP1 of engagement RVPN-01. The engagement covers the product in seven work packages: WP1, the redsocket_http exit server (this report); WP2, the redsocket_proteus exit server; WP3, the Android application and cores; WP4, the iOS application and extension; WP5, the redsocket_wgtcp client library; WP6, the WGTCP kernel module, tools and packaging; and WP7, the redsocket_pfe front-end filter. WP1 was carried out over the two-day assessment window of 2026-10-13 to 2026-10-14 on a macOS assessment host, and on 2026-10-15 the maintainers' remediation pass was re-verified on commit `147ab5e` for this version 1.1. The inputs were the complete tree with its vendored dependencies and test harness, and an earlier static review whose items were treated as hypotheses and re-verified against the tree.

The methodology was white-box. Every file under `src/` and `tests/` was read, the vendored crates were consulted wherever a finding depended on their behaviour, and the server was then exercised on loopback. Forty-six dynamic test groups were executed, A01 to A21 in the first test session, A23 to A28 as targeted re-runs during report preparation and A29 to A47 against the remediation pass, together with four static scans. Every process was stopped by its PID and only loopback services in the assigned port range were targeted; the few test inputs that made the server attempt outbound traffic are disclosed in Appendix A.

Coverage was high for every module of the server and for the dependency set. Two areas could be assessed statically only: the Linux-only io_uring relay backend, and the reachability of operator infrastructure from the exit's egress addresses, which depends on a deployment that is not in the tree.

Red Team Ltd identified 24 issues: 15 vulnerabilities with a demonstrated exploit path or impact (5 High, 9 Medium, 1 Low) and 9 miscellaneous issues (7 Low, 2 Info). The High items concern the product's core promise against a national censor and the availability of the node. The port-80 front answers any `Upgrade` request with a

non-standard 101 from any `Host`. The bytes that follow the upgrade identify every genuine session to a passive observer. Any client can read and rewrite another user's mailbox, entitlement view and protection rules by asserting that user's `cid`. The WebSocket path buffered unauthenticated data without limit, and MUX2 relays to a target that never reads pinned memory and descriptors beyond the carrier. Against that stand length-checked parsers that no network input crashed, a destination policy that held against nearly every form tried, correct verification of the signed protection bundle, and a passing test suite with a clean clippy run. The re-verification of `147ab5e` found 8 issues fixed, 9 partially fixed and 7 open. Both resource-exhaustion High items were addressed (RVPN-01-103 fixed, RVPN-01-113 in part) and the `cid` bearer token remains (RVPN-01-102); for the front and the wire image (RVPN-01-101, RVPN-01-114) the development team names the Proteus transport of WP2 as the replacement, with this server retained for legacy clients.

Scope fixes the target, threat model and exclusions; Test Methodology and Coverage describes how each area was examined; the two findings sections share one numbering sequence; the Conclusions walk through every area and prioritise remediation; Appendices A to D hold the test log, the dependency inventory, the ASVS checklist and the open questions.

Scope

The target is https://github.com/RostamVPN/redsocket_http, branch `main`. The reviewed state is commit `3012aa6` ↪ ; the follow-up commit `4430a1d` changes only test fixtures and one comment in `src/http.rs` and `src/proxy.rs`, and every `file:line` reference in the main text of the findings is to `4430a1d`. The working tree on the assessment host matched the index of `4430a1d` file for file (10,211 tracked files, none modified, none untracked outside `target/`), established by hashing the files rather than by running git. A release build of `3012aa6` identifies itself as `RedSocket HTTP-Standalone v3.3.0 (3012aa6 2026-10-15)`. For version 1.1, `main` stood at `147ab5e`: two fixture commits (`60cf688`, `f202f2d`), twenty remediation commits from `171aed6` to `3398920`, and the removal of the earlier static review. Its working tree again matched the index (10,212 files), and the notes under the findings and Appendix A from A29 onward refer to it.

The crate is `redsocket-http` 3.3.0 (edition 2021, `rust-version = "1.87"`, Apache-2.0) with the workspace member `e2e-harness` 0.1.0. The release profile sets `lto = "fat"`, `codegen-units = 1`, `panic = "abort"` and `strip = true` ↪ (`Cargo.toml:65-70`), and the global allocator is mimalloc (`src/main.rs:14-15`). `src/` holds 22 source files with 11,316 lines and four binaries under `src/bin/` with 1,942 lines: the server `redsocket-http` and the test clients `client`, `mux_client`, `legacy_client` and `flooder`. `tests/` holds six end-to-end test binaries and the shared harness (2,038 lines). The modules are `config`, `control`, `http`, `iouring_relay` (Linux) with `iouring_relay_stub`, `messaging`, `metrics`, `mux`, `noise`, `noise_framer`, `noise_proxy`, `noise_stream`, `protection`, `proxy`, `proxy_protocol`, `shed`, `stats`, `valkey`, `ws` and `ws_codec`. `Cargo.lock` resolves 263 packages: the two workspace members and 261 crates.io crates, all vendored under `vendor/` (Appendix B).

The threat model is the one that applies across RVPN-01. The primary adversary is a state-level censor that observes, fingerprints and blocks at national scale, actively probes suspected exits, operates hostile networks and man-in-the-middle positions, seizes devices, installs co-resident malicious applications, impersonates the configuration, bootstrap, relay and CDN endpoints, and can compromise distribution. For the exit server this means that every byte on port 80 is a potential fingerprint, that the server's static public key is in the adversary's hands because it ships in every client, that per-user records can reach the adversary through seizure of a node or of its back ends, and that the adversary can run any number of client sessions. The secondary adversary is the ordinary application attacker: a hostile tunnel client that wants to reach what it should not, impersonate other users, exhaust the node, or manipulate the records that drive billing and policy.

Excluded from WP1 are files outside the named repository, including the private deployment tree; the client applications and cores (WP3 to WP5); the cashier, the bundle publisher and the production Valkey instances; host hardening and service units, which were removed for publication; the correctness of vendored dependencies beyond the behaviour a finding depends on; and the memory safety of the `unsafe` io_uring backend, which cannot be built or run on the assessment host. No RostamVPN production endpoint was contacted.

Test Methodology and Coverage

Red Team Ltd followed NIST SP 800-115 for the structure of the work: planning on the basis of the tree and the earlier review, discovery by complete source reading, attack and verification by dynamic testing on loopback, and reporting against the standards listed at the end of this section. The assessment host was macOS 26.6.2 on Apple Silicon with rustc 1.91.1 and cargo 1.91.1 (the crate declares 1.87), clippy 0.1.91, cargo-audit 0.22.2 with the RustSec advisory database at commit `edb58f59` (2026-10-14), Python 3.15.0, OpenSSL 4.0.3, curl 8.7.1, pandoc 3.8.2.1, and the Homebrew `redis-server` 8.4.0 as a RESP-compatible stand-in for Valkey. Builds, scripts, captures and binaries were kept under `/tmp/rvpn01/wp1/`; the repository was not modified apart from this report, and no git command was run by the testers. The repository's `build.rs` itself runs `git rev-parse` and `git status --porcelain` to stamp the version string, so the builds of the first session invoked those two read-only commands, inside the repository for A01 to A03 and in the scratch copy, where they failed, for A05; the later re-runs A23 and A29 to A47 used a stub on `PATH` so that no git binary executed, and the history was read from the plain files under `.git/`. The version 1.1 re-verification used the same host and tools on 2026-10-15.

HTTP front and probing resistance. `src/http.rs` was read in full: the request parser, real-IP and `cid` extraction, the host gate and the three response paths (raw 101, WebSocket 101, 404 decoy). Test group A06 sent 33 request shapes to a server with the default configuration, to a server with `allowed_upgrade_hosts` set, and to a Python `http.server` reference, recording status line, header set, time to first byte and connection behaviour, then probed the post-101 behaviour with garbage and with silence. A27 recorded the bytes of a complete v1 and MUX2 session as a passive on-path observer would see them. These drive RVPN-01-101, RVPN-01-114 and RVPN-01-116.

PROXY protocol and client-address attribution. `src/proxy_protocol.rs` was read against the HAProxy PROXY protocol text (v1 line parsing, v2 binary parsing, `auto` detection). Test group A07 sent v1, v2, `UNKNOWN` and CDN-header requests to servers configured `disabled`, `auto` and with the key omitted, and correlated the server's attribution in its debug log through tagged `User-Agent` values; A24 measured the read deadlines before the HTTP parser in `auto` and `disabled` modes. These drive RVPN-01-105, RVPN-01-106, RVPN-01-108 and RVPN-01-116.

Noise NK session. `src/noise.rs`, `src/noise_framer.rs`, `src/noise_stream.rs` and `src/noise_proxy.rs` were checked against the Noise Protocol Framework revision 34: pattern `Noise_NK_25519_ChaChaPoly_BLAKE2s`, empty prologue, framing of a two-byte big-endian length plus at most 65,535 bytes of ciphertext, nonce handling by snow's transport state, absence of rekeying, and the key-rotation loop that builds a fresh responder per candidate key. Test group A08 sent ten malformed `msg1` shapes and A11f four malformed transport frames after a good handshake and four non-yamux payloads after a MUX2 header, watching for crashes and resource holds. The outcome feeds RVPN-01-108, RVPN-01-114, RVPN-01-124 and the positive observations.

v1 relay, MUX2 relay and UDP association. `src/noise_proxy.rs`, `src/mux.rs`, `src/proxy.rs` and `docs/udp-over-mux2.md` were read against RFC 1928 for the CONNECT request, the ten-byte reply and the UDP request header. Groups A10 and A11 drove the v1 path with the repository's `legacy_client` and with a scratch tool built from the project's own protocol code (`wp1tool`), including the in-tunnel control channel; A14 drove the UDP association with a loopback UDP echo target and a third-party sender; A17 ran the repository's SOCKS5 `client` binary in MUX2 mode end to end. These drive RVPN-01-109, RVPN-01-110 and RVPN-01-112.

Destination policy. `DEFAULT_DENY_CIDRS`, `is_unroutable`, the resolver chain and `connect_resolved` were traced. Group A10 attempted 20 destinations and four control calls through the default policy and four destinations through a server with `deny_cidrs = []`, covering loopback, link-local, RFC 1918, CGNAT, documentation, multicast, IPv4-mapped and IPv4-compatible IPv6, the NAT64 well-known prefix, decimal, hexadecimal and leading-zero numeric forms, `localhost` with and without a trailing dot, the host's own LAN address and the server's own metrics port; A11e added ATYP 4 literals on the MUX2 path and A14d a UDP target. These drive RVPN-01-117.

WebSocket path. `src/ws.rs` and `src/ws_codec.rs` were read against RFC 6455. Groups A09, A20 and A25 measured the server's resident set and socket states while connections declared 16 MiB frames, streamed `FIN=0` fragments, or stayed silent, before and after the connection cap. These drive RVPN-01-103, RVPN-01-107 and RVPN-01-116.

Connection lifetime, shedding and resource limits. The 600-second `tokio::time::timeout` wrapper, the relay task structure, `TunnelGuard`, the accept loop and the shedding sampler were traced, and the vendored tokio

JoinHandle semantics were checked. A18 ran a scratch build with the constant lowered to 20 seconds against v1, MUX2, WebSocket and idle connections; A13 ran the unmodified server under a 256-descriptor limit and opened 300 idle post-101 connections; A15 opened 256 MUX2 streams to a sink that never reads and sampled the resident set; A26 repeated that with 32 streams and observed what remained after the carrier closed; A19 ran a scratch build whose sampler reads a simulated memory figure and stepped it through the thresholds. These drive RVPN-01-107, RVPN-01-108, RVPN-01-113 and RVPN-01-121.

Messaging and protection. src/messaging.rs, src/protection.rs, src/valkey.rs and the six end-to-end tests were read in full; the dispatcher's Register, Ack and Rules work items, the per-stream rules gate, the broadcast cache, the bundle verification order and the judge were traced. A16 ran a server with both modules enabled, a bundle signed with the project's own protection::fixture and a loopback Redis as the mailbox store, and executed an impersonation chain with two clients sharing one cid, followed by forged and oversize broadcast entries written directly to the store; A28 tested a single-label user allow entry. These drive RVPN-01-102, RVPN-01-111, RVPN-01-115 and RVPN-01-120.

Statistics and logging. src/stats.rs, src/metrics.rs and the TSV row builders were read. A12 ran a server with [stats.valkey] pointed at a TLS Redis stand-in (self-signed CA generated for the test) and compared rows produced by plain HTTP requests carrying ?cid= and spoofed headers with rows produced by a genuine tunnel; A18a captured the usage row written at the cap; A11g injected a line feed through a destination name. These drive RVPN-01-104, RVPN-01-107 and RVPN-01-110.

Configuration, secrets and metrics. src/config.rs and src/main.rs were traced for defaults and start-up handling. A21 and the start-up logs verified the metrics loopback clamp. The remediation commit 4fb070c (manual Debug for valkey::Config that redacts the password, with a regression test; fixtures moved to documentation ranges) was compared with its parent d0d348b and verified in the tree and through the passing unit test debug_output_never_contains_the_password. These drive RVPN-01-105, RVPN-01-118, RVPN-01-119 and RVPN-01-124.

Build, tests, dependencies and publication hygiene. cargo build --offline --release (A01), cargo test --offline (A02), cargo clippy --offline --all-targets (A03) and cargo audit -n (A04) were run with the target directory in the scratch area, and the library tests were re-run on 4430a1d (A23). Scrub scans for live identifiers were run on 3012aa6 (S01) and 4430a1d (S02), and Cargo.lock was inventoried (S03). For version 1.1 the release build, the library tests with the two new end-to-end tests, the dependency audit and the scrub scan were repeated on 147ab5e (A29, A45, A46, S04). These drive RVPN-01-122 and RVPN-01-123.

Re-verification of the remediation pass (version 1.1). Each of the twenty remediation commits and the two fixture commits before them was compared with its parent by reading the repository's objects directly, and every claimed change was traced to the code it touches. The release binary of 147ab5e, which identifies itself as RedSocket HTTP-Standalone v3.3.0 (unknown 2026-10-15) because the git stub withholds the commit, and a scratch build of it with the cap lowered to 20 seconds were then run against the shapes that had demonstrated each issue, using the tooling of A01 to A28 and five further helpers: a PROXY header sender, a port holder without SO_REUSEPORT, an IPv6 UDP echo, a relay that keeps the server side of a connection open and silent after the server's close, and a relay that carries a client's byte stream over the WebSocket path, either as complete frames or as one message whose FIN bit is never set. Where the version 1.0 evidence had been static or had come from a different shape, the same test was also run against the 3012aa6 build to show that it discriminates (A32c, A32d, A43b). Issues without a remediation commit were re-checked statically and, where a test existed, by repeating it.

The dynamic test groups are summarised below; Appendix A holds the commands and trimmed outputs.

Group	Subject	Server configuration	Result
A01-A05	build, tests, clippy, audit, scratch builds	n/a	build ok; 118 tests pass, 2 ignored; clippy clean; 4 advisories
A06	probing oracle, 33 request shapes	default; host gate set; reference	101 for any Upgrade from any Host; gate works
A07	PROXY v1/v2 and CDN headers, 25 cases	disabled; auto; key omitted	headers trusted from any peer; omitted key means auto

Group	Subject	Server configuration	Result
A08, A11f	malformed msg1, transport and yamux inputs, 18 cases	default	no crash; idle holds persist
A09, A20	WebSocket fragment growth	default; cap 20 s	512 MiB sent, 923.6 MiB resident; growth after the cap
A10, A11e, A14d	destination policy and CONNECT edge cases, 35 cases	default; deny off	policy holds except IPv4-embedding IPv6 forms
A11a-d, g	control channel, JSON and log injection	default; deny off	node figures and banner disclosed; injection confirmed
A12	statistics rows via TLS Redis	stats on	presence rows forged from plain HTTP
A13	300 idle post-101 connections, 256 descriptors	default	process exited: too many open files
A14a-c	UDP association	deny off	third-party datagrams delivered
A15	256 streams to a non-reading sink	deny off	201,326,592 B accepted, +92.5 MiB resident
A16	messaging impersonation chain	messaging and protection on	disclosure, acknowledgement, rules poisoning
A17	repository client binary end to end	deny off	TCP and UDP through the tunnel work
A18	connection cap	cap 20 s, stats on	v1 relay outlives the cap; usage row at the cap
A19	shedding thresholds	simulated memory	decoy bytes identical; hysteresis works
A21	metrics bind exposure	default; public bind requested	loopback only; clamp works
A23	library tests on 4430a1d	n/a	110 pass, 1 ignored
A24	read deadline before HTTP, auto vs disabled	auto; disabled	auto: no deadline; disabled: 400 at 5.003 s
A25	silent WebSocket client across the cap	cap 20 s	socket held in FIN_WAIT_2 after the cap
A26	32 stalled MUX2 streams, then carrier closed	deny off	32 relays and sockets still held 60 s later
A27	passive view of a v1 and a MUX2 session	default	fixed 57-byte 101, 50-byte first records
A28	single-label user allow entry	messaging and protection on	accepted; listed domain cleared
A29	release builds of `147ab5e` and of a 20 s cap copy	n/a	build ok
A30	A06 request shapes; silence after the 101	default	101 for any Upgrade; silence closed at 10.0 s
A31	spoofed, `UNKNOWN` and v2 `LOCAL` PROXY headers; silence	key omitted; auto; required	header trusted only in required; 5 s deadline
A32	cap: v1 relay; silent WebSocket and MUX2 clients	cap 20 s; `3012aa6` comparison	ended at the cap; old build held `FIN_WAIT_2`
A33	300 idle connections at 256 descriptors; occupied extra bind	default	server survived; failed bind stopped start-up
A34	`_rs.stat`, `_rs.config`, unknown subcommand	default	`cid` and key only; 404 line for unknown
A35	line feed in a name and in header fields; `info` log	default	name refused; header fields unescaped at debug
A36	forged and oversize broadcast entries	messaging and protection on	oversize skipped and counted; forgery delivered

Group	Subject	Server configuration	Result
A37	UDP datagrams from another and from the same address	deny off	other address dropped; same address delivered
A38	32 stalled MUX2 streams; carrier closed, then held 75 s	deny off	released with the carrier; 60 s write timeout
A39	single-label and public-suffix user rules	messaging and protection on	label refused; `co.uk` accepted
A40	IPv4-embedding IPv6 forms of loopback	default	refused
A41	placeholder, former example and command-line keys	n/a	refused; upper-case form and argv accepted
A42	malformed msg1, 10 cases	default	counted once each
A43	WebSocket growth shapes; MUX2 inside one unfinished message	default; deny off	flat resident set; streamed without `FIN`
A44	statistics rows via TLS Redis	stats on	fields cut to 64; plain-HTTP row still created
A45	library and new end-to-end tests on `147ab5e`	n/a	127 pass, 1 ignored; both new tests pass
A46	`cargo audit -n` on `147ab5e`	n/a	same 4 advisories and warning
A47	metrics after start, refusals and one bad msg1	default	uptime from start; failures counted once

The standards and specifications applied were OWASP ASVS 5.0.0 (30 May 2025) as the checklist for Appendix C and for the control references, OWASP Top 10:2025 for category mapping where a web or API surface is concerned, CWE for every finding and CVSS v4.0 (FIRST, November 2023) vectors, NIST SP 800-115 for the testing approach, NIST SP 800-218 SSDF v1.1, SLSA v1.0, CycloneDX 1.7 and SPDX 3.0.1 for the supply-chain observations, and Regulation (EU) 2024/2847 (Cyber Resilience Act) for the vulnerability-handling remarks in the Conclusions. The protocol references were the Noise Protocol Framework revision 34 (Noise NK), RFC 9110 and RFC 9112 for the HTTP front, RFC 6455 for the WebSocket path, the current HAProxy PROXY protocol text, RFC 1928 for the SOCKS5-derived headers, RFC 9293 for TCP behaviour, RFC 8446 for the TLS client, and RFC 5737 and RFC 3849 for the documentation addresses used throughout this report.

Identified Vulnerabilities

RVPN-01-101 WP1: HTTP front answers any Upgrade request with a non-standard 101 from any Host (High)

Client Note: The development team's remediation for the carrier's signature and the active-probing oracle is the Proteus transport (redsocket_proteus, WP2), which replaces the HTTP front; this server is retained for legacy clients.

Note: Unchanged in `147ab5e`: A30 again received a 101 for every `Upgrade` shape tried.

The HTTP front is the only part of the exit a censor sees before the Noise session starts, so every response it gives to a non-client is a potential fingerprint. `handle_http_connection` treats a request as an upgrade as soon as it carries an `Upgrade` header of any value; it checks neither `Connection: Upgrade`, the method, the path nor `Sec-WebSocket-Version` (`src/http.rs:470-472`), and the parser accepts any first line with two whitespace-separated tokens (`src/http.rs:25-28`). The only gate is `allowed_upgrade_hosts`, which is empty by default and then admits every `Host`, including none (`src/http.rs:377-384`, `src/config.rs:221-225`; the test `the_host_gate_defaults_to_empty_which_is_fail_open` pins that default). The raw-path 101 carries only `Connection : Upgrade` and omits the `Upgrade` header that RFC 9110 section 15.2.2 requires (`src/http.rs:542-544`). The negative responses are fixed byte strings without `Date` or `Server`, followed by a close (`src/http.rs:442`, `:455`, `:572`; `src/shed.rs:43-47`), and after a 101 the server waits in silence for a two-byte length.

Affected: src/http.rs:20–45 (HttpRequest::parse), :377–384 (upgrade_host_allowed), :470–472, :508–516 (WebSocket 101), :542–544 (raw 101), :572 (404 decoy); src/config.rs:221–225.

Proof of Concept: Test A06a against the default configuration (server A, 127.0.0.1:20100):

```
GET / HTTP/1.1
Host: anything.invalid
Upgrade: foo

HTTP/1.1 101 Switching Protocols
Connection: Upgrade
```

The same 101 was returned for Upgrade: h2c, for POST /x, for a request without Host and (A06d) for a request whose first line was a PROXY protocol v1 header. Sec-WebSocket-Key: garbage with Sec-WebSocket-Version: 7 received a 101 with a computed Sec-WebSocket-Accept where RFC 6455 sections 4.2.1 and 4.4 call for a 400 or a 426. A plain GET / received the dateless 404 Not Found with the body Not Found and an immediate close; a TLS ClientHello received 400 Bad Request after 5.003 seconds and a TCP reset, as did a 17,036-byte header block. The Python reference server (A06c) sent Server and Date with every answer and did not answer the ClientHello within seven seconds. With allowed_upgrade_hosts = ["front.example"] (A06b) wrong and missing hosts received the 404 and the right host the 101, but a request with two Host headers was judged on the first.

Impact: The primary adversary confirms an exit with one unauthenticated request per address, without the Noise key or any client secret, and blocks it. The host gate only defeats blind scanning, because the Host value travels in cleartext in every genuine session (RVPN-01-114). The five-second 400 with a reset and the post-101 silence are further discriminators, and the public key in every client lets the adversary call _rs.config (RVPN-01-109) for a definitive banner.

Preconditions: Network reachability of port 80. The default configuration is affected; the host gate reduces the exposure to passive observation of one genuine session.

Recommendation: Require a client proof in the upgrade request that a prober cannot derive from the public key, for example an HMAC over a coarse timestamp keyed with a per-deployment secret, and answer everything else exactly as a chosen mainstream server would, from status line and header order to Date, Server, body and timeout behaviour. Refuse to start with an empty allowed_upgrade_hosts unless allow_any_host = true is set, and make the positive path conformant (RVPN-01-116).

Classification: CWE-203 · OWASP ASVS 5.0.0 V13.4 (Unintended Information Leakage), 4.1.4 · CVSS:4.0/AV:N/AC:L/AT:N/PF

RVPN-01-102 WP1: Client-asserted cid is a bearer token for another user's mailbox, rules and entitlement (High)

Note: Unchanged in 147ab5e: no remediation commit touched cid handling, and in A36 a new session asserting a cid was replayed the rules message that an earlier session with that cid had written (A39).

The MUX2 header is split on | and its cid field is taken as the client identifier (src/mux.rs:38–56). The identifier is validated for shape only, 16 to 64 lowercase hexadecimal characters (src/mux.rs:194–199), and nothing binds it to the Noise session, which authenticates the server and not the client. With [messaging] enabled, run_control_stream registers the stream under the claimed cid once the client's first upstream frame arrives, and the dispatcher reads msg:<cid>:last, msg:<cid>, rules:<cid> and cid:<cid> and replays the unread notices and the entitlement record to the caller (src/messaging.rs:795–828, :1129–1189). {"ack":N} advances msg:<cid>:last for any sequence number already sent to that stream (:1204–1209, :829–848), and a valid {"rules":...} frame is written to rules:<cid>, appended to the mailbox and published (:1210–1226, :849–899). Every live control stream of the same cid applies a rules mailbox message to its tunnel context (:1231–1241) and later registrations load the stored rules (:1178). Rule entries are validated as single destinations, but a single label passes that check and matches every name beneath it (RVPN-01-115).

Affected: src/mux.rs:38–63, :96–121, :194–199; src/messaging.rs:795–899, :1112–1267; src/protection.rs:619–641, :686–693, :921–943.

Proof of Concept: Test A16 against server D (both modules on, loopback Redis as the store, a fixture bundle listing evil.example, a phishing range containing 203.0.113.5 and a controller range containing 198.51.100.7).

The victim `cid` was seeded with two unread notices, a ledger record `state=paid tier=pro` and the rule `allow=["mybank.example"]`; a second client then announced the same `cid`:

```
mux -> header: MUX2|Asia/Tehran|rostam|android|c1d00000000000aa||blk1,msg1
ctl -> {"since":0}
ctl <- {"seq":1,...,"type":"notice","payload":{"title":"private notice one"},"scope":"cid"}
ctl <- {"seq":2,...,"type":"notice","payload":{"title":"private notice two"},"scope":"cid"}
ctl <- {...,"type":"entitlement","payload":{"paid_until":"2027-01-01T00:00:00","state":"paid","tier":"pro
    ↪ "},...}
ctl -> {"ack":2}
ctl -> {"rules":{"allow":["evil.example"],"deny":[],"categories_off":65535}}
```

Afterwards the store held `msg:<cid>:last = 2` and the forged rules, and the victim's open control stream had received the forged `rules` frame. A victim client that reconnected, sent `{"since":0}` and opened its streams 1.5 seconds later obtained CLEAR for `evil.example:443` and 203.0.113.5:80 instead of BLOCK; 198.51.100.7:80 stayed BLOCK (`controller` floor). Streams opened before registration completed were still judged without the stored rules (A16 V2), and restoring the seeded rules restored BLOCK (A16c). A28 showed that the single label `example` was accepted as an allow entry and cleared `evil.example`.

Impact: Any client that knows a `cid` reads that user's unread notices and entitlement, acknowledges the notices away so the user never sees them, and persistently weakens the user's exit-side protection, including whole name spaces through single-label entries. Identifiers travel in the cleartext `?cid=` query of legacy clients (RVPN-01-110) and sit in the statistics store and on devices, so the primary adversary has several ways to obtain them. The attack costs one tunnel per victim and leaves no distinguishing trace.

Preconditions: `[messaging]` enabled on the exit and knowledge of the victim's `cid`.

Recommendation: Give the server a verifiable client identity before any mailbox or rules operation, either a per-`cid` secret issued by the cashier and proven with an HMAC over the Noise handshake hash, or a client static key registered against the `cid` (Noise XK or IK), behind a new capability. Until that proof exists, do not enable `[messaging]` on a public exit, treat upstream `rules` and `ack` as advisory for the current tunnel only, and never replay a mailbox to an unproven session.

Classification: CWE-639 · OWASP ASVS 5.0.0 8.2.2, 8.3.1; OWASP Top 10:2025 A01 · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/

RVPN-01-103 WP1: Unbounded WebSocket fragment reassembly before any handshake, continuing after the cap (High)

Fix Note: This issue has been fixed by the development team in commit `41078d3` and verified by Red Team Ltd to be working as expected. On `147ab5e` the A09 shapes left the resident set at 12.2 MiB where 923.6 MiB had been reached, the A20 shape was reset as soon as it streamed after the server's close (A43), and a MUX2 session carried inside one WebSocket message that never set `FIN` completed and relayed 25 MB through the 16 KiB pipe, which `3012aa6` could not complete (A43b).

A request that carries `Upgrade` and `Sec-WebSocket-Key` is answered with a 101 and handed to `ws::wrap`, which spawns a detached reader task (`src/http.rs:472-477`, `:501-538`; `src/ws.rs:19-66`). Each frame may declare up to 16 MiB (`MAX_FRAME`), and `BINARY` and `CONT` payloads are appended to a reassembly vector that is flushed only when a frame has `FIN` set; nothing bounds the reassembled message (`src/ws.rs:15`, `:53-61`). While `FIN` is clear the reader never waits on its consumer, so the server takes data into memory at line rate before a single byte of Noise has been checked. The reader owns the TCP read half and continues after the connection cap for as long as the client sends or keeps its side open. The decoder of the opt-in io_uring backend, `ws_codec::WsDecoder`, reassembles without a bound in the same way (`src/ws_codec.rs:100-105`; static only).

Affected: `src/ws.rs:15-16`, `:29-66`, `:108-159`; `src/http.rs:472-477`, `:501-538`; `src/ws_codec.rs:100-105`.

Proof of Concept: Described by parameters and measured outcome. A09b, default configuration: four WebSocket-upgraded connections sent 512 MiB in total as masked 1 MiB `BINARY`/`CONT` fragments with `FIN` never set, without any Noise handshake. The server's resident set rose from 11.0 MiB to 923.6 MiB within 1.1 seconds of the first connection (including a one-second settle pause), a new connection was still answered, and the resident set was still 923.6 MiB two seconds after all four connections had been closed. Merely declaring 16 MiB frames on eight connections added 1.2 MiB (A09a). A20, on the build with the cap lowered to 20 seconds: one

connection received the server's FIN at t+20.0 s and then streamed 200 MiB more, all of which was accepted; the resident set rose from 14.6 MiB to 471.8 MiB with the server-side socket in `FIN_WAIT_2`. A25: a silent client kept its server-side socket in `FIN_WAIT_2` from the cap until it closed 25 seconds later.

Impact: An unauthenticated remote party controls the node's resident memory at the rate of its own uplink, and the cap does not end the growth. Under `panic = "abort"` an allocation failure, or the out-of-memory killer, ends every tunnel on the node; before that, load shedding refuses new users while the attacker's connections keep growing. The repository's clients send `Upgrade: websocket` without a key and never use this path.

Preconditions: Network reachability of port 80. No key and no handshake are needed.

Recommendation: Remove the WebSocket framing path if no shipped client needs it. Otherwise bound the reassembled message to the Noise frame ceiling (65,537 bytes covers one length-prefixed Noise frame), reject longer `FIN=0` sequences, read payloads incrementally instead of allocating from the declared length, end the pumps through a cancellation token tied to the connection future, and count WebSocket bytes against a per-connection budget before the handshake completes. Apply the same bound in `ws_codec`.

Classification: CWE-770 · OWASP ASVS 5.0.0 15.2.2 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:

RVPN-01-104 WP1: Presence and usage records keyed on an unauthenticated cid; presence rows forged from plain HTTP (Medium)

Note: Partially fixed in commit `3398920` and verified by Red Team Ltd (A44): client-supplied row fields are cut to 64 characters, `sni` to 253, and a 300-character `tz` was stored as 64. Not yet addressed: a non-upgrade request with `?cid=` still creates a presence row (`src/http.rs:557–561`), which A44 attributed to a forged `X-Forwarded-For` address (RVPN-01-106), and rows remain keyed on the asserted `cid` (RVPN-01-102) without a per-peer rate limit.

When a stats connection is configured, every request creates a `stats::Capture`, and for a non-upgrade request whose path contains a `cid=` value of 1 to 64 characters of `[A–Za–z0–9–]` the presence row is emitted at once, before any handshake (`src/http.rs:219–228`, `:468`, `:478–487`; `src/stats.rs:55–87`). The row carries the client address of RVPN-01-106 and has the same shape as the row a legacy client produces at tunnel open (`src/noise_proxy.rs:161–163`). Usage rows at tunnel close carry whichever `cid` the client asserted (`src/metrics.rs:238–253`, `src/stats.rs:149–167`), and nothing marks an asserted identity as such. The `tz`, `app`, `os`, `cid` and `sni` fields of META and MUX2 headers are neutralised for tabs and line breaks (`src/stats.rs:121–129`) but not bounded in length, so a single row can carry about 64 KiB, and the outbound queue is bounded by count (200,000 rows) rather than by bytes (`src/stats.rs:89–91`, `:107–112`; static only).

Affected: `src/http.rs:219–228`, `:468`, `:478–487`; `src/stats.rs:49–91`, `:140–167`; `src/metrics.rs:238–253`; `src/noise_proxy.rs:127–133`, `:161–169`.

Proof of Concept: Test A12 against server E with `[stats.valkey]` pointed at a TLS Redis stand-in:

```
curl –H "X–Forwarded–For: 203.0.113.9" "http://127.0.0.1:20110/?cid=abcdef0123456789"
HTTP/1.1 404 Not Found
```

The request produced the `stats:cid` entry `2026–10–15T02:18:30Z 203.0.113.9 abcdef0123456789 (four empty columns)tunnel <host>`. Two hundred such requests with distinct `cid` values and addresses took 1.2 seconds and added 200 rows (`LLEN stats:cid` was 202 afterwards). A genuine `legacy_client` session produced `... 127.0.0.1 1111222233334444 ... tunnel <host>`, the same shape. That session's destination was refused by policy and no `stats:tunnel–closed` row followed, which shows that presence is recorded before a destination is accepted.

Impact: Anyone who can reach port 80 seeds the statistics with presence records placing any `cid` at any address and time, at HTTP line rate and without a handshake, and any tunnel client attributes its usage to any `cid`. Billing, abuse handling and analytics cannot tell forged rows from genuine ones. Unbounded field lengths combined with a row-counted queue let a client that outpaces the stats writer, or a stats outage, grow the queue towards roughly 13 GB in the worst case (static estimate).

Preconditions: Reachability of port 80 and a stats connection configured on the exit.

Recommendation: Do no work on the non-upgrade path beyond sending the response and drop the URL `cid` entirely; emit presence rows only after a successful destination connect; carry a server-generated session identifier in every row and mark the `cid` as asserted until RVPN-01-102 provides a proof; truncate each header field at ingestion (64 bytes for `tz`, `app`, `os` and `cid`, 253 for `sni`) and bound the queue in bytes; rate-limit rows per peer address.

Classification: CWE-345 · OWASP ASVS 5.0.0 2.4.1, 15.3.4 · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:L/VA:N/SC:N/SI:

RVPN-01-105 WP1: proxy_protocol silently becomes auto when the [http] table omits the key (Medium)

Fix Note: This issue has been fixed by the development team in commits `45ee847` and `cbce7d0` and verified by Red Team Ltd to be working as expected. In A31 an `[http]` table without the key started in `Disabled` mode and recorded the socket peer for a spoofed PROXY line, `auto` stripped the header and recorded the peer, and a silent PROXY phase was closed after 5 seconds in `auto` and `required` mode, as was a partial one in `auto`.

Note: Operators should note that an `[http]` table without `proxy_protocol` now means `disabled` rather than `auto`.

Note: Operators should note that `auto` no longer records the address carried in a PROXY header, so a node whose load balancer must supply client addresses needs `required`, which trusts any peer that can reach the port.

`HttpConfig::proxy_protocol` carries `#[serde(default)]`, which resolves to `PpMode::default()`, and the enum's `#[default]` variant is `Auto` (`src/config.rs:215–216`; `src/proxy_protocol.rs:15–21`; the project's own test `the_default_mode_is_auto` pins it). The struct-level `Default` that sets `Disabled` (`src/config.rs:228–239`) applies only when the whole `[http]` table is absent. Unknown keys are ignored by design (`src/config.rs:395–426`), so a misspelled key is silent as well. In `Auto` the parser honours a v1 or v2 header from any peer, with no list of trusted sources (`src/proxy_protocol.rs:39–64`), and the PROXY phase has no read deadline at all because it runs before the 600-second timeout wrapper and before the five-second header deadline (`src/http.rs:345–363`).

Affected: `src/config.rs:213–216`, `:228–239`; `src/proxy_protocol.rs:15–21`, `:39–91`; `src/http.rs:345–363`.

Proof of Concept: Server F was started from `config.example.toml` with the single line `proxy_protocol = "disabled"` deleted; its start-up banner printed `proxy_protocol: Auto`. In test A07c it attributed the request below to the address in the PROXY line, and a v2 header the same way:

```
PROXY TCP4 203.0.113.7 198.51.100.1 51234 80
GET /?cid=wp1spoofF-v1 HTTP/1.1
Upgrade: websocket

[HTTP] GET /?cid=wp1spoofF-v1 from edge=203.0.113.7 real_ip=203.0.113.7 ...
```

Server A (`disabled`) attributed the same bytes to the socket peer. In `auto` mode (server C) `PROXY UNKNOWN` was accepted and yielded `0.0.0.0`. A24 compared the read deadlines: a connection that sent nothing received `400 Bad Request` after 5.003 seconds from a `disabled` server, but no response and no close within 20 seconds from an `auto` server, and the same held after the six bytes `PROXY`.

Impact: On any node that does not sit behind a PROXY-emitting load balancer but whose configuration has an `[http]` table without the key, every client chooses the address the server believes it has: statistics rows, log lines and the sticky egress selection for clients without a `cid`. Combined with RVPN-01-104 this forges presence records at the transport layer. The missing deadline turns every idle connection into an indefinite hold, which feeds RVPN-01-108.

Preconditions: An `[http]` table without `proxy_protocol`, a misspelled key, or `auto` chosen deliberately.

Recommendation: Make `Disabled` the enum default (move the `#[default]` attribute or use a dedicated default function), require a `proxy_protocol_trusted_cidrs` list before `auto` or `required` take effect and check the socket peer against it, put the PROXY phase under the same short deadline as the header read, and either deny unknown fields on `HttpConfig` or log every ignored key at start-up. The PROXY protocol specification also forbids guessing whether a header is present (RVPN-01-116).

Classification: CWE-1188 · OWASP ASVS 5.0.0 4.1.3, 15.3.4; OWASP Top 10:2025 A02 · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:

RVPN-01-106 WP1: CDN real-IP headers trusted from any peer (Medium)

Note: Unchanged in `147ab5e`: in A44 a direct peer's `X-Forwarded-For` value was again recorded as the client address of a statistics row.

`extract_real_ip` honours `CloudFront-Viewer-Address`, `True-Client-IP`, `CF-Connecting-IP`, `X-Real-IP` and the first element of `X-Forwarded-For` from any connection (`src/http.rs:61-96`). The code comment calls the first "unspoofable when origin is locked to CF", but nothing in the code checks that the peer is a CDN. The values are parsed as `IpAddr` before use, so the issue is attribution rather than injection, and the parser splits headers on CRLF only, which the project tests against bare-LF smuggling (`src/http.rs:172-194`). The resulting address is what the statistics rows (`src/stats.rs:76-87`), the log lines (`src/noise_proxy.rs:174-179`, `src/mux.rs:45-53`) and the sticky-egress key of clients without a `cid` (`src/noise_proxy.rs:185`, `src/mux.rs:260`) use.

Affected: `src/http.rs:61-96`, `:461-462`; consumers `src/stats.rs:76-87`, `src/noise_proxy.rs:185`, `src/mux.rs:45-53`, `:260`.

Proof of Concept: Test A07a against server A (`proxy_protocol = "disabled"`). An upgrade request with `X-Forwarded-For: 203.0.113.9, 198.51.100.2` was logged as `real_ip=203.0.113.9`:

```
[HTTP] GET /?cid=wp1spoofA-xff from edge=127.0.0.1 real_ip=203.0.113.9 ... xff="203.0.113.9,
    ↪ 198.51.100.2"
```

`X-Real-IP`, `CF-Connecting-IP`, `CloudFront-Viewer-Address: 203.0.113.12:443` and `CloudFront-Viewer-Address: [2001:db8::12]:443` were each honoured; `X-Forwarded-For: not-an-ip` fell through to the socket peer. A non-upgrade request with a spoofed header was attributed the same way, and the forged presence row of RVPN-01-104 used this path.

Impact: Every record that carries a client address is attacker-chosen, so the statistics, the logs and any abuse handling built on them can be steered or poisoned, and a client without a `cid` selects its egress address from the pool by varying the header.

Preconditions: Reachability of port 80.

Recommendation: Honour CDN headers only when the socket peer, after PROXY processing, lies inside a configured `trusted_proxy_cidrs` list, and accept the one header that the deployment's CDN actually sets rather than a priority list; otherwise use the peer address.

Classification: CWE-348 · OWASP ASVS 5.0.0 4.1.3, 15.3.4 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:L/VA:N/SC:N/SI:

RVPN-01-107 WP1: The 600-second connection cap ends neither v1 relays nor WebSocket sockets, and usage is recorded at the cap (Medium)

Fix Note: This issue has been fixed by the development team in commits `9bb9add`, `10d8942` and `41078d3` and verified by Red Team Ltd to be working as expected. On a `147ab5e` build with the cap lowered to 20 seconds (A32) the v1 relay ended at the cap, the later send failed and both gauges fell to zero; the sockets of a MUX2 client and of a WebSocket-path client that stayed silent after the cap were released at once, while the same tests against `3012aa6` left them in `FIN_WAIT_2`.

Each connection future is wrapped in `tokio::time::timeout(600 s)` (`src/http.rs:353-363`). On the v1 path the two relay directions run in spawned tasks and the future keeps only their `JoinHandle`s and the `TunnelGuard` (`src/noise_proxy.rs:252-338`). Dropping a `JoinHandle` detaches its task (`vendor/tokio/src/runtime/task/join.rs:18-20`, `:35-36`), so when the timeout fires the relay continues while `TunnelGuard::drop` decrements `active_tunnels` and emits the usage row with the bytes counted so far (`src/metrics.rs:238-253`). On the WebSocket path the reader task keeps the socket (RVPN-01-103). On the MUX2 path dropping the yamux connection ends the carrier, but the `noise_stream` decryption pump still holds the read half until the client sends or closes (`src/noise_stream.rs:28-59`; static), and stream relays blocked on a non-reading target outlive the carrier (RVPN-01-113). The README does not say what the cap is meant to bound.

Affected: `src/http.rs:353-363`; `src/noise_proxy.rs:252-338`; `src/metrics.rs:238-253`; `src/noise_stream.rs:28-59`; `src/ws.rs:29-66`.

Proof of Concept: Test A18 on the scratch build with the constant lowered to 20 seconds and statistics enabled. A v1 tunnel to a loopback echo target exchanged `hello`, idled for 30 seconds and then sent `after-cap`, which was echoed at t+32 s:

```
[02:27:35.608] v1 -> after-hold payload (9 B): after-cap
[02:27:35.611] v1 <- 9 B plaintext: after-cap
```

Meanwhile `redsocket_active_tunnels` fell from 1 to 0 at the cap, the server logged `Connection timeout (20s-PATCHED)`, and the usage row recorded 19 s, 5 bytes up and 12 bytes down. Over MUX2 the same sequence ended at t+20 s, a silent post-101 connection was closed at 20.0 s, and a silent WebSocket client's socket stayed in `FIN_WAIT_2` after the cap until the client closed it (A25).

Impact: A v1 client that keeps a tunnel open past ten minutes is thereafter unmetered and uncounted while it continues to relay, which breaks usage accounting and the `active_tunnels` gauge that shedding and capacity decisions rely on. Silent clients on the WebSocket path, and potentially on MUX2, hold descriptors for as long as they like. The cap is also inconsistent between carriers: MUX2 users lose every connection after ten minutes while v1 users do not.

Preconditions: A v1 tunnel older than the cap, or a WebSocket-path client that stays connected.

Recommendation: Decide what the cap means. If tunnels may live longer, apply the timeout to the pre-relay phases only and emit the usage row when the relay actually ends. If they may not, abort the relay tasks on cancellation by holding their `AbortHandle`s in a guard whose `Drop` aborts them, or drive the relay with `select!` over the futures directly, and give the WebSocket and `noise_stream` pumps a cancellation signal tied to the connection.

Classification: CWE-404 · OWASP ASVS 5.0.0 15.2.2, 13.1.3 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:L/VA:L/SC:N/S

RVPN-01-108 WP1: An accept() error terminates the process, and idle connections are free to hold (Medium)

Fix Note: This issue has been fixed by the development team in commits `7594504` and `a16b42a` and verified by Red Team Ltd to be working as expected. Under `ulimit -n 256`, 300 idle connections produced ten `Too many open files` accept errors that were retried after one second while the server kept serving, and an extra bind on an occupied port stopped start-up (A33); post-101 silence was closed after 10 seconds (A30) and a silent PROXY phase after 5 (A31). The recommended per-peer and global connection ceilings were not added.

The accept loop propagates every error from `listener.accept()` with `?` (`src/http.rs:316-317`); `http::run` returns it, `main` returns it, and the process exits (`src/main.rs:276-284`). Tokio retries only `WouldBlock`, so `EMFILE` and `ENFILE` end the service and every tunnel on it. Listeners in `extra_binds` fail individually and only log (`src/main.rs:262-275`), and the metrics listener stops on its first accept error (`src/main.rs:293-294`). Reaching descriptor pressure is cheap: a connection that has received its 101 and sends nothing is held for the full 600-second cap because the handshake read has no deadline of its own (`src/noise_proxy.rs:54-56`); in PROXY `auto` and `required` modes a connection that sends nothing is held without any deadline (RVPN-01-105); and descriptors pinned by stalled MUX2 relays survive the carrier (RVPN-01-113). The TCP keepalive (`src/http.rs:330-336`) only detects peers that are gone.

Affected: `src/http.rs:316-317`; `src/main.rs:262-284`, `:289-316`; `src/noise_proxy.rs:54-56`.

Proof of Concept: Described by parameters and outcome. Test A13 started an unmodified server with a descriptor limit of 256 and opened 300 connections, each completing the upgrade and then sending nothing. 245 connections received a 101; the 246th was reset and every later attempt was refused. The server's last log line was `Error: Too many open files (os error 24)`, the process had exited, and all 245 established connections had been closed. A08 showed a partial `msg1` and a silent post-101 connection both still open at the end of a two-second read window.

Impact: An attacker who holds enough sockets, or who times a burst to coincide with legitimate load, drops every user on the node at once. Shedding does not help because `accept()` runs before the shed check. The effort depends on the production descriptor limit, which is not in the tree (Appendix D); every attacker connection costs the node a descriptor and a task for ten minutes or, with RVPN-01-105 and RVPN-01-113, indefinitely.

Preconditions: Enough held connections or pinned descriptors to reach the service's limit, or a transient `accept` error.

Recommendation: On an `accept()` error, log at warn level, sleep briefly and continue; treat `EMFILE` and `ENFILE` as a shedding signal; keep a reserve descriptor; add per-peer and global connection ceilings below the descriptor limit; give the Noise handshake and the PROXY phase their own short deadlines; and set `LimitNOFILE` explicitly in the service unit.

Classification: CWE-755 · OWASP ASVS 5.0.0 16.5.4, 15.2.2 · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/S

RVPN-01-109 WP1: In-tunnel control channel discloses node-wide load and a product banner to any key holder (Medium)

Fix Note: This issue has been fixed by the development team in commit `c5037bf` and verified by Red Team Ltd to be working as expected. In A34 `_rs.stat` returned only the `cid`, JSON-encoded with the injected quotes escaped, and `_rs.config` only the Noise public key. An unknown subcommand is still answered with an HTTP-style 404 line that names it rather than with silence, which Red Team Ltd considers hardening only.

Note: Operators should note that `_rs.stat` and `_rs.config` no longer carry the node's counters or banner; the counters remain on the loopback metrics endpoint.

A CONNECT to the pseudo-host `_rs.stat` returns `active_tunnels`, `total_tunnels`, `bytes_up` and `bytes_down` for the whole node, and `_rs.config` returns a JSON document naming the product, `"server":"redsocket-http"`, together with the server public key (`src/control.rs:41–83`). The project deliberately clamps the Prometheus endpoint to loopback because "these counters describe tunnel volume and node load on a port reachable from Iran" (`src/main.rs:127–134`), yet the control channel hands the same figures to every tunnel client, and every holder of the public key that ships in the app can be a tunnel client. The `cid` is interpolated into the `_rs.stat` reply without escaping (`src/control.rs:70–77`), which malforms only the caller's own reply, and an unknown subcommand is answered with a product-specific line (`src/control.rs:32–36`).

Affected: `src/control.rs:24–83`; callers `src/noise_proxy.rs:173–183`, `src/mux.rs:292–302`.

Proof of Concept: Test A11 on the v1 path: a `_rs.stat` call after a META frame whose `cid` field was `evilcid","injected":true,"x":"` (A11a), and a `_rs.config` call on a second tunnel (A11b):

```
A11a v1 -> first frame (13 B): \x05\x03\x08_rs.stat\x00\x01
A11a v1 <- {"cid":"evilcid","injected":true,"x":"","active_tunnels":0,"total_tunnels":0,"bytes_up":0,"
    ↪ bytes_down":0}
A11b v1 <- {"noise_pubkey":"<server public key>","carriers":["http"],"server":"redsocket-http"}
```

Both calls also succeeded on the MUX2 path after the ten-byte reply (A11c), and `_rs.bogus` was answered with `HTTP/1.1 404 unknown _rs subcommand: bogus` (A10a).

Impact: A censor completes the handshake, calls `_rs.config` for a definitive banner that confirms what RVPN-01-101 and RVPN-01-114 indicate, and calls `_rs.stat` to rank exits by user count for blocking priority; a competitor or researcher measures usage per node over time.

Preconditions: The server public key, which every client carries.

Recommendation: Return per-session figures only, or nothing, from `_rs.stat`; drop `"server"` from `_rs.config` or replace it with a capability list that does not name the product; JSON-encode the `cid`; answer unknown subcommands with silence; consider requiring the client proof of RVPN-01-102 for control calls.

Classification: CWE-200 · OWASP ASVS 5.0.0 13.4.5, 13.4.6 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/S

RVPN-01-110 WP1: Destination names forge journal lines, and identities and destinations are logged by default (Medium)

Note: Partially fixed in commits `ffe0c7b` and `cef1cbe` and verified by Red Team Ltd (A35): a destination name with a line feed is refused, and at `info` no line named a `cid`, a client address or a destination. Not yet addressed: `debug` lines still carry header fields unescaped, so a `tz` with a line feed forged a line (A35b); two `warn` lines, limited to

one a minute, name a host or a client address (src/proxy.rs:325, src/mux.rs:223–226); and the statistics records and the ?cid= request line are unchanged.

Destination host names are taken from the SOCKS request as any UTF-8 string (src/proxy.rs:358–390) and written into log lines at warn level when a MUX2 connect fails (src/mux.rs:356) or the DNS fallback triggers (src/proxy.rs:312), so a name containing a line feed produces additional physical lines in the journal; the META and MUX2 header fields are logged verbatim at info level (src/mux.rs:45–53). Separately, at the production default level warn (src/main.rs:52) the node records the destination of every failed MUX2 connect, and at info it records tz, app, os, cid, sni and the client address for every MUX2 tunnel open and close and every control call (src/mux.rs:45–53, :177–184, :300; src/noise_proxy.rs:174–179), against the protection module's own rule that a node "must not record where anyone went" (src/protection.rs:1025–1027). With a stats connection configured, every tunnel open writes timestamp, client address, cid, tz, app, os and sni to a central store by default (src/stats.rs:76–87; src/config.rs:184–185), and legacy clients send the cid in the cleartext request line (src/http.rs:216–228).

Affected: src/mux.rs:45–53, :177–184, :300, :356; src/noise_proxy.rs:174–179; src/proxy.rs:312, :358–390; src/stats.rs:64–87.

Proof of Concept: Test A11g sent a MUX2 CONNECT whose domain was evil.example, a line feed, and the text of a plausible tunnel-open record. The server's journal then contained (A11g, abridged):

```
[2026-10-15T02:23:15.215Z WARN redsocket_http::mux] [MUX2] target connect evil.example
   [2026-10-15T00:00:00.000Z WARN redsocket_http::mux] [MUX2] tunnel open: tz=UTC app=rostam os=android
       ↪ cid=0000deadbeef0000 sni= ip=203.0.113.250:1 failed: DNS lookup failed for evil.example
```

The injected text starts its own physical line with an attacker-chosen timestamp, level, module and cid; env_logger's default continuation indent of four spaces and the trailing :1 failed: ... distinguish it on inspection, but line-oriented collection and searches for the cid or address treat it as a separate event. The statistics rows of A12 show the per-tunnel record shape.

Impact: An attacker fabricates journal evidence, false tunnel-open events for a chosen cid and address or noise that hides its own activity, wherever journals feed abuse handling or forensics. The default records bind a stable device identifier to client addresses, time zones and the first SNI in a central store, so a seized node, an exported journal or a compromised stats instance yields per-device connection histories, the primary adversary's objective.

Preconditions: Any tunnel for the injection; node, journal or stats-store access for the disclosure.

Recommendation: Escape control characters in every client-supplied string before logging (or log such strings with {:?}); keep destination names out of warn and info lines or gate them behind debug_log_destinations; reduce the statistics record to what billing needs (a server-side session identifier, a proven cid, bytes, coarse duration), replace the client address with a country code derived on the node or a keyed short-lived hash, drop sni, tz and os unless each has a documented consumer, and stop accepting ?cid= in the URL.

Classification: CWE-117 · OWASP ASVS 5.0.0 16.4.1, 16.2.5, 14.2.1; OWASP Top 10:2025 A09 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:L/SA:N

RVPN-01-111 WP1: Mailbox, broadcast, rules and entitlement content from Valkey is unsigned, and one oversize entry ends every control stream (Medium)

Note: Partially fixed in commit eb6625d and verified by Red Team Ltd (A36): a 70,085-byte broadcast entry was skipped and counted in redsocket_messaging_dropped_total, and the next entry reached the same control stream. Not yet addressed: store content is still unsigned, and a forged entitlement broadcast was again delivered.

Everything read from msg:*, msg:broadcast, rules:*, cid:* and cid:events is deserialised and forwarded verbatim; the exit adds only the scope field (src/messaging.rs:51–65, :93–106, :131–151, :1295–1312). A rules entry in a user's mailbox is applied to the live tunnel context (src/messaging.rs:1231–1241). The protection bundle, by contrast, is Ed25519-signed precisely so that a writer to the store cannot alter it (src/protection.rs:394–404). A broadcast entry whose serialised form exceeds 65,535 bytes makes write_frame fail (src/messaging.rs:210–218), and because the error propagates with ? the control stream of every msg1 tunnel on every exit ends; the dispatcher

keeps the entry in its in-memory cache until the next broadcast publish, so deleting it from the store does not recover the fleet by itself (`src/messaging.rs:786–792`, `:909–930`, `:1242–1247`).

Affected: `src/messaging.rs:51–65`, `:93–151`, `:210–218`, `:786–792`, `:909–930`, `:1180–1190`, `:1231–1247`, `:1295–1312`.

Proof of Concept: Test A16b against server D with a connected `msg1` client:

```
HSET msg:broadcast 0000000050 '{"seq":50,"ts":1792031639,"type":"entitlement","payload":{"state":"banned
    ↪ ","tier":"free"}}'
PUBLISH msg *
ctl <- {"seq":50,...,"type":"entitlement","payload":{"state":"banned","tier":"free"},"scope":"broadcast"}
```

A second entry with a 70,000-byte title and another publish ended the client's control stream (`early eof`). A fresh client that connected after the entry had been deleted, but before the next publish, was ended the same way; only a further `PUBLISH msg *` restored normal service (A16c).

Impact: An adversary with write access to the messaging store, which the threat model includes under compromise of the configuration and bootstrap plane, tells every client that it is banned or throttled, switches users' protection categories off, announces a false protection status, or takes the control plane of the whole fleet down with one entry.

Preconditions: Write access to the messaging Valkey, which is protected by CA-pinned TLS or mutual TLS and a password.

Recommendation: Sign mailbox and broadcast entries with the same Ed25519 key family as the bundle (the publisher signs `seq|ts|type|payload`, the exit verifies before forwarding), or document that write access to the store is equivalent to the signing key; skip and count, rather than fail on, an entry that cannot be framed; bound stored entries at the publisher.

Classification: CWE-345 · OWASP ASVS 5.0.0 4.1.5, 16.5.2; OWASP Top 10:2025 A08 · CVSS:4.0/AV:N/AC:L/AT:N/PR:H/UI:N

RVPN-01-112 WP1: UDP association delivers datagrams from any source to the client (Low)

Fix Note: This issue has been fixed by the development team in commit `5a7019d` and verified by Red Team Ltd to be working as expected. With the target on `::1` (A37), a datagram from 127.0.0.1 was dropped and one from `::1` on another port was delivered, which is the address-dependent filtering recommended; the end-to-end test `udp_filter_e2e` passed (A45).

One unconnected UDP socket serves every target of an association. The downlink task forwards whatever `recv_from` returns to the client, with the sender's address in the SOCKS header, whether or not that sender was ever a target; the `targets` list is maintained for the uplink but not consulted on receive (`src/mux.rs:505–545` ↪ , `:572–609`). RFC 1928 section 7 obliges a relay to filter only the client side of an association and leaves the remote side to the implementation, and `docs/udp-over-mux2.md` states no filtering behaviour. The result is endpoint-independent filtering, which RFC 4787 accepts for NATs that favour application transparency, but which on an exit lets any host reach a user's association. Every forwarded datagram also restarts the 120-second idle timer (`src/mux.rs:579–584`).

Affected: `src/mux.rs:475–476`, `:572–609`.

Proof of Concept: Test A14a through server K (deny list disabled so that a loopback UDP target is permitted). A client opened an association and sent one datagram to a loopback UDP echo target, which printed the association's source port; two datagrams then sent to that port from unrelated loopback sockets reached the client:

```
udp <- 31 B from 127.0.0.1:20153: echo:ping-from-client
udp <- 33 B from 127.0.0.1:61365: INJECTED-by-third-party
udp <- 24 B from 127.0.0.1:61347: INJECTED-again
```

A datagram with `FRAG` set was dropped without an echo (A14b); the code continues the association after such a datagram, as `docs/udp-over-mux2.md:60–61` documents (`src/mux.rs:501–503`). The zero-length close marker closed the stream as documented (A14c).

Impact: The association behaves as a full-cone mapping: a third party who learns the egress address and port injects datagrams into a user's association, keeps it alive, and has the injected bytes metered to the user. QUIC authenticates its payloads; plain DNS and other unauthenticated protocols do not.

Preconditions: Knowledge of the association's egress address and port, which every target observes.

Recommendation: Drop datagrams whose source is not among the association's pinned targets (address-dependent filtering, which RFC 4787 recommends where stricter filtering matters more than transparency); this also stops strangers from keeping the idle timer alive.

Classification: CWE-940 · RFC 4787 section 5; RFC 1928 section 7 · CVSS:4.0/AV:N/AC:H/AT:N/PR:N/UI:N/VC:N/VI:L/VA:N/S

RVPN-01-113 WP1: MUX2 relays to a non-reading target pin memory and descriptors that outlive the carrier (High)

Note: Partially fixed in commit `4a09a5d` and verified by Red Team Ltd (A38): stream tasks end with their carrier, and a target that accepts nothing for 60 seconds ends its stream. Not yet addressed: the yamux windows, a per-peer carrier limit and relay-aware shedding, so an open carrier still pins memory for up to 60 seconds per stall.

The server uses `yamux::Config::default()` with `set_max_num_streams(256)` (`src/mux.rs:90–93`); the vendored defaults are a 1 GiB connection receive window and a 256 KiB initial stream window that auto-tunes towards twice the bandwidth-delay product (`vendor/yamux/src/lib.rs:45`, `:87–92`, `:98–125`). Each stream is relayed by two spawned tasks with 64 KiB buffers (`src/mux.rs:372–427`). When the target does not read, the uplink task blocks in `write_all` without a timeout, the yamux stream buffers up to its window, and the kernel send buffer fills. The stream tasks are spawned detached from the carrier (`src/mux.rs:149–174`), so they stay blocked after the carrier ends, holding the target socket, the buffers and the stream state for as long as the target keeps its side open. Shedding gates only new upgrades (`src/http.rs:489–498`) and the cap does not reach detached tasks (RVPN-01-107). The client chooses the target, so a listener that accepts and never reads suffices.

Affected: `src/mux.rs:90–93`, `:149–174`, `:372–427`; `vendor/yamux/src/lib.rs:45`, `:87–125`.

Proof of Concept: Described by parameters and measured outcome. A15, through server K with the deny list disabled so that a loopback sink is reachable: one carrier opened 256 streams to a TCP sink that accepts and never reads and offered 2 MiB on each. Every stream accepted 786,432 bytes before stalling, 201,326,592 bytes in total, and the server's resident set rose from 10.6 MiB to 103.1 MiB within three seconds and stayed there after the carrier had closed. A26 repeated the setup with 32 streams on server S and watched what remained:

```
before: active_relays 0 active_tunnels 0 | sockets to sink 0 | fds 15 | rss 9.9 MiB
carrier open: active_relays 32 active_tunnels 1 | sockets to sink 32 ESTABLISHED | fds 47 | rss 23.7 MiB
t+60s after close: active_relays 32 active_tunnels 0 | sockets to sink 32 ESTABLISHED | fds 47 | rss 23.7
    ↪ MiB
```

The server logged `tunnel closed` for the carrier, and the samples at 2 and 15 seconds after the close were identical to the one at 60 seconds.

Impact: Each carrier pins about 90 MiB of server memory plus up to 256 descriptors and their kernel buffers, the pinning survives the carrier, and the attacker repeats it with fresh carriers without holding a connection to the exit. By extrapolation a dozen carriers pin a gibibyte and four exhaust a 1,024-descriptor limit, at which point RVPN-01-108 ends the process and every tunnel. Any holder of the public key, the censor included, thereby has a cheap, repeatable way to take exits down.

Preconditions: A tunnel, for which the public key suffices, and a reachable target that does not read; any public address the attacker controls qualifies.

Recommendation: Put write and idle timeouts on every stream relay and abort both tasks when the carrier ends, for example by holding their `AbortHandle`s in a per-carrier set that is dropped with the connection. Set `max_connection_receive_window` explicitly to the acceptable per-carrier footprint (the vendored assertion requires at least 256 KiB times `max_num_streams` and runs at call time, so set the stream count first), reduce `max_num_streams`, cap carriers per peer address, and count established relays and descriptors in the shedding decision.

Classification: CWE-770 · OWASP ASVS 5.0.0 15.2.2 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA

RVPN-01-114 WP1: Post-upgrade wire image identifies every session to a passive observer (High)

Client Note: The development team's remediation for the carrier's signature and the active-probing oracle is the Proteus transport (redsocket_proteus, WP2), which replaces the HTTP front; this server is retained for legacy clients.

Note: Unchanged in `147ab5e` (A30).

The README says the server "hides inside plain HTTP on port 80", but genuine sessions are easy for a passive classifier to recognise. Every repository client sends `GET /noise HTTP/1.1` with `Upgrade: websocket` and `Connection : Upgrade` but no `Sec-WebSocket-Key` (`src/bin/client.rs:382`, `src/bin/mux_client.rs:321`, `src/bin/legacy_client.rs:56-59`), so the server answers with the fixed 57-byte raw 101 that names no protocol (`src/http.rs:542-544`). What follows is not WebSocket framing: both sides exchange Noise messages, each behind a two-byte big-endian length in cleartext (`src/noise_proxy.rs:54-66`, `:320-326`; `src/noise_stream.rs:30-39`, `:76-82`). Handshake payloads are empty on the server and in every repository client (`src/noise.rs:63-65`; `src/bin/client.rs:82`), so the first record in each direction is `00 30` followed by 48 bytes, and every later record exposes its plaintext length plus 16; yamux's 12-byte control frames appear as 30-byte records. No padding exists or can be configured.

Affected: `src/http.rs:542-544`; `src/noise.rs:63-65`; `src/noise_proxy.rs:54-66`, `:320-326`; `src/noise_stream.rs:30-39`, `:76-82`; `README.md:11-29`. Also affects WP3, WP4 and WP5, whose clients implement the same framing.

Proof of Concept: Test A27 recorded a v1 session (META frame, then the control call `_rs.ping` with an eight-byte nonce) through a loopback relay, as an on-path observer sees it. The first bytes of each record are shown; the relay received some records as a length read followed by a body read:

```
c2s 104 B GET /noise HTTP/1.1\r\nHost: example.com\r\nUpgrade: websocket\r\nConnection: Upgrade\r\n...
s2c 57 B HTTP/1.1 101 Switching Protocols\r\nConnection: Upgrade\r\n\r\n
c2s 50 B 00 30 56 0f 18 af 80 24 f8 88 58 01 ... msg1
s2c 50 B 00 30 ae 48 63 e1 a6 f5 57 85 3b 4b ... msg2
c2s 54 B 00 34 d5 c2 f8 6c 21 08 87 9b 6d 7c ... META frame (36 B plaintext)
c2s 31 B 00 1d c6 3e 10 d5 1d a2 ... CONNECT _rs.ping (13 B plaintext)
s2c 26 B 00 18 4c 12 01 be 42 cf ... nonce echo (8 B plaintext)
```

The MUX2 session matched this exchange up to `msg2`; after its 54-byte header record the server's next record was 30 bytes (`00 1c`), a 12-byte yamux frame.

Impact: A national DPI system can classify sessions on port 80 at line rate with a rule as simple as "a 101 without an `Upgrade` header, followed by a 50-byte record starting `00 30` in each direction", without sending a single probe, so neither the host gate nor any fix to RVPN-01-101 helps. The censor thereby learns the client address of every user who connects directly and can block or record them selectively, which places the issue in the High tier of the engagement's calibration.

Preconditions: Clients connecting directly to port 80, the mode the README describes; sessions that reach the exit only through an encrypting front are not exposed on the censor's network.

Recommendation: Make the upgrade real: speak RFC 6455 framing end to end, or another protocol the cover traffic actually uses, send a conformant 101 that names it, and carry Noise inside it. Hide the record structure by encrypting the length prefix (as the Lightning BOLT 8 transport does) and padding handshake payloads and records to randomised lengths, behind a new capability so that the clients of WP3 to WP5 move together.

Classification: CWE-205 · RFC 9110 section 7.8; RFC 6455 section 5 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/

RVPN-01-115 WP1: Protection allow lists exempt whole shared hosting suffixes and accept single-label user entries (Medium)

Note: Partially fixed in commit `de17d9c` and verified by Red Team Ltd (A39): the single-label rule `example` was refused and counted in `rules_rejected_total`, and `evil.example` stayed blocked. Not yet addressed: the default and example `allow_hosts` still exempt `*.cloudfront.net` and `*.amazonaws.com` (`src/config.rs:117-118`, `config`

↪ .example.toml:73); public suffixes are accepted, A39 storing an allow rule for co.uk; and allowed names still bypass the controller and malware verdicts.

The protection judge consults allow_hosts before anything else and returns Allowed on a match, so neither the bundle's domain set nor its address ranges are consulted (src/protection.rs:921–943; src/mux.rs:307–309 ↪). The default list, used whenever allow_hosts is not set, is rostam.app, *.rostam.app, *.cloudfront.net and *.amazonaws.com (src/config.rs:116–121). The comment calls it the app's own control plane, but a suffix entry matches every name beneath it (src/protection.rs:599–609), and those two suffixes cover every customer distribution of a public CDN and every S3, EC2 and other AWS endpoint name. User rules have the same shape: valid_rule_entry is meant to admit "a single destination", yet it accepts any dot-separated label sequence including a single label such as com (src/protection.rs:617–641), and UserRules::allows_host matches the entry and every name under it (:686–693). An allowed name is cleared before its addresses are judged, so for names the allow list also overrides the controller and malware floor that categories_off cannot clear (src/protection.rs ↪ :211–218, :237–238).

Affected: src/config.rs:63–67, :116–121; src/protection.rs:582–610, :612–697, :921–943; src/mux.rs:305–333; src/ ↪ messaging.rs:298–317.

Proof of Concept: A16 V0 against server D, whose fixture bundle listed the domains evil.example and cloudfront.net, with the default allow_hosts:

```
evil.example:443 verdict BLOCK bits=0x0001 reason="domain"
<label>.cloudfront.net:443 verdict CLEAR (0x00)
```

The second name is a test label under the CloudFront suffix, withheld here under the publication rules.

A28 against server U with the same bundle: the rules frame {"rules":{"allow":["example"],"deny":[]," ↪ categories_off":0}} was accepted (redsocket_messaging_rules_rejected_total 0, stored as allow ["example"]), and the same cid's next blk1 stream to evil.example:443 received CLEAR instead of BLOCK; resetting the rule restored BLOCK.

Impact: Exit-side protection never blocks a phishing or malware host served from a CloudFront distribution or an AWS host name, however clearly the bundle lists it, and those platforms routinely host such content. One user rule, or one written by another party through RVPN-01-102, exempts a whole top-level domain from judgement, floor categories included, for every name-based connection of that user.

Preconditions: [protection] enabled with the default allow_hosts, or a user rule set by the user or through RVPN-01-102.

Recommendation: Replace the suffix entries with the exact host names, or account-specific distribution names, that the clients use, and refuse wildcard entries for shared platforms at start-up; require at least two labels and reject public suffixes in valid_rule_entry; keep controller and malware verdicts in force for user-allowed names by judging the resolved addresses before honouring an allow entry.

Classification: CWE-183 · OWASP ASVS 5.0.0 2.2.1 · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:L/VA:N/SC:N/SI:L/SA:N

Miscellaneous Issues

RVPN-01-116 WP1: HTTP, WebSocket and PROXY protocol handling departs from the specifications (Low)

Client Note: For the HTTP and WebSocket parts, the development team's remediation for the carrier's signature and the active-probing oracle is the Proteus transport (redsocket_proteus, WP2), which replaces the HTTP front; this server is retained for legacy clients.

Note: The PROXY part was fixed in 171aed6 and cbce7d0 and verified by Red Team Ltd (A31); the HTTP and WebSocket departures remain, apart from the control-frame length limit of 41078d3.

The front departs from the specifications in ways that make it distinguishable (RVPN-01-101) and that matter whenever an intermediary interprets the same bytes differently. On the HTTP side (RFC 9110, RFC 9112), the raw 101 names no protocol in an Upgrade header (RFC 9110 sections 7.8 and 15.2.2; src/http.rs:542–544); the

400 and 404 responses carry no `Date` (RFC 9110 section 6.6.1; `src/http.rs:442`, `:455`, `:572`); the request line is not validated beyond two whitespace-separated tokens and the HTTP version is ignored (RFC 9112 section 3; `src/http.rs:25–28`); an HTTP/1.1 request without `Host`, or with two `Host` field lines, is accepted (the latter judged on the first) instead of being answered with 400 (RFC 9112 section 3.2; `src/http.rs:48–53`); whitespace between a field name and the colon is trimmed instead of rejected (RFC 9112 section 5.1; `src/http.rs:35–37` ↪); and the server closes after a 404 without announcing `Connection: close` (RFC 9112 section 9.6). On the WebSocket side (RFC 6455), any `Sec-WebSocket-Key` and any `Sec-WebSocket-Version` are accepted where a 400 or a 426 is due (sections 4.2.1 and 4.4; `src/http.rs:472–477`, `:508–516`); unmasked client frames are accepted where the server must close the connection (section 5.1), reserved bits and the length and fragmentation limits of control frames are not checked (sections 5.2 and 5.5), and unknown opcodes are ignored instead of failing the connection (`src/ws.rs:62`, `:108–140`). On the PROXY protocol side, `auto` mode guesses whether a header is present, which the HAProxy specification forbids because it lets untrusted parties spoof their addresses; `UNKNOWN` and the v2 `LOCAL` command are mapped to `0.0.0.0:0` where the specification tells the receiver to use the real connection endpoints (`src/proxy_protocol.rs:47–58`, `:116–120`, `:148–153`); and the PROXY phase has no read deadline (RVPN-01-105).

Affected: `src/http.rs:25–53`, `:442`, `:455`, `:472–477`, `:508–516`, `:542–544`, `:572`; `src/ws.rs:62`, `:108–140`; `src/` ↪ `proxy_protocol.rs:39–64`, `:93–181`.

Proof of Concept: A06d: a request whose first line was `PROXY TCP4 203.0.113.7 198.51.100.1 51234 80` received a 101 from a server with `proxy_protocol = "disabled"`. A06b: `Host: front.example` followed by `Host: evil.example` received a 101 and the reverse order the 404. A06a: `Sec-WebSocket-Version: 7` received a 101. A07b: `PROXY` ↪ `UNKNOWN` was attributed `real_ip=0.0.0.0`. The handling of unmasked frames, reserved bits and control frames is static only (`src/ws.rs:113–139`).

Impact: Each departure is another stable discriminator for a prober. Where a CDN or load balancer sits in front of the exit, inconsistent handling of duplicate `Host` fields or malformed request lines between the two hops is the precondition for request-smuggling and routing confusion, and the `0.0.0.0` mapping loses attribution for health checks and other `LOCAL` connections.

Preconditions: Network reachability of port 80; an intermediary for the interoperability aspects.

Recommendation: Answer malformed requests, missing or duplicate `Host` fields and whitespace before the colon with 400, validate the HTTP version, add `Date` (and `Connection: close` when closing) to every negative response, send `Upgrade` in every 101; on the WebSocket path enforce version 13, a 16-byte base64 key, client masking, reserved bits and control-frame limits; drop `auto` in favour of an explicit trusted-source list (RVPN-01-105) and map `UNKNOWN` and `LOCAL` to the real peer.

Classification: CWE-1286 · OWASP ASVS 5.0.0 4.1.4, 1.5.3; RFC 9110; RFC 9112; RFC 6455 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N

RVPN-01-117 WP1: Destination policy permits IPv4-embedding IPv6 forms and the node's own addresses, and an empty list disables it (Low)

Note: Partially fixed in commit `7ba574f` and verified by Red Team Ltd (A40): `::7f00:1` and `64:ff9b::7f00:1` were refused as unroutable, and the README and example now say that `deny_cidrs` replaces the defaults. Not yet addressed: the local-use NAT64 prefix `64:ff9b:1::/48` (`src/proxy.rs:234–241`), the node's own addresses, and a floor that configuration cannot remove, so `deny_cidrs = []` still disables the policy.

`DEFAULT_DENY_CIDRS` covers RFC-special IPv4 and IPv6 space (`src/proxy.rs:125–146`), and `is_unroutable` unwraps IPv4-mapped addresses before judging them (`:232–246`). It unwraps neither IPv4-compatible addresses, which RFC 4291 section 2.5.5.1 deprecates, nor the NAT64 well-known prefix `64:ff9b::/96`; a comment in the project's tests states that NAT64-embedded addresses deliberately stay routable (`src/proxy.rs:657–659`), although RFC 6052 section 3.1 forbids representing non-global IPv4 addresses with the well-known prefix. The node's own addresses (`egress_ips` and interface addresses) are not refused, and the protection judge explicitly skips them (`src/protection.rs:963–965`). Finally, a configured list replaces the default instead of extending it: `deny_cidrs = []` refuses nothing, loopback included (`src/proxy.rs:197–229`), and the commented example in `config.example.toml` ↪ `:13–15` names three ranges, so uncommenting it silently drops the other seventeen default ranges, among them `::1/128`, `fc00::/7`, `192.168.0.0/16` and `100.64.0.0/10`.

Affected: src/proxy.rs:125–146, :197–246, :657–659; src/protection.rs:960–965; config.example.toml:13–15.

Proof of Concept: A10a, default policy, v1 CONNECTs: ::ffff:127.0.0.1 and ::ffff:7f00:1 were refused as unroutable, but ::7f00:1 and 64:ff9b::7f00:1 (both encoding 127.0.0.1) were not; the server attempted both and logged connect to [::7f00:1]:20101 timed out after 8s and the same for the NAT64 form. A10b, deny_cidrs = ↪ []: 127.0.0.1:20121 and localhost:20121 reached the server's own metrics endpoint through the tunnel and returned HTTP/1.1 200 OK.

Impact: On an exit whose network offers a NAT64 translator that does not itself refuse non-global destinations, a tunnel client reaches the translator's view of private IPv4 space. On any exit, services bound to the node's public addresses are reachable from the tunnel as connections originating from the node itself, which host firewalls commonly trust. Which services exist depends on the deployment, which is not in the tree, so this is a potential issue. An operator who narrows the list by uncommenting the example loses IPv6 loopback and private-range protection without warning.

Preconditions: A NAT64 path or local services on public addresses; or a customised deny_cidrs.

Recommendation: Unwrap IPv4-compatible addresses and the NAT64 prefixes (64:ff9b::/96, 64:ff9b:1::/48) and judge the embedded IPv4 address; add every egress_ips entry and local interface address to the deny set at start-up; make loopback, unspecified, link-local and the node's own addresses a floor that configuration cannot remove, and let deny_cidrs extend the defaults; correct the example comment.

Classification: CWE-918 · OWASP ASVS 5.0.0 1.3.6 · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:L/VI:L/VA:N/SC:L/SI:L/SA:N

RVPN-01-118 WP1: Valkey password written to the start-up log through a derived Debug implementation (Low)

Fix Note: This issue has been fixed by the development team in commits 4fb070c and 2d6abd7 and verified by Red Team Ltd to be working as expected. Commit 2d6abd7 removed Debug from config::Config and redacts the statistics password in config::ValkeyConfig, the latent hazards described below; the library tests passed on 147ab5e (A45), and the debug log of A44 contained neither the statistics password nor the private key.

In the initial public release d0d348b, valkey::Config derived Debug over all of its fields, including password. With [protection] enabled and a Valkey bundle source, main logs the source at info level with {:?} (src/ ↪ main.rs:230) and protection::Source derives Debug (src/protection.rs:1046–1052), so the start-up banner printed the password of the store that holds mailboxes, rules, the ledger and the bundle, together with the client certificate and key paths. Commit 4fb070c removed the derived implementation, implemented Debug by hand with <redacted> in place of the password (src/valkey.rs:31–43) and added the regression test debug_output_never_contains_the_password (src/valkey.rs:432–442). Red Team Ltd compared the commit with its parent, confirmed the change in 4430a1d, and saw the test pass on 3012aa6 (A02) and 4430a1d (A23). Two related structures still derive Debug over secrets, config::Config with private_key and extra_private_keys (src/config.rs:9–14) and config::ValkeyConfig with the stats password (src/config.rs:201–207), but no format site in the tree prints either, so they are a latent hazard only.

Affected: src/valkey.rs:14 (in d0d348b); src/main.rs:230; src/protection.rs:1046–1052.

Proof of Concept: Static; the fix preceded the dynamic tests. Code excerpts:

```
d0d348b src/valkey.rs:14 #[derive(Debug, Clone, serde::Deserialize, PartialEq, Eq)]
4430a1d src/main.rs:230 log::info!(" protection: on, bundle from {:?}", source);
4fb070c src/valkey.rs:37 .field("password", &"<redacted>")
```

Impact: Before the fix, anyone able to read the journal of a node started at info level (the default is warn) obtained the credential that grants the write access described in RVPN-01-111.

Preconditions: Before 4fb070c: [protection] with a Valkey source and a log level of info or lower.

Recommendation: Apply the same treatment to config::Config and config::ValkeyConfig, or wrap secrets in a type whose Debug redacts, so that a future debug print cannot reintroduce the issue.

Classification: CWE-532 · OWASP ASVS 5.0.0 16.2.5, 13.3.1 · CVSS:4.0/AV:L/AC:L/AT:P/PR:L/UI:N/VC:N/VI:N/VA:N/SC:H/SI

RVPN-01-119 WP1: Example configuration ships a complete server keypair; the private key is accepted on the command line (Low)

Note: Partially fixed in commit `629ea63` and verified by Red Team Ltd (A41): the shipped examples carry placeholders, and a placeholder or the former example key was refused as `private_key`, in `extra_private_keys` and through `--private-key`. Not yet addressed: `--private-key` is still accepted and visible in the process list; the comparison is case-sensitive, so the former example key in upper-case hex was accepted; a mismatching `public_key` only causes a warning; and key material is not zeroised.

`config.example.toml:3-4` contains a complete X25519 private and public key pair, and `client.example.toml:2` the matching public key. `Config::load` checks only the length of the keys (`src/config.rs:337-365`) and nothing refuses to start with the published private key. An operator who copies both files unchanged runs an exit whose private key is public, so anyone can impersonate that exit to clients configured with the example public key and read their traffic, because Noise NK authenticates the server by that key alone. The README states that signing material was replaced by `REPLACE_WITH_*` placeholders for publication (`README.md:158-161`); these keys were not. The server also accepts `--private-key` on the command line (`src/main.rs:37-39`, `:68`), where the value is visible to other local users through the process list and ends up in shell history and service units. A `public_key` that does not match the private key is only warned about and the derived key is used (`src/main.rs:106-119`), so a copy-paste error goes unnoticed. The key material is held in ordinary heap memory and never zeroised.

Affected: `config.example.toml:3-4`; `client.example.toml:2`; `src/config.rs:337-365`; `src/main.rs:37-43`, `:68-69`, `:106-119`.

Proof of Concept: Static. `config.example.toml:3` holds `private_key = "<64 hex characters>"`, the value is accepted by `Config::load`, and `src/main.rs:37-39` declares the `--private-key` option; the test configurations of this assessment replaced the example pair with fresh keys from `redsocket-http --keygen`.

Impact: A copy-paste deployment yields an exit that any party can impersonate, including the national censor in its man-in-the-middle position; a local user or a backup of the service definition reveals a key passed on the command line.

Preconditions: A deployment that reuses the example key, or a key passed on the command line on a shared host.

Recommendation: Replace the example values with placeholders and refuse to start with a placeholder or the former example key; remove `--private-key` or read the key from a file or file descriptor; fail on a public key mismatch; zeroise key material after use.

Classification: CWE-1394 · OWASP ASVS 5.0.0 13.3.1, 13.2.3 · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/

RVPN-01-120 WP1: Protection fails open until the first bundle loads, and rollback protection does not survive a restart (Low)

Note: Unchanged in `147ab5e` (static; `src/protection.rs:929`, `:953-956`).

`judge_host` and `judge_addrs` return `Allowed` and `Clear` when no bundle is installed and only count the stream as unchecked (`src/protection.rs:921-928`, `:949-954`). `status()` then reports `enabled: false`, so a current client knows that it is unprotected (`src/protection.rs:862-867`; `tests/no_bundle_e2e.rs`). The stale-bundle check compares `generated_at` with the bundle currently loaded (`src/protection.rs:802-821`), so the first load after a restart accepts any validly signed bundle of any age; nothing persists a high-water mark and no maximum age is enforced.

Affected: `src/protection.rs:802-821`, `:862-867`, `:921-928`, `:949-954`.

Proof of Concept: Static, supported by the project's end-to-end test `no_bundle_means_enabled_false_not_silent_clear` ↪ , which passed in A02. Excerpt (`src/protection.rs:925-928`):

```
let Some(bundle) = current() else {
   st.unchecked.fetch_add(1, Ordering::Relaxed);
   return HostVerdict::Allowed;
};
```

Impact: Every restart opens a window without exit-side protection whose length is the bundle fetch time, or which stays open if the fetch fails; an adversary with write access to the store can replay an older signed bundle, lacking recent entries, into that window.

Preconditions: A restart, combined with a Valkey outage or with write access to the store.

Recommendation: Persist the last known good bundle, or at least its `generated_at`, on disk and load it before serving; refuse bundles older than a configured maximum age; consider answering `NOTIFY` rather than `Clear` for enrolled brands while no bundle is loaded.

Classification: CWE-636 · OWASP ASVS 5.0.0 16.5.3 · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:N/VI:L/VA:N/SC:N/SI:L/SA:

RVPN-01-121 WP1: The opt-in io_uring backend has no back-pressure, metering or lifetime cap (Low)

Note: Unchanged in `147ab5e` (static): `src/iouring_relay.rs` was not modified, although the WebSocket decoder it uses no longer reassembles fragments (`41078d3`).

With `[relay] io_backend = "iouring"` on Linux, a v1 tunnel is handed to a reactor thread before `RelayGuard` and `TunnelGuard` are created (`src/noise_proxy.rs:212–250` run before `:253–256`), so tunnels on this path appear in no usage row, no active-tunnel gauge and no byte counter. The reactor arms multishot receives on both sockets, re-arms them whenever the kernel ends a multishot sequence, and appends decrypted data to per-connection queues without any limit and without pausing reception when the opposite socket does not drain (`src/iouring_relay.rs:266–272`, `:382–429`, `:450–500`). Nothing in the reactor ends a connection on age or idleness, and the 600-second wrapper no longer applies because the reactor owns duplicated descriptors. The WebSocket decoder that the reactor uses reassembles fragments without a bound (RVPN-01-103).

Affected: `src/noise_proxy.rs:212–256`; `src/iouring_relay.rs:266–272`, `:382–429`, `:450–500`; `src/ws_codec.rs:100–105` ↪ .

Proof of Concept: Static only; the backend is Linux-only and could not be built or run on the macOS assessment host. Excerpt (`src/iouring_relay.rs:422–423`), executed for every received buffer regardless of how much is already queued:

```
c.pend_ab.extend(to_target);
c.pend_ba.extend(to_client);
```

Impact: Enabling the backend would reintroduce the memory growth of RVPN-01-113 in an unbounded form, at the client's upload rate towards any target that does not read, and would remove usage accounting and the connection cap for v1 tunnels.

Preconditions: Linux and `io_backend = "iouring"`, which is off by default.

Recommendation: Stop re-arming the receive on a direction whose opposite queue exceeds a bound, cap both queues, add idle and age timers, create the accounting guards before the hand-off and update the byte counters in the reactor, and keep the backend disabled until this is done.

Classification: CWE-770 · OWASP ASVS 5.0.0 15.2.2 · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:L/VA:H/SC:N/SI:N/SA:

RVPN-01-122 WP1: rustls 0.23.42 carries four advisories and rustls-pemfile is unmaintained (Low)

Note: Unchanged in `147ab5e`: `Cargo.lock` was not modified, and `cargo audit -n` reported the same four advisories and the same warning (A46).

`cargo audit` (A04, advisory database `edb58f59` of 2026-10-14) reports four advisories against rustls 0.23.42 (`Cargo` ↪ `.lock:1569–1570`): RUSTSEC-2026-0285, TLS 1.3 handshake messages accepted across encryption-level boundaries (fixed in 0.23.45 and later); RUSTSEC-2026-0346, `Debug` implementations for client session values expose TLS secrets; RUSTSEC-2026-0348, Brotli certificate decompression can allocate up to 1 GiB before authentication; and RUSTSEC-2026-0349, an unauthenticated ServerHello can downgrade later connections to classical key exchange (the last three fixed in 0.23.46 and later). It also reports RUSTSEC-2025-0134, rustls-pemfile 2.2.0 unmaintained (`Cargo.lock:1584–1585`). rustls serves only as the TLS client towards the stats and

messaging Valkey instances, which are pinned to a configured CA (`src/stats.rs:207–221`, `src/valkey.rs:148–174` ↪), and is built with the features `ring`, `std`, `tls12` and `logging` (`Cargo.toml:45`). No Brotli crate is in `Cargo.lock` ↪ , so RUSTSEC-2026-0348 is not reachable as built, and the tree never formats rustls session values, so RUSTSEC-2026-0346 is not either. Whether RUSTSEC-2026-0349 applies depends on hybrid key exchange being negotiated with the ring provider, which was not determined.

Affected: `Cargo.toml:45–47`; `Cargo.lock:1569–1570`, `:1584–1585`.

Proof of Concept: `cargo audit -n` in the repository root (A04):

```
Crate: rustls
Version: 0.23.42
ID: RUSTSEC-2026-0285
Solution: Upgrade to >=0.23.45
...
error: 4 vulnerabilities found!
warning: 1 allowed warning found
```

Impact: Exploitation of RUSTSEC-2026-0285 requires a hostile or impersonated Valkey endpoint, which the CA pinning makes unlikely; the item is primarily dependency hygiene, but it sits on the path that carries the statistics and the messaging credentials.

Preconditions: A hostile or impersonated Valkey endpoint.

Recommendation: Update rustls to 0.23.46 or later with `cargo update -p rustls` and re-vendor; replace `rustls` ↪ `-pemfile` with the PEM parsing in `rustls-pki-types`; disable the `tls12` feature if the Valkey instances support TLS 1.3; run `cargo audit` on every change and on a schedule.

Classification: CWE-1395 · OWASP ASVS 5.0.0 15.2.1, 15.1.1 · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:L/VI:L/VA:L/SC:N/S

RVPN-01-123 WP1: Publicly routable addresses and a hard-coded server public key remain in fixtures, code and documentation (Info)

Note: Partially fixed. Commits `4fb070c` and `4430a1d` had moved the fixtures that resembled operator infrastructure in `src/config.rs`, `src/bin/mux_client.rs`, `src/http.rs` and `src/proxy.rs` to documentation ranges before version 1.0 (A23). Commits `60cf688` and `f202f2d` then removed the provider-assigned IPv6 address and the labelled bundle fixtures listed first and second below, the placeholders in `src/messaging.rs` and `src/protection.rs`, and the hard-coded `mux_client` key, whose default is now the former example public key, paired with a private key that the server refuses (RVPN-01-119). S04 on `147ab5e` left 53 hits outside `Cargo.lock` and the two key constants, against 79 in S02. Not yet addressed: the resolver, boundary and third-party operator values, the placeholder addresses in `src/http.rs` and `src/proxy.rs`, the CloudFront-style names, now at `src/protection.rs:2012–2013`, and the published history. The line numbers below are those of `4430a1d`.

The scrub scan S02 of `4430a1d` (every file except `.git/`, `target/` and `vendor/`) found the following classes of value outside the RFC 5737 and RFC 3849 documentation ranges; in line with the publication rules of this engagement the values themselves are not reproduced:

- `src/proxy.rs:743`: a provider-assigned IPv6 address of unknown ownership whose low bits mirror a private IPv4 address used in the same test; the earlier review flagged it as a possible infrastructure address and `4430a1d` left it unchanged.
- `src/protection.rs:1517–1527`, `:1807–1823`: public IPv4 and IPv6 addresses and ranges of unknown origin, used as bundle fixtures and labelled `controller`, `phishing` or `scanner`.
- `src/bin/mux_client.rs:301–303`: a hard-coded X25519 server public key used as the default `MUX_PUBKEY`; `4fb070c` replaced the paired default server address but not the key.
- `src/proxy.rs:99`: a third-party public DNS resolver address used as the target of the start-up IPv6 route probe (functional code; the UDP `connect()` sends no packet).
- `src/proxy.rs:671`, `:732–733`, `:744–745`, `:747`, `:992`; `src/protection.rs:1635`, `:1641`, `:1812`, `:1815`; `src/http.rs:179`, `:188`; `docs/userspace-tcp-dpdk-research.md:110`: third-party public DNS resolver addresses.
- `src/proxy.rs:734–742`, `:748–749`: public addresses used as boundary values next to denied ranges.

- src/proxy.rs:746, :813, :815, :825, :837, :865, :940–941, :967, :971; src/proxy_protocol.rs:235, :240; src/protection.rs:2022, :2037; src/http.rs:130, :133: IPv6 addresses in prefixes of third-party CDN, hosting and social-network operators.
- src/proxy.rs:812, :814, :856, :960, :964; src/protection.rs:1831, :2043–2054; src/messaging.rs:1459; src/http.rs:158–188: placeholder public IPv4 addresses, one of them as a /24 prefix in a format string.
- src/protection.rs:2009–2010: synthetic host names under the CloudFront suffix.

Affected: the files and lines listed above.

Proof of Concept: Scrub scan S02 (Appendix A): 356 raw hits, of which 261 are crate checksums in Cargo.lock, three are the example keys of RVPN-01-119, one is a source-tree digest in audit-results.txt, and twelve were discarded on review as multicast, reserved or partially matched IPv4-mapped forms; the remaining 79 hits fall on the lines listed above.

Impact: None for the running server. The risk is that a value of unknown origin is a fleet, client or third-party address that publication exposes, or that the fixtures publicly label real third-party addresses as malicious.

Preconditions: None.

Recommendation: Replace every listed literal with documentation addresses or derive it at test time; generate the mux_client default key at run time or require MUX_PUBKEY; add the scan to CI; and if any replaced or remaining value is live infrastructure, rotate it, because a published commit cannot be recalled.

Classification: CWE-540 · RFC 5737; RFC 3849 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N

RVPN-01-124 WP1: Handshake-failure and uptime metrics and the shedding documentation misreport behaviour (Info)

Fix Note: This issue has been fixed by the development team in commit 6f970a0 and verified by Red Team Ltd to be working as expected. Ten malformed handshakes raised redsocket_hs_fail by 10 (A42); a refused destination and a control call left it unchanged, the first scrape 5.8 seconds after start reported an uptime of 5 seconds, redsocket_stats_dropped_total was exported, and the banner described the 404 decoy (A47).

redsocket_hs_fail is incremented inside run_streaming_tunnel_with_prefix when msg1 fails to decrypt (src/noise_proxy.rs:58–61) and again by its caller for every error the tunnel returns (src/http.rs:535, :567), so a failed handshake counts twice, and post-handshake errors such as a refused destination or a closed connection count as handshake failures. redsocket_uptime_seconds measures from the first metrics scrape, because the start instant is initialised lazily inside the scrape (src/main.rs:322, :436–442). The README, the start-up banner, the shedding warning, the configuration comment and the metric help text all say that shedding answers with a 503 (README.md:91; src/main.rs:190–197, :403; src/shed.rs:139–141; src/config.rs:138), whereas the code sends the 404 decoy (src/shed.rs:43–47), which is the safer behaviour. The README also states that dropped statistics rows are counted in metrics (README.md:133–134), but the counter stats::dropped_records() (src/stats.rs:101–105) is not exported anywhere.

Affected: src/noise_proxy.rs:58–61; src/http.rs:535, :567; src/main.rs:190–197, :322, :403, :436–442; src/shed.rs:139–141; src/stats.rs:101–105; README.md:91, :133–134.

Proof of Concept: A08: ten malformed handshake attempts moved redsocket_hs_fail from 17 to 34. A19: while the banner read “Clients see a 503 and rotate to another node”, the shedding response was byte-identical to the 404 decoy.

Impact: The handshake-failure counter is the operator’s main signal of active probing (RVPN-01-101), and it is inflated and mixed with unrelated errors; the uptime figure misleads restart detection; operators who follow the documentation look for a 503 that never comes, and cannot see statistics rows being lost.

Preconditions: None.

Recommendation: Count each handshake failure once, where it occurs, with separate counters for post-handshake errors; initialise the start instant in main; export the dropped-row counter; describe the decoy in the README, the log lines and the help text.

Classification: CWE-1068 · OWASP ASVS 5.0.0 16.3.1 · CVSS:4.0/AV:N/AC:H/AT:N/PR:N/UI:N/VC:N/VI:L/VA:N/SC:N/SI:N/SA

Conclusions

redsocket_http is, at the level of individual functions, a carefully built piece of software. Its parsers check lengths before they slice, its relays are written against tokio's cancellation semantics with tests that pin the subtle cases, the signed protection bundle is verified in the right order, and the code carries the marks of earlier security work: bare-LF smuggling is tested, statistics fields are neutralised, the metrics endpoint is clamped to loopback, and replies from Valkey are bounded before anything is allocated from them. No network input that Red Team Ltd sent crashed the server under the `panic = "abort"` profile, the destination policy held against every loopback, private, mapped and numeric form tried except two IPv4-embedding IPv6 notations, and the project's own suite of 118 tests passed with a clean clippy run on `3012aa6`, its 110 library tests again on `4430a1d`, and 127 library tests and two new end-to-end tests on `147ab5e`.

The weaknesses are of a different kind. They sit in the design assumptions rather than in individual lines: that an exit on port 80 is hidden when it answers 101 to anything and then speaks a length-prefixed Noise stream behind a 101 that names no protocol; that a client-asserted `cid` can key mailboxes, rules and billing records; and, until the remediation pass, that the resource lifetimes of detached tasks were bounded by a timeout that did not reach them. Against the engagement's primary adversary these assumptions fail in ways that matter for the product's purpose. A national censor can recognise every exit by probing it and every direct session by watching it (RVPN-01-101, RVPN-01-114); with `[messaging]` enabled any client can read and rewrite another user's control-plane state (RVPN-01-102); and, before the remediation pass, any holder of the public key could take a node down with memory or descriptors that outlived the connection that consumed them (RVPN-01-103, RVPN-01-108, RVPN-01-113).

Red Team Ltd recorded 24 issues: 15 vulnerabilities (5 High, 9 Medium, 1 Low) and 9 miscellaneous issues (7 Low, 2 Info). One of them, the Valkey password in the start-up log, was fixed by the development team during the engagement in commit `4fb070c` and verified. The fixture scrub of `4430a1d` was verified for the lines it changed but did not cover every publicly routable value in the tree (RVPN-01-123). After version 1.0 the maintainers landed a remediation pass, which Red Team Ltd re-verified on `147ab5e` for this version: 8 issues are now fixed, 9 are partially fixed and 7 remain open. The paragraphs below walk through the component area by area, recording what was examined, what held and what did not. The area paragraphs describe `4430a1d` as assessed for version 1.0, so that the record of each issue stays intact; the paragraphs from the re-verification of the remediation pass onward describe `147ab5e`.

Severity follows the engagement's calibration. Fingerprintability that lets the censor identify or block users is High unless mitigated, which places RVPN-01-101 and RVPN-01-114 in that tier although neither breaks confidentiality. RVPN-01-102 is High because, wherever `[messaging]` is enabled, any client can act as any other user in the control plane; on exits without the module it does not apply, and its CVSS vector records that attack requirement. RVPN-01-103 and RVPN-01-113 are High because a single remote party, unauthenticated in the first case and holding only the published key in the second, can end every tunnel on a node, and in the second case can repeat the attack without holding a connection. RVPN-01-108 is Medium on its own because the descriptor limit of production nodes is not known; where that limit is low it is the shortest path from RVPN-01-113 to a crash. The integrity and privacy issues in records and logs (RVPN-01-104, RVPN-01-106, RVPN-01-110) are Medium because their impact depends on how the records are used and who can read them, and hygiene and specification issues are Low or Info. The CVSS v4.0 vectors are given for comparability only; the qualitative rating decides.

HTTP front and probing resistance. The front was read in full and exercised with 33 request shapes against the default configuration, the host gate and a reference server (A06), with the post-upgrade behaviour probed separately. Several properties held well. Headers are split on CRLF only, so a bare line feed cannot smuggle a higher-priority real-IP header (a unit test covers this); the header block is bounded to 16 KiB and five seconds; the 404 decoy and the shedding response are byte-identical, which a unit test pins and A19 confirmed on the wire; and the host gate, where configured, refused wrong and missing `Host` values. What did not hold is the property the front exists for. Any request with an `Upgrade` header receives a 101 from any `Host` in the default configuration, whatever its method, path, `Connection` header or first line; the raw 101 omits the `Upgrade` header; the negative responses lack `Date` and `Server`; a TLS ClientHello earns a 400 after exactly five seconds followed by a reset; and a request with two `Host` fields is judged on the first. Each of these separates the exit from any mainstream server in a single exchange (RVPN-01-101, RVPN-01-116). The host gate is a useful stopgap but not a defence, because the `Host` value it relies on travels in cleartext in every genuine session. The verdict for

this area is that the front does not achieve probing resistance, and that the remedy is a combination of a client proof that a prober cannot derive from the public key and responses copied faithfully from a chosen reference server.

Wire image and traffic analysis. The bytes of genuine sessions were recorded through a loopback relay (A27) and compared with the code. The result is that probing resistance would not be enough even if the front were fixed. The repository's clients request `Upgrade: websocket` without a key, receive the fixed 57-byte 101, and then both sides exchange 50-byte records that begin `00 30`, followed by records whose cleartext two-byte length reveals the plaintext size of every message (RVPN-01-114). A passive classifier needs neither a probe nor the key to flag every direct session and the address of its user. The fix belongs to the protocol rather than to the server alone: carry Noise inside a protocol whose framing the cover traffic actually uses, mask or encrypt the length prefix, pad handshake payloads and records to randomised sizes, and move the clients of WP3 to WP5 together behind a capability. Red Team Ltd notes that the server is otherwise well placed for such a change, because framing is already isolated in `noise_stream`, `noise_framer` and the v1 relay loop.

Noise session. The Noise layer was checked against revision 34 of the Noise Protocol Framework. The pattern is `Noise_NK_25519_ChaChaPoly_BLAKE2s` through snow 0.9.6; the server is the responder and authenticates with its static key, which every client knows; `msg2` carries an empty payload; transport messages are limited to 65,535 bytes of ciphertext, and the framer rejects lengths outside 16 to 65,535 before decrypting. The key-rotation loop builds a fresh responder for each candidate key, which is the correct way to cope with snow consuming its state on a failed read. Forward secrecy holds for transport data against a later compromise of the static key, because the transport keys also depend on the ephemeral-ephemeral exchange; only the `msg1` payload, which the repository's clients leave empty, depends on the static key alone. The absence of rekeying is acceptable with ChaChaPoly's 64-bit nonce space. Malformed handshakes and transport frames (A08, A11f) were rejected without a crash. Three observations remain. The handshake read has no deadline of its own, so a silent client holds its connection until the outer cap (RVPN-01-108). The handshake-failure counter double-counts and mixes in unrelated errors (RVPN-01-124). And NK authenticates only the server, which is a deliberate choice for a censorship-circumvention tunnel but is also the root of the identity problems in messaging and statistics (RVPN-01-102, RVPN-01-104). The X25519 exchange is classical; against an adversary that records traffic today in order to decrypt it later, a migration plan towards a hybrid post-quantum handshake belongs in the roadmap (OWASP ASVS 5.0.0 11.1.4). Replay was considered as well: a recorded `msg1` replayed to the exit produces a fresh `msg2` under a new server ephemeral key, and the replaying party cannot complete the session without the client's ephemeral private key, so replay yields no data, but every replay is answered and therefore confirms the exit to anyone who has recorded one genuine session. The prologue is empty, so nothing from the HTTP exchange (the request path, the `Host` value or a future client proof) is bound into the handshake transcript; if the upgrade request gains a client proof as recommended in RVPN-01-101, binding it into the prologue would stop an on-path party from moving a valid proof to a different session.

PROXY protocol and client attribution. The PROXY parser is bounded where it matters: the v1 line is limited to 107 bytes, the v2 body length is read before allocation, and unit tests drive the real parser over real sockets. The configuration around it is the problem. An `[http]` table without the `proxy_protocol` key yields `auto` rather than `disabled` (RVPN-01-105), `auto` honours a header from any peer and guesses whether one is present, and the PROXY phase runs before both the five-second header deadline and the 600-second cap, so in `auto` mode a silent connection is held indefinitely (A24). Independently, the CDN real-IP headers are trusted from any peer (RVPN-01-106). Every record that carries a client address is therefore attacker-chosen unless the deployment is configured exactly right. Attribution should rest on an explicit list of trusted sources, checked against the socket peer, for both mechanisms. In practice there are three deployment patterns, and each needs a different setting: a node that receives clients directly should run with `proxy_protocol = "disabled"` and ignore every CDN header; a node behind a load balancer that prepends PROXY headers should use `required` with the balancer's addresses as the only trusted sources; and a node behind a CDN should trust that CDN's one header only when the peer is inside the CDN's published ranges. The current code cannot express the second and third patterns safely, because neither mechanism looks at the peer.

v1 relay and connection lifetime. The v1 relay splits the client and target streams into two tasks, selects on both and aborts the survivor, and its byte counters are shared atomics so that usage stays correct when one direction is cancelled; the project's tests cover exactly those cancellation cases. Target sockets get keepalive, and egress binding sets `IP_BIND_ADDRESS_NO_PORT` on Linux so that bind-before-connect does not exhaust ephemeral

ports. The weakness is the outer cap. It is a timeout around the connection future, and the relay tasks are not owned by that future, so a v1 relay continues after the cap while the usage row is written as if the tunnel had ended (RVPN-01-107, A18). The WebSocket path keeps its socket after the cap as long as the client wishes (A20, A25), and the MUX2 path relies on the yamux connection being dropped, which ends the carrier but not stream relays blocked on a target (RVPN-01-113). The project should decide what the cap is for and then make every task that serves a connection end with it.

MUX2 carrier. The multiplexed carrier was traced from the header parse through the yamux configuration to the per-stream relay, and exercised end to end with the repository's own SOCKS5 client (A17), with malformed stream requests (A11e) and with stalled streams (A15, A26). The stream request path is well bounded: a CONNECT longer than 512 bytes, a one-byte request, an unknown address type and a zero-length domain were all refused, and the carrier is capped at 256 streams. The control stream is opened by the server and announced with a magic byte so that a client never mistakes a verdict for data. The yamux configuration, however, leaves the connection-wide receive window at the vendored 1 GiB default, and the stream relays are spawned detached from the carrier with no write or idle timeout. A15 measured about 90 MiB of server memory pinned by one carrier whose 256 streams target a listener that never reads, and A26 showed that the relays, their target sockets and the memory remained after the carrier was gone (RVPN-01-113). Combined with the accept-loop behaviour of RVPN-01-108 this gives any key holder a repeatable way to exhaust a node. The fix is small in code: abort the stream tasks when the carrier ends, put timeouts on relay writes, and set the yamux window explicitly.

UDP association. The UDP-over-MUX2 design (`docs/udp-over-mux2.md`) matches the code: one unconnected socket per association bound to the client's sticky egress address, targets resolved once and pinned, the destination policy and the protection judge applied to every new target, a cap of 64 pinned targets, `FRAG` datagrams dropped individually, a zero-length close marker and a 120-second idle cut-off. Datagrams up to 9 KiB pass without loss and a larger one costs a single dropped datagram while the buffer grows, as documented. A14 confirmed the forwarding, the `FRAG` drop and the close marker, and A14d confirmed that a loopback target was silently refused under the default policy. The one gap is on the downlink, which forwards datagrams from any source, so that a stranger who learns the association's egress port can inject datagrams and keep the association alive (RVPN-01-112). Filtering on the pinned targets closes it at no cost.

Destination policy. This area held up best under dynamic testing. The policy is evaluated on resolved addresses rather than on names; IPv4-mapped IPv6 is unwrapped before judging; the protection judge and the dialer share one resolution, so there is no window for DNS rebinding between the check and the connect; the UDP path applies the same check; and the default set covers loopback, RFC 1918, CGNAT, link-local, documentation, benchmarking, multicast and reserved space for both families. A10 refused `127.1`, `0x7f000001`, `2130706433`, `localhost` with and without a trailing dot, `0.0.0.0`, `::1`, both IPv4-mapped forms, the host's own LAN address and every private and special range tried, and A11e did the same for ATYP 4 literals on the MUX2 path. The leading-zero form `0177.0.0.1` was read by the macOS resolver as a decimal, public address, and the policy correctly judged the address that resulted. The gaps are at the edges (RVPN-01-117): IPv4-compatible and NAT64 notations are not unwrapped, the node's own public addresses are not refused, and a configured `deny_cidrs` replaces the defaults instead of extending them, so that `deny_cidrs = []` exposes loopback services including the metrics endpoint (A10b). Making loopback, link-local and the node's own addresses a non-configurable floor would remove the operator foot-guns entirely. One further property was noted but not tested. Names are resolved through an in-process hickory cache of 65,536 entries shared by all users of the node, with a five-second floor on positive and negative TTLs, so a client can in principle tell from response timing whether a name was resolved recently by someone else on the same exit. The same is true of any shared recursive resolver, and it is listed here only as a potential cross-user side channel worth bearing in mind if per-user privacy guarantees are ever stated.

WebSocket path. The WebSocket code path serves no client in the repository: every client sends `Upgrade: ↪ websocket` without a key and takes the raw path. It is nevertheless reachable by anyone, before any handshake, and its reassembly buffer has no bound (RVPN-01-103). A09 drove the server's resident set from 11.0 MiB to 923.6 MiB with 512 MiB of unauthenticated fragments, and A20 showed the growth continuing after the cap had fired. The handshake also accepts any key and version, and the frame reader accepts unmasked frames and ignores unknown opcodes (RVPN-01-116). Red Team Ltd recommends removing the path unless a shipped client depends on it; if it stays, the reassembled message should be bounded to one Noise frame

and the pumps tied to the connection's lifetime. The decoder used by the io_uring backend, `ws_codec`, has the same unbounded reassembly despite its careful incremental parsing and split-at-every-offset tests.

Resource management, shedding and the accept loop. Load shedding is a thoughtful design: it samples the cgroup v2 or v1 limit (falling back to the resident set against total memory), refuses only new upgrades, answers them with the same bytes as the decoy, and keeps an eight-point hysteresis band. A19 confirmed the thresholds, the hysteresis and the decoy response with a simulated memory reading; the cgroup and procfs sampling itself is Linux-specific and was reviewed statically. Shedding cannot, however, help against the issues above, because it sees neither memory that established connections keep acquiring (RVPN-01-103, RVPN-01-113) nor descriptor pressure, and on a platform where memory cannot be read it is disabled with a warning. The accept loop is the weakest point: any `accept()` error, including running out of descriptors, ends the process and every tunnel on it (RVPN-01-108). A13 showed the process exit after 245 idle connections under a 256-descriptor limit. Idle connections are cheap to hold, for 600 seconds after a 101 and without limit in PROXY `auto` mode, and RVPN-01-113 pins descriptors without holding a connection at all. The relay buffers themselves are modest and configurable: `initial_buffer` lets idle tunnels start small and grow on demand, which helps nodes with many quiet tunnels. The node needs per-peer and global connection ceilings, deadlines on every pre-relay phase, an accept loop that survives transient errors, and shedding that also counts established relays and descriptors.

In-tunnel control channel. The `_rs.*` pseudo-host is a compact design: `ping` echoes an eight-byte nonce, `notify` answers with an empty frame, and unknown subcommands receive an answer instead of a hang, which a unit test pins. But `_rs.stat` hands node-wide tunnel counts and byte totals to every client, and `_rs.config` names the product, the same information that the project deliberately keeps off the public metrics port (RVPN-01-109). The unescaped `cid` in `_rs.stat` malforms only the caller's own reply. Returning per-session figures and dropping the product name would remove the disclosure without affecting clients that use the calls for health checks.

Messaging. The messaging module was read in full and exercised with two clients sharing one `cid` (A16) and with a single-label rule (A28). Much of it is carefully bounded. A capability tunnel's `cid` must be 16 to 64 lowercase hexadecimal characters, which keeps mailbox names out of the reserved key space (the `reserved_cid_e2e` test covers it); upstream frames over 8 KiB are skipped rather than fatal; rules are validated to at most 200 entries and written at most once per ten seconds per stream, with the latest pending set deferred rather than dropped; acknowledgements only move forward and only up to what was sent on that stream; mailboxes are capped at 100 entries with TTLs on the counters; and the Valkey link is protected by a circuit breaker. The weakness is identity. The `cid` is asserted by the client and never proven, so with `[messaging]` enabled any client reads another user's notices and entitlement, acknowledges them away and rewrites that user's protection rules persistently (RVPN-01-102). The content itself is unsigned, so a writer to the store controls what every client is told, and one oversize broadcast entry ends every control stream until the next publish (RVPN-01-111). Red Team Ltd also observed that stored rules are loaded only after the client's first upstream frame, so streams opened immediately after a reconnect are judged without the user's rules; that is the safer direction for allow rules but not for a user's own deny entries. The verdict is that `[messaging]` should not be enabled on a public exit until the client proves ownership of its `cid`. It is worth stating what the module does not do: the exit enforces no tier, quota or entitlement. The `entitlement` message only relays the ledger's view to the client, and the usage rows are informational, so whatever enforcement exists happens in the clients or the cashier, outside this work package.

Protection. The bundle pipeline is the strongest security mechanism in the tree. `meta.json` is verified with Ed25519 before anything else is parsed; every data file is bound by SHA-256 and size; the address tables must be sorted, non-overlapping and well formed; bit names come from the signed table; a bundle older than the installed one is refused; the version written by the publisher must equal the SHA-256 of `meta.json`; and a failed verification keeps the previous bundle. The `bundle_reload_e2e` test exercises stale and mismatched bundles, `no_bundle_e2e` the honest `enabled: false` status, and A16 the verdict framing on the wire. The `controller` and `malware` floor survives `categories_off`. Verdicts reach the client as a compact frame at the head of each `blk1` stream, and a client can skip judgement for one stream with the user-allowed flag (A16), which is the intended escape hatch for a user's own exceptions; the exit counts those streams separately, so the operator can see how often the protection is bypassed by choice. Three weaknesses remain. The default `allow_hosts` exempts every name under two shared hosting suffixes, so a phishing or malware host on those platforms is never blocked

even when listed (A16 V0), and user rules accept single labels that exempt whole name spaces, floor included (A28; RVPN-01-115). The judge fails open until the first bundle loads, and rollback protection does not survive a restart (RVPN-01-120). And because the rules that drive the judge are writable by any client that knows a `cid`, the module inherits the identity problem of messaging (RVPN-01-102). With the allow lists narrowed, a persisted high-water mark and proven identities, the module would provide what it promises.

Statistics, logging and privacy. The statistics pipeline is technically sound. Rows are queued without ever delaying a tunnel, shipped in batches over TLS to an instance pinned to a configured CA, every client-supplied field is neutralised for tabs, line breaks and C0 controls (a unit test pins it), the client address is a parsed `IpAddr` ↪ , and RESP commands are length-prefixed, so client input can neither split a row nor add one. Two lists receive rows: `stats:cid`, one presence row per tunnel open with timestamp, client address, `cid`, time zone, app, operating system, SNI, kind and node name, and `stats:tunnel-closed`, one usage row per close with the times, the duration and the bytes in each direction. Rows are dropped once the queue is full, for example while the store is unreachable; the drop counter exists but, contrary to the README, is not exported (RVPN-01-124), and the rows' consumers and retention lie outside the tree (Appendix D). What the pipeline records, and on whose word, is the problem. Presence rows are created by anyone who can send an HTTP request carrying `?cid=`, without a handshake, and usage rows are attributed to whatever `cid` a client asserts (RVPN-01-104, A12). Unbounded header fields and a queue bounded in rows rather than bytes leave a large worst case if the stats writer stalls. By default every tunnel open writes a stable device identifier together with the client address, time zone, operating system and the SNI of the first connection to a central store, and at the `info` level the journal records the same tuple for every MUX2 tunnel and control call; the `warn` level, the production default, still records the destination of every failed MUX2 connect (RVPN-01-110). The protection module states that a node "must not record where anyone went" and honours that in its own block logging behind `debug_log_destinations`; the rest of the server does not yet follow the same rule. Destination names also reach the journal with line feeds intact, which A11g used to forge tunnel-open lines. For a product whose users face a state adversary, the records are the most valuable target after the traffic itself. Red Team Ltd recommends minimising them to what billing needs, attributing them to a server-generated session and a proven identity, replacing addresses with coarse derived values, and escaping every client-supplied string before it is logged.

Configuration, secrets and the metrics endpoint. Configuration handling is mostly careful. A malformed file is an error rather than a set of silent defaults; key lengths are checked; Valkey connections with TLS must name a server and a CA, and a client certificate requires its key; the protection module refuses to start with an empty brand list or a malformed public key; and the metrics endpoint refuses any non-loopback bind unless `allow_public` is set, which unit tests cover and A21 confirmed (the endpoint itself answers any bytes with the full page, which is acceptable on loopback). The weaknesses are defaults and secret hygiene. `proxy_protocol` defaults to `auto` in the common case (RVPN-01-105), the host gate defaults to open (RVPN-01-101), unknown keys are ignored silently, `deny_cidrs` replaces rather than extends (RVPN-01-117), and the default `allow_hosts` is far broader than the control plane it is meant to protect (RVPN-01-115). The example configuration carries a complete keypair, the private key may be passed on the command line, and a mismatched public key is only warned about (RVPN-01-119). The Valkey password reached the start-up log through a derived `Debug` until commit `4fb070c`; the fix was verified, and two other configuration structures that derive `Debug` over secrets should receive the same treatment before a future debug print exposes them (RVPN-01-118).

Configuration defaults in review. Read setting by setting, `config.example.toml` and the built-in defaults give the following picture. Secure as shipped: the metrics endpoint (`127.0.0.1:9090`, clamped without `allow_public`), the destination policy when `deny_cidrs` is left commented, `[protection]` and `[messaging]` off, `debug_log_destinations` ↪ off, shedding on with an eight-point hysteresis, the epoll relay, and TLS required for the stats link and the default for the other Valkey links. Insecure or fragile as shipped: the example keypair (RVPN-01-119), the empty `allowed_upgrade_hosts` (RVPN-01-101), `cids = true` with its per-device records (RVPN-01-110), the default `allow_hosts` (RVPN-01-115), and two settings whose behaviour differs from what an operator would expect, namely `proxy_protocol`, which becomes `auto` when the key is missing (RVPN-01-105), and `deny_cidrs`, which replaces rather than extends the defaults (RVPN-01-117). Unknown keys are ignored by design so that new binaries run against old files; a start-up line listing every ignored key would keep that property while making misspellings visible.

io_uring backend. The Linux-only io_uring relay could be assessed statically only; its `unsafe` blocks were outside the scope of this work package. Its logic already shows that it hands tunnels over before any usage

accounting starts and queues data without back-pressure or a lifetime limit (RVPN-01-121). The backend is off by default and should stay off until those properties match the tokio relay.

Build, tests and supply chain. The release build, the full test suite and clippy all succeeded on `3012aa6` (A01 to A03), and the library tests passed again on `4430a1d` (A23) and, with the two new end-to-end tests, on `147ab5e` (A45). The supply-chain posture has a solid base. `Cargo.lock` resolves 261 crates from crates.io, every one with a checksum, all vendored and resolved offline through `.cargo/config.toml`, with no git or unpinned sources; the release profile aborts on panic and is built with full LTO. `cargo audit` reported four advisories against rustls 0.23.42 and one unmaintained crate (RVPN-01-122); eleven crates are present in more than one version, among them three major versions of `rand`. The test suite is good at what it covers: parser bounds, cancellation accounting, bundle verification and the messaging invariants each have targeted tests, and the end-to-end binaries run a real server against a fake Valkey. At `4430a1d` it did not cover the properties this report found wanting, such as the bytes a non-client receives, the lifetime of relays after a cap or a carrier close, behaviour under descriptor exhaustion, or ownership of a `cid`, and the one end-to-end test that compares against a pre-branch binary is ignored by default; the remediation pass added tests for several of them, among them `mux_stall_e2e` and `udp_filter_e2e`, while the bytes a non-client receives and the ownership of a `cid` remain untested. Measured against NIST SP 800-218 SSDF v1.1 and SLSA v1.0, the gaps are those of process rather than code. The tree contains no CI configuration, so nothing runs the tests, clippy or `cargo audit` on every change (SSDF PW.7, PW.8 and RV.1). No software bill of materials is produced, although `Cargo.lock` makes a CycloneDX 1.7 or SPDX 3.0.1 document cheap to generate (SSDF PS.3). No build provenance or signature accompanies releases (SLSA Build L1 and above). `build.rs` stamps the binary with the output of `git` and `date`, so two builds of one commit differ and cannot be compared byte for byte. The crate targets the 2021 edition; moving to the 2024 edition is not a security fix but keeps the code on the maintained language baseline. Where the product is placed on the EU market, Regulation (EU) 2024/2847 already requires the manufacturer to report actively exploited vulnerabilities and severe incidents from 11 September 2026, and its remaining obligations, including a software bill of materials and a coordinated vulnerability disclosure policy, apply from 11 December 2027; the repository contains no security policy or disclosure contact yet.

Publication hygiene and fix verification. Both remediation commits made during the engagement were compared with their parents by reading the repository's objects directly. Commit `4fb070c` replaced the derived `Debug` of `valkey::Config` with a redacting implementation and a regression test, and moved three fixture values in `src/config.rs` and `src/bin/mux_client.rs` to documentation ranges; the fix of RVPN-01-118 was verified. Commit `4430a1d` replaced the fixtures that resembled operator infrastructure in `src/http.rs` and `src/proxy.rs`, and the library tests still pass on it. The scrub scan S02 nevertheless found publicly routable addresses of unknown origin, third-party resolver and CDN addresses, placeholder addresses and a hard-coded server public key elsewhere in the tree, including the address the earlier review had flagged as possible infrastructure (RVPN-01-123). The replaced values also remain in the published history. If any of them belongs to the fleet, it should be rotated. The two fixture commits of the remediation pass are covered below and in RVPN-01-123.

Re-verification of the earlier static review. The earlier static review listed seventeen hypotheses, RVPN-SRV-001 to RVPN-SRV-017. Red Team Ltd re-verified each against `4430a1d` and the dynamic evidence, and none had to be dropped entirely. Twelve were confirmed and carried over with dynamic proof: the probe oracle (now RVPN-01-101), the `cid` bearer token (RVPN-01-102), the WebSocket reassembly (RVPN-01-103), forged statistics rows (RVPN-01-104), the `proxy_protocol` default (RVPN-01-105), the CDN headers (RVPN-01-106), the connection cap (RVPN-01-107), the accept-loop exit (RVPN-01-108), the control-channel disclosure (RVPN-01-109), the logging and records (RVPN-01-110), unsigned messaging content (RVPN-01-111) and the UDP source filter (RVPN-01-112). Parts of these were corrected on the way: the eager 16 MiB allocation per declared WebSocket frame did not materialise as resident memory (A09a), whereas the reassembly did. The yamux part of the amplification hypothesis was confirmed and strengthened, because A26 showed that pinned relays outlive the carrier (RVPN-01-113, now High); its statistics-queue part was confirmed statically and folded into RVPN-01-104. The destination-policy hypothesis was narrowed: the deny list held against every private and loopback form tried, while the embedding notations, the node's own addresses and the replace-not-extend semantics remain (RVPN-01-117). The key-handling hypothesis was split into the logged Valkey password, fixed in `4fb070c` (RVPN-01-118), and the example keypair and command-line key, which remain (RVPN-01-119). The fail-open and rollback hypothesis was confirmed (RVPN-01-120), the advisories were confirmed by `cargo audit` (RVPN-01-122), and the review's open question about fixture addresses became RVPN-01-123. Two of the review's positive observations were qualified: the RESP readers in `valkey.rs` do not bound simple-string

lines, and the destination policy does not unwrap embedding forms. Five issues were not in the earlier review: the passive wire image (RVPN-01-114), the breadth of the protection allow lists (RVPN-01-115), the specification departures (RVPN-01-116), the io_uring back-pressure (RVPN-01-121) and the misreporting metrics (RVPN-01-124).

Re-verification of the remediation pass. After version 1.0 the maintainers pushed two fixture commits (`60cf688`, `f202f2d`), twenty remediation commits from `171aed6` to `3398920`, and the housekeeping commit `147ab5e`, which removed the superseded static review. Red Team Ltd compared each commit with its parent by reading the repository's objects, traced each change to the code it touches, rebuilt `147ab5e` together with a variant whose cap was lowered to 20 seconds, and repeated the shapes that had demonstrated each issue (A29 to A47, S04). The maintainers' notes on their own re-runs served as pointers only; as they suggested, the UDP test was repeated with the target on `::1` and the stranger on 127.0.0.1, which is what separates address-dependent filtering from none on a single loopback host. Where the version 1.0 evidence for an issue had been static, or the fixed code takes a different path from the one first exercised, the new test was also run against the `3012aa6` build, so that a pass on `147ab5e` shows the fix rather than a test that cannot fail (A32c, A32d, A43b). Every behaviour that was tested matched what the commit messages claim. Several messages name the redsocket_proteus commit a change was ported from; whether WP2 carries the same fixes is a matter for the WP2 report. The outcome is 8 issues fixed, 9 partially fixed and 7 open, recorded in a Fix Note or Note under each finding and summarised by area below.

Resource exhaustion after the remediation. This area improved most. The WebSocket path now streams data into the tunnel through a 16 KiB pipe instead of reassembling messages, so the A09 shapes that had driven the resident set to 923.6 MiB left it at 12.2 MiB, and a complete MUX2 session ran inside one WebSocket message that never set `FIN`, a session that `3012aa6` could not even open (A43, A43b, RVPN-01-103). The v1 relay, the WebSocket reader and the `noise_stream` pump now end with their connection, so the cap ends tunnels on every carrier, and the usage row, still written at the cap, now matches the relay's end; silent MUX2 and WebSocket clients no longer keep a descriptor after the cap, which the old build still did in the same tests (A32, A32d, RVPN-01-107). The accept loop waits out descriptor exhaustion, a failed bind stops start-up, the PROXY phase has a 5-second deadline and the opening exchange a 10-second one (A30, A31, A33, RVPN-01-108). MUX2 stream tasks now belong to their carrier, and a target that accepts nothing for 60 seconds ends its stream (A38). What remains is the footprint of an open carrier: the yamux windows and the limit of 256 streams are unchanged, carriers per peer address are not limited, and the shedding decision does not count relays, so a key holder that keeps carriers open can still make each of them hold on the order of 90 MiB, now released within a minute of each stall rather than never (RVPN-01-113). The recommended per-peer and global connection ceilings were not added. The io_uring backend is unchanged and should stay disabled (RVPN-01-121).

Attribution, records and logging after the remediation. `proxy_protocol` now defaults to `disabled` when an `[http]` table omits the key, `auto` strips a header without trusting it, `UNKNOWN` and `LOCAL` headers yield the socket peer, and the PROXY phase has a deadline (A31, RVPN-01-105, RVPN-01-116). Operators who relied on `auto` behind a load balancer need `required` to keep client addresses. The CDN real-IP headers are still honoured from any peer, so a direct client still chooses the address recorded for it (A44, RVPN-01-106). Statistics rows now cut client-supplied fields to 64 characters, `sni` to 253, which bounds both a row and the queue, but a plain HTTP request with `?cid=` still creates a presence row, and A44 recorded one under a forged `X-Forwarded-For` address (RVPN-01-104). Destination names with control characters are now refused, and at the default level no line names a `cid`, a client address or a destination (A35). At `debug` the header fields are still logged unescaped, so a time-zone field with a line feed forged a journal line (A35b); two `warn` lines, limited to one a minute, still name a host or a client address; and the statistics records and the `?cid=` request line are unchanged (RVPN-01-110). The direction is right, and the remaining work is to treat every client string as untrusted at every log level and to minimise the records themselves.

Control plane, protection and destination policy after the remediation. `_rs.stat` and `_rs.config` now return the session's `cid` and the server's public key only, and the counters remain on the loopback metrics endpoint (A34, RVPN-01-109). The `cid` is still a bearer token: no commit touched it, and a session asserting a `cid` was again replayed state that another session had written (A36, RVPN-01-102). An oversize store entry is now skipped and counted instead of ending every control stream, while store content remains unsigned and a forged entitlement broadcast was again delivered (A36, RVPN-01-111). User rules now need two labels, but a public suffix such as `co.uk` was accepted, the default `allow_hosts` still exempt two shared hosting suffixes,

and allowed names still bypass the address verdicts (A39, RVPN-01-115); protection still fails open until the first bundle loads (RVPN-01-120). The destination policy now judges the IPv4 address inside IPv4-compatible and NAT64 well-known-prefix forms (A40), and the README and example now say that `deny_cidrs` replaces the defaults; the local-use NAT64 prefix, the node's own addresses and a floor that configuration cannot remove are still missing (RVPN-01-117). UDP associations now accept datagrams only from the addresses of their own targets, as recommended (A37, RVPN-01-112).

Front, wire image and the Proteus transport. The front still answers any `Upgrade` with a 101 (A30), and the wire image of a session is unchanged (RVPN-01-101, RVPN-01-114); of the specification departures only the PROXY part and the WebSocket control-frame length were addressed (RVPN-01-116). The development team's remediation for the carrier's signature and the active-probing oracle is not a change to this server but the Proteus transport (redsocket_proteus, WP2), which replaces the HTTP front; redsocket_http is retained for legacy clients. Red Team Ltd's assessment of these issues therefore stands for every exit that still serves the HTTP front, and their practical exposure now depends on how quickly the legacy clients that need the front are retired and the front is withdrawn from public exits. The Proteus transport itself is assessed in the WP2 report, and nothing in this report should be read as a statement about it.

Configuration, secrets, metrics and publication after the remediation. The example configurations now ship placeholders, and the server refuses a placeholder or the former example key as its private key, as a rotation key and on the command line (A41, RVPN-01-119). The command-line option itself, the case-sensitive comparison that let the upper-case form of the former key through, the warning-only public-key mismatch and the missing zeroisation remain. `config::Config` no longer implements `Debug` and the statistics password is redacted, which completes RVPN-01-118. The handshake-failure counter, the uptime, the dropped-row counter and the shedding wording now match the code (A42, A47, RVPN-01-124). The two fixture commits removed the values that resembled operator infrastructure and the hard-coded `mux_client` key, while resolver, boundary, third-party and placeholder literals and two CloudFront-style names remain (S04, RVPN-01-123). `Cargo.lock` was not touched, so the rustls advisories stand (A46, RVPN-01-122), and the library tests and the two new end-to-end tests pass on `147ab5e` (A45).

Standards conformance in summary. Appendix C applies OWASP ASVS 5.0.0 control by control. Of its 170 rows at `147ab5e`, 52 pass, 47 pass in part, 23 fail and 48 were not assessed, mostly because the component has no browser, file, password or token surface; at `4430a1d` the figures were 46, 43, 33 and 48. The remaining failures cluster in authorization (V8), where every data-specific and field-level requirement fails on the asserted `cid`; per-peer limits and their documentation (2.3.2, 2.4.1, 13.1.2, 13.1.3); trusted client attribution (4.1.3, 15.3.4); data protection (V14); and the supply-chain and inventory requirements (V15, 16.1.1). Cryptography (V11) and secure communication (V12) pass almost entirely, with the missing post-quantum migration plan (11.1.4) as the notable gap. Mapped to the OWASP Top 10:2025, the issues fall under A01 Broken Access Control (RVPN-01-102), A02 Security Misconfiguration (RVPN-01-101, -105, -115, -117), A03 Software Supply Chain Failures (RVPN-01-122, -123), A04 Cryptographic Failures (RVPN-01-119), A05 Injection (RVPN-01-109, -110), A06 Insecure Design (RVPN-01-113, -114), A07 Authentication Failures (RVPN-01-102, -104), A08 Software or Data Integrity Failures (RVPN-01-111, -120), A09 Security Logging and Alerting Failures (RVPN-01-110, -124) and A10 Mishandling of Exceptional Conditions (RVPN-01-108). Against the protocol specifications, the Noise layer conforms to the framework's NK pattern, the SOCKS-derived requests follow the trimmed RFC 1928 format that the README documents, and the HTTP, WebSocket and PROXY handling departs from RFC 9110, RFC 9112, RFC 6455 and the PROXY protocol text as listed in RVPN-01-116. No part of the code targets a superseded protocol revision; the 2021 language edition is the only dated baseline, and this report applies ASVS 5.0.0 and CVSS v4.0 where the earlier review used ASVS 4.0.3 and CVSS 3.1.

Operational monitoring. The server already exports most of the signals an operator needs: handshake successes and failures, destinations refused by policy, shedding state and refusals, protection verdicts per brand, bundle age and reload outcomes, messaging link state and rejected rules. Three additions would make attacks of the kinds in this report visible. Counting connections that receive a 101 but never complete a handshake would expose probing (RVPN-01-101); the handshake counter itself is now correct (RVPN-01-124). Exporting the number of live relay tasks against live carriers, and the process's open descriptors, would expose stalled relays and descriptor pinning before they exhaust the node (RVPN-01-108, RVPN-01-113). A gauge of the statistics queue length in bytes would expose a stalled stats writer (RVPN-01-104). Alerting on `redsocket_protection_unchecked_total` and on the bundle age would catch the fail-open window of RVPN-01-

120.

Assessment against the threat model. Measured against the capabilities of the primary adversary, the component's posture at `147ab5e` remains uneven. Observing and fingerprinting at national scale: the wire image of every direct session is distinctive (RVPN-01-114) and the front confirms an exit to a single probe (RVPN-01-101), so blocking requires neither the key nor any client secret; this is the area where the HTTP front falls furthest short of the product's purpose, and the development team's answer is to replace the front with the Proteus transport of WP2. Active probing with the public key: the control channel no longer ranks exits by load or names the product (RVPN-01-109), although an unknown subcommand still draws a recognisable reply. Hostile networks and man-in-the-middle positions: Noise NK authenticates the exit, so an on-path party cannot read or alter tunnel traffic unless the exit's private key is known; the server now refuses the former example key, so the copy-paste risk is limited to exits started with that key before the fix and not yet rotated (RVPN-01-119); the Valkey connections pin their CA. Seizure of devices and nodes: a seized device yields the user's `cid`, which with `[messaging]` enabled is enough to act as that user (RVPN-01-102); a seized node or stats store yields per-device connection histories (RVPN-01-104, RVPN-01-110). Impersonation of configuration, bootstrap or relay infrastructure: the protection bundle is signed and resists it; mailbox, broadcast and rules content is not and does not (RVPN-01-111). Compromise of distribution: the vendored, checksummed dependency set is a good base, but there is no provenance, signing or bill of materials to show what was built from what. For the ordinary application attacker, the destination policy holds against the usual server-side request forgery forms with the remaining exceptions of RVPN-01-117. Resource exhaustion, the dominant risk in version 1.0, is now bounded: resources end with their connections and silent connections close within seconds, and what remains is the footprint of MUX2 carriers that a key holder keeps open (RVPN-01-113).

Interim guidance for current deployments. For the issues that remain, operators can reduce exposure through configuration: set `allowed_upgrade_hosts` on every exit and serve the HTTP front only for as long as legacy clients need it; set `proxy_protocol` explicitly in every `[http]` table and use `required` only where the port is reachable solely through the load balancer; leave `[messaging]` disabled on public exits; replace the default `allow_hosts` with the exact host names the clients use; keep the built-in deny list rather than uncommenting the example; keep the log level at `warn` and `debug_log_destinations` off; run the service with an explicit `LimitNOFILE` and a cgroup memory limit so that shedding can see memory; keep `io_backend` at `epoll`; never pass keys on the command line; and rotate the key of any exit that was ever started with the example keypair.

Engineering practices worth keeping. Several practices in the tree deserve to be kept and extended. Security invariants are encoded as tests rather than comments: the decoy and shedding responses must stay byte-identical, a bare line feed must not smuggle a header, reserved `cid` values must not reach the mailbox namespace, hostile RESP lengths must be errors rather than allocations, a missing bundle must report `enabled : false`, and stale bundles must be refused. Comments record the threat reasoning behind defaults, and the metrics clamp refuses a public bind instead of merely warning. Defaults for the newer modules are conservative: `[protection]` and `[messaging]` are off, destination logging is off, the metrics endpoint is loopback-only and the stats link is TLS-only. Dependencies are vendored and locked, and the release profile aborts on panic instead of running on with state left inconsistent by an unwinding task. Extending the same discipline to the issues in this report, with tests asserting for example that a non-client request receives the reference server's bytes or that stream relays end with their carrier, would protect the fixes against regression.

Limitations. The assessment host runs macOS, so the io_uring backend, the cgroup memory sampling and the Linux-specific socket options were assessed statically; shedding was exercised through a scratch build that reads a simulated memory figure, and the connection cap through a scratch build with the constant lowered to 20 seconds (A18, A20, A25). Production configurations, service units, descriptor and memory limits and the Valkey deployments are not in the tree, so the likelihood of RVPN-01-105, RVPN-01-108 and RVPN-01-117 depends on the answers requested in Appendix D. The passive-classification claim of RVPN-01-114 rests on recorded bytes and the code, not on a test against a production DPI system, and the yamux window autotuning mentioned in RVPN-01-113 was not measured. No packet capture was taken in the first test session, so the off-host connection attempts disclosed in Appendix A are known from the server's logs only. The re-runs A24, A26, A27 and A28 used the release binary built from `3012aa6`, whose runtime code is identical to `4430a1d`. Statements about clients rest on the repository's own test clients; the client cores belong to WP3 to WP5. In version 1.1 the WebSocket path, which no client in the repository speaks, was exercised through a scratch relay (A32d, A43b); the exemption of names under the default `allow_hosts` suffixes was confirmed statically, because

a dynamic test would have sent a lookup through the host's resolver; and the io_uring backend and the handling of the local-use NAT64 prefix were again assessed statically.

Retest guidance. Version 1.1 repeated the shapes named in the version 1.0 guidance for every issue with a remediation commit (A29 to A47). For the issues that remain, the following re-runs will verify further fixes: A06, A27 and A30, where every non-client request should receive the reference server's bytes and the wire should show neither fixed record sizes nor cleartext lengths, or the HTTP front should be gone from public exits; A16, A28, A36 and A39, where a second client with a known `cid` but without the ownership proof should receive no mailbox and write no rules, forged store content should be refused, a public-suffix rule should be refused, and a listed name under a shared hosting suffix should be blocked; A15 and A38, where one carrier should no longer hold tens of MiB for the length of the write timeout; A07, A12 and A44, where CDN headers from untrusted peers should be ignored and non-upgrade requests should create no rows; A35b, where header fields should appear escaped at every log level; A40 extended to the local-use NAT64 prefix, and A10b, where an empty `deny_cidrs` should still refuse loopback; A41, where the command-line key and the upper-case form of the former example key should be refused; and A46 and S04, which should report no advisories and no literals outside the documentation ranges.

Prioritised recommendations. For the issues that remain at `147ab5e`, Red Team Ltd recommends the following order of work:

1. Before `[messaging]` is enabled on any public exit, give clients a verifiable identity bound to their `cid` and stop persisting rules from unproven sessions (RVPN-01-102); sign mailbox and broadcast content (RVPN-01-111).
2. Retire the HTTP front in favour of the Proteus transport as soon as the legacy clients allow, and until then set the host gate on every exit; if the front is to stay, redesign it and the wire image together as described in RVPN-01-101, RVPN-01-114 and RVPN-01-116.
3. Bound what a key holder can hold through open carriers: explicit yamux windows, a lower stream limit, carriers per peer address, relays in the shedding decision, and per-peer and global connection ceilings (RVPN-01-113, RVPN-01-108).
4. Make attribution and records trustworthy and minimal: trusted-source lists for CDN headers (RVPN-01-106), no work and no rows on the non-upgrade path, server-side session identifiers and data minimisation, and escaping of every client string at every log level (RVPN-01-104, RVPN-01-110).
5. Tighten protection: exact allow-list entries and refusal of public-suffix rules (RVPN-01-115), and a persisted bundle high-water mark (RVPN-01-120).
6. Complete the destination policy with the local-use NAT64 prefix, the node's own addresses and a floor that configuration cannot remove (RVPN-01-117).
7. Finish the key handling: remove `--private-key` or read the key from a file, compare against the former example key case-insensitively, fail on a public-key mismatch and zeroise key material (RVPN-01-119); replace the residual literals (RVPN-01-123).
8. Establish the supply-chain routine: update rustls and replace rustls-pemfile (RVPN-01-122), run tests, clippy and `cargo audit` in CI, publish a bill of materials and build provenance, add a security policy, and keep the io_uring backend disabled until it matches the tokio relay (RVPN-01-121).

Red Team Ltd would like to thank the RostamVPN maintainers for the open access to the source tree, for the quick turnaround of the two remediation commits made while the assessment was in progress and of the remediation pass that followed, whose commit messages made every change easy to trace, and for the care evident throughout the code, much of which anticipates exactly the attacks this report describes.

Appendix A: Dynamic Test Log

Environment. All tests up to S03 ran on the assessment host (macOS 26.6.2, Apple Silicon) on 2026-10-14, and the re-verification tests from A29 onward on 2026-10-15 (see the paragraph before A29); times below are host local time (UTC-7), while the server logs quoted carry UTC timestamps and therefore read 2026-10-15. Servers were configured from `config.example.toml` with a fresh keypair from `redsocket-http --keygen`, bound to 127.0.0.1 in the assigned range 20100 to 20199 and run with `RUST_LOG=debug`. Three server binaries were used: the release build of `3012aa6` from A01b (`RedSocket HTTP-Standalone v3.3.0 (3012aa6 2026-10-15)`, runtime code identical to `4430a1d`); a scratch build of a copy of `3012aa6` with the 600-second cap constant set to 20 seconds

(A18, A20, A25); and the same scratch copy with a shedding sampler that reads a simulated `used limit` figure from a file (A19). `wp1tool` is a scratch client built in the copied tree from the project's own protocol code; `probe.py`, `proxyspoof.py`, `noise_fuzz.py`, `ws_growth.py`, `fd_hold.py`, `targets.py` and the A23 to A28 helpers are scratch scripts. Loopback targets were a TCP echo (20151), a TCP sink that never reads (20152, 20179), a UDP echo (20153), a Python `http.server` reference (20150) and Redis 8.4.0 stand-ins (20106 plain, 20109 TLS with a test CA, 20183 plain).

Process handling. Every process was started with its PID recorded and stopped by PID; A22 lists the first session's processes and each re-run log ends with its own stop lines. No listener remained in the port range afterwards.

Off-host traffic. The tests targeted loopback services only. Some inputs nevertheless made the server under test attempt outbound traffic, which is disclosed here: in A10a the leading-zero form `0177.0.0.1` was resolved by the platform resolver to a public, non-documentation IPv4 address and the server made an eight-second TCP connect attempt towards it; in A10a the forms `::7f00:1` and `64:ff9b::7f00:1` were permitted by the policy and timed out after eight seconds; in A16c the poisoned rules cleared 203.0.113.5:80, a documentation address, and the server attempted to connect to it; and several test names (`evil.example`, a name under `*.cloudfront.net`, `127.1`) were looked up through the host's configured resolver, which returned no records. No packet capture was taken, so the number of segments that left the host is not known; no reply was received in any case.

Redactions. Server and bundle keys, bundle digests, the host name and the host's LAN address are replaced by placeholders; the body of the reference server's directory listing (A06c, A17), which listed its working directory outside the repository, is omitted; non-ASCII arrows and dashes in log lines are shown as `->` and `-`; long outputs are trimmed where marked.

A01, A01b: release build (19:04 and 19:23). `cargo build --offline --release` in the repository with the target directory in the scratch area, then again for the server binary alone. Result: success in 47.2 s and 45.4 s; 261 vendored crates compiled offline.

```
   Finished `release` profile [optimized] target(s) in 47.17s
exit=0
```

A02: project test suite (19:15). `cargo test --offline` on `3012aa6`. Result: pass. 110 library unit tests passed and 1 was ignored (`memory_footprint_300k_ranges_1m_domains`), the `client` binary's 2 unit tests passed, and six end-to-end binaries ran one test each and passed (`legacy_server_e2e` has one further ignored test), 118 passed in total.

```
test valkey::debug_tests::debug_output_never_contains_the_password ... ok
test result: ok. 110 passed; 0 failed; 1 ignored; 0 measured; 0 filtered out; finished in 0.98s
    Running tests/no_bundle_e2e.rs ...
test no_bundle_means_enabled_false_not_silent_clear ... ok
exit=0
```

A03: clippy (19:15). `cargo clippy --offline --all-targets`. Result: no warnings.

A04: dependency audit (19:06). `cargo audit -n` in the repository root (cargo-audit 0.22.2, 1,329 advisories loaded, 263 crates scanned). Result: four vulnerabilities in rustls 0.23.42 (RUSTSEC-2026-0285, -0346, -0348, -0349) and one unmaintained warning for rustls-pemfile 2.2.0 (RUSTSEC-2025-0134); see Appendix B.

```
error: 4 vulnerabilities found!
warning: 1 allowed warning found
```

A05, A05b, A05c: scratch builds (19:15, 19:28, 19:35). Release builds of the scratch copy (`3.3.0-wp1scratch`) with the cap patch and `wp1tool`, then with the shedding sampler patch, then `wp1tool` alone. Result: success.

A06: probing oracle (19:16 to 19:20). `probe.py 20100` (server A, default), `probe.py 20102` (server B, `allowed_upgrade_hosts = ["front.example"]`), `probe.py 20150` (Python reference), and a re-run of three cases that the first pass aborted on a TCP reset (A06d). 33 request shapes in total. Result: any `Upgrade` header receives a 101 from server A; the gate works on server B except for duplicate `Host` headers (RVPN-01-101, -116).

```
A06a upgrade-foo -> HTTP/1.1 101 Switching Protocols | Connection: Upgrade ttfb=0.000s open
A06a upgrade-ws-bad-key -> HTTP/1.1 101 ... Sec-WebSocket-Accept: w7cyq/iodDhF6MfTw9LCsDqUyBQ=
```

```
A06a plain-get -> HTTP/1.1 404 Not Found | Content-Type: text/plain | Content-Length: 10 closed
A06a tls-clienthello -> HTTP/1.1 400 Bad Request | Content-Length: 0 ttfb=5.003s, then reset
A06b gate-host-wrong -> 404; gate-host-right -> 101; gate-no-host -> 404
A06b gate-two-hosts -> 101 (first Host front.example); reversed order -> 404
A06c reference -> HTTP/1.0 200 OK | Server: SimpleHTTP Python/3.15.0 | Date: ... (body omitted)
A06c tls-clienthello -> no bytes within 7.0s
A06d oversize-headers -> HTTP/1.1 400 Bad Request (17,036-byte header block), then reset
A06d proxyline-as-request -> HTTP/1.1 101 Switching Protocols
```

A07: PROXY protocol and CDN headers (19:17). `proxyspoof.py` against server A (`disabled`, 20100), server C (`auto`, 20104) and server F (key omitted, 20112), each request tagged with a unique `User-Agent` and correlated in the debug log; 25 cases. Result: CDN headers honoured from any peer; PROXY honoured in `auto` and when the key is omitted; `UNKNOWN` yields `0.0.0.0` (RVPN-01-105, -106).

```
A07a xff [HTTP] GET /?cid=wp1spoofA-xff from edge=127.0.0.1 real_ip=203.0.113.9
A07a cfva6 [HTTP] GET /?cid=wp1spoofA-cfva6 from edge=127.0.0.1 real_ip=2001:db8::12
A07a xffbad ... real_ip=127.0.0.1 ... xff="not-an-ip"
A07a proxy-v1 [HTTP] PROXY TCP4 from edge=127.0.0.1 real_ip=127.0.0.1 (disabled: line read as request)
A07b v1unknown [HTTP] GET /?cid=wp1spoofC-v1unknown from edge=0.0.0.0 real_ip=0.0.0.0
A07c banner proxy_protocol: Auto (server F, [http] table without the key)
A07c proxy-v1 [HTTP] GET /?cid=wp1spoofF-v1 from edge=203.0.113.7 real_ip=203.0.113.7
```

A08: malformed handshakes (19:17). `noise_fuzz.py 20100 78540 20101` (78540 being server A's PID): ten `msg1` shapes after a 101, each with a two-second read window. Result: no crash; malformed messages closed by the server; a partial `msg1`, a single byte and silence left the connection open; `redsocket_hs_fail` rose by 17 for 10 attempts (RVPN-01-108, -124).

```
hs_fail before: 17
msg1 len=0 sent= 2 B server_closed=True bytes_back=0 pid_alive=True
msg1 len=65535 random sent=65537 B server_closed=True bytes_back=0 pid_alive=True
msg1 len=48, only 10 sent sent= 12 B server_closed=False bytes_back=0 pid_alive=True
nothing after 101 sent= 0 B server_closed=False bytes_back=0 pid_alive=True
hs_fail after: 34 (delta 17)
```

A09: WebSocket growth (19:18). Server G (default policy, 20114). A09a: `ws_growth.py 20114 79841 declare 8` (79841 being server G's PID), eight connections each declaring a 16 MiB `FIN=0` frame without payload. A09b: `ws_growth.py 20114 79841 stream 4 512`, four connections streaming masked 1 MiB fragments with `FIN` never set, no handshake. Result: declared frames cost 1.2 MiB; streamed fragments were held in memory (RVPN-01-103).

```
A09a before rss=9.8 MiB; after 8 declarations rss=11.0 MiB
A09b sent 132 MiB rss=310.6 MiB; sent 388 MiB rss=830.6 MiB
A09b after 512 MiB of FIN=0 fragments in 1.1s: rss=923.6 MiB (includes a 1 s pause)
A09b liveness: new connection answered 'HTTP/1.1 404 Not Found'
A09b 2 s after closing all 4 connections: rss=923.6 MiB
```

A10: destination policy (19:18). A10a: `legacy_client` v1 CONNECTs (domain address type) through server A (default policy) to 20 destinations on the metrics port 20101 and four `_rs` calls. A10b: four destinations through server K (`deny_cidrs = []`, metrics on 20121). Result: refused as unroutable: `127.0.0.1`, `localhost`, `localhost.` ↪ , `127.1`, `0x7f000001`, `2130706433`, `0.0.0.0`, `::1`, `::ffff:127.0.0.1`, `::ffff:7f00:1`, `10.0.0.1`, `169.254.1.1`, `192.0.2.1`, `100.64.0.1`, `224.0.0.1`, the host's LAN address and `127.0.0.1:20151`; not refused: `::7f00:1` and `64:ff9b::7f00:1` (connect attempted, timed out) and `0177.0.0.1` (read as a public address, connect attempted, timed out); with `deny_cidrs = []` the metrics endpoint was reached through the tunnel (RVPN-01-117).

```
A10a [HTTP/Noise] 127.0.0.1 tunnel error: refused unroutable destination 2130706433:20101 (1 address(es),
    ↪ none routable from an exit)
A10a [EGRESS] connect to [::7f00:1]:20101 timed out after 8s
A10a [EGRESS] connect to [64:ff9b::7f00:1]:20101 timed out after 8s
A10a [EGRESS] connect to <public address>:20101 timed out after 8s (input 0177.0.0.1)
A10a _rs.bogus:1 -> HTTP/1.1 404 unknown _rs subcommand: bogus
A10b 127.0.0.1:20121 -> [+] through-tunnel response: HTTP/1.1 200 OK
A10b ::1:20121 -> Connection reset by peer (metrics bound to 127.0.0.1 only)
```

A11: control channel and request edges (19:23). `wp1tool v1` and `wp1tool mux` against server A (default) and server K (deny off). A11a/b: `_rs.stat` with an injecting META `cid`, `_rs.config`, `_rs.notify` on v1; A11c: the

same on MUX2; A11d: v1 relay to the echo target; A11e: six MUX2 stream-request edge cases; A11f: eight malformed transport and yamux inputs; A11g: a line feed in a destination name. Result: node figures and banner disclosed, JSON injection into the caller's reply, journal line injection; all malformed requests refused without a crash (RVPN-01-109, -110).

```
A11a v1 <- {"cid":"evilcid","injected":true,"x":"","active_tunnels":0,"total_tunnels":0,"bytes_up":0,"
    ↪ bytes_down":0}
A11b v1 <- {"noise_pubkey":"<server public key>","carriers":["http"],"server":"redsocket-http"}
A11d v1 <- 23 B plaintext: TARGET\x0ahello-through-v1
A11e CONNECT length 600 / 1-byte request / ATYP 9 / empty domain -> stream closed before a reply
A11e ATYP 4 ::1 and ::ffff:127.0.0.1 on server A -> refused unroutable destination
A11f garbage, oversize and empty transport frames -> server closed the connection; server alive
A11g [... WARN redsocket_http::mux] [MUX2] target connect evil.example
A11g [2026-10-15T00:00:00.000Z WARN redsocket_http::mux] [MUX2] tunnel open: ... cid=0000deadbeef0000 ...
    ↪  ip=203.0.113.250:1 failed: ...
```

A12, A12b: statistics rows (19:18 to 19:20). Server E (20110) with `[stats.valkey]` pointed at the TLS Redis stand-in (20109, test CA). (1) One non-upgrade request with `?cid=` and a spoofed `X-Forwarded-For`; (2) 200 such requests with distinct values; (3) one genuine `legacy_client` session; the rows were read back with `redis-cli --tls`. Result: presence rows created without a handshake, indistinguishable from a genuine row (RVPN-01-104). The first read-back failed on shell quoting and was repeated in A12b.

```
curl -H "X-Forwarded-For: 203.0.113.9" "http://127.0.0.1:20110/?cid=abcdef0123456789" -> HTTP/1.1 404 Not
    ↪  Found
200 requests: 1.228 s total; LLEN stats:cid -> 202
2026-10-15T02:18:30Z 203.0.113.9 abcdef0123456789 (4 empty columns) tunnel <host>
2026-10-15T02:18:31Z 127.0.0.1 1111222233334444 (4 empty columns) tunnel <host> (genuine legacy session)
stats:tunnel-closed rows: none (the genuine session's destination was refused by policy)
```

A13: descriptor exhaustion (19:18). Server I (20118) started with `ulimit -n 256`; `fd_hold.py 20118 300 5 87682` (87682 being server I's PID) opened idle post-101 connections. Result: the process exited (RVPN-01-108).

```
  200 attempted: 101=200 failed=0 server_alive=True
conn 245 failed: [Errno 54] Connection reset by peer
conn 246 failed: [Errno 61] Connection refused
  300 attempted: 101=245 failed=55 server_alive=False
Error: Too many open files (os error 24) (last line of the server log)
connections still open from the client side: 0 of 245
```

A14: UDP association (19:23). `wp1tool mux-udp` through server K (deny off) to the UDP echo (20153), plus two third-party datagrams from unrelated loopback sockets (A14a); a `FRAG` datagram (A14b); a domain-form target and the close marker (A14c); a loopback target through server A under the default policy (A14d). Result: third-party datagrams delivered; `FRAG` dropped; close marker honoured; loopback target refused silently (RVPN-01-112).

```
A14a udp <- 31 B from 127.0.0.1:20153: echo:ping-from-client
A14a udp <- 33 B from 127.0.0.1:61365: INJECTED-by-third-party
A14a udp <- 24 B from 127.0.0.1:61347: INJECTED-again
A14b udp -> [len 20] payload fragmented ... hold window over (no echo)
A14c udp -> len 0 close marker -> stream closed by server after the marker
A14d no echo; redsocket_egress_refused_unroutable 20
```

A15: stalled MUX2 streams (19:23). `wp1tool mux-window` through server K: one carrier, 256 streams to the sink (20152), 2 MiB offered per stream, 12-second hold. Result: 786,432 bytes accepted per stream; resident set +92.5 MiB and not released after the carrier closed (RVPN-01-113).

```
before: rss=10.6 MiB t+3s ... t+18s: rss=103.1 MiB
window: the server accepted 201326592 B in total across 256 stalled streams
after the carrier closed: rss=103.1 MiB
```

A16, A16c: messaging impersonation (19:33 to 19:35). Server D (20107, `[messaging]` and `[protection]` on, `deny_cidrs = []`, fixture bundle from `protection::fixture` listing `evil.example` and `cloudfront.net` and three IPv4 ranges) against the plain Redis stand-in (20106). The victim `cid` `c1d00000000000aa` was seeded with two unread

notices, a ledger record and an allow rule; a second client with the same `cid` replayed, acknowledged and rewrote (A16); forged and oversize broadcast entries were written to the store (A16b); verdicts and cache recovery were checked afterwards (A16c). Result: disclosure, acknowledgement and rules poisoning; forged broadcast delivered; oversize entry ends control streams until the next publish (RVPN-01-102, -111, -115).

```
V0 evil.example:443 BLOCK reason="domain"; 203.0.113.5:80 BLOCK "phishing"; 198.51.100.7:80 BLOCK "
    ↪ controller"
V0 <label>.cloudfront.net:443 CLEAR (0x00)
X ctl <- notices seq 1 and 2, entitlement {"paid_until":"2027-01-01T00:00:00","state":"paid","tier":"pro
    ↪ "}
X ctl -> {"ack":2}; ctl -> {"rules":{"allow":["evil.example"],"deny":[],"categories_off":65535}}
   store: msg:<cid>:last=2; rules:<cid> = allow ["evil.example"] deny [] categories_off 65535
V1 victim's live stream <- frame 5 {"seq":3,...,"type":"rules","payload":{"allow":["evil.example"],...}}
V2 immediate reconnect: verdicts still BLOCK (rules not yet loaded)
A16b ctl <- {"seq":50,...,"type":"entitlement","payload":{"state":"banned","tier":"free"},"scope":"
    ↪ broadcast"}
A16b 70,000-byte broadcast entry -> control stream ended: early eof (also for a fresh client after HDEL)
A16c after PUBLISH msg *: evil.example:443 CLEAR; 203.0.113.5:80 CLEAR; 198.51.100.7:80 BLOCK "controller
    ↪ "
A16c seeded rules restored -> evil.example:443 BLOCK; 203.0.113.5:80 BLOCK
```

A17: repository client end to end (19:26). The repository's `client` binary in MUX2 mode (SOCKS5 on 127.0.0.1:20160) through server K. Result: a TCP request to the reference server returned HTTP 200 (body omitted), and a UDP ASSOCIATE echo round trip succeeded.

A18: connection cap (19:27). Server H (20116, 20-second cap build, deny off, statistics to the TLS Redis). A18a: v1 tunnel to the echo, idle 30 s, then a send; A18b: the same over a MUX2 stream; A18c: WebSocket path with one fragment, then silence; A18d: post-101 silence. Result: the v1 relay outlived the cap while the usage row was written at the cap (RVPN-01-107).

```
A18a t+10s active_tunnels 1; t+25s active_tunnels 0; [HTTP] Connection timeout (20s-PATCHED)
A18a [02:27:35.608] v1 -> after-hold payload (9 B): after-cap [02:27:35.611] v1 <- 9 B plaintext: after-
    ↪ cap
A18a stats:tunnel-closed: 1792031243 cafe0000000000aa 1792031223 19 5 12 <host>
A18b [02:27:51.748] mux: yamux connection closed by the server
A18c t+25s client view=server closed; server-side ESTABLISHED sockets 0 (FIN_WAIT_2 not counted; see A25)
A18d server closed after 20.0s
```

A19: shedding thresholds (19:30). Server J (20122, simulated-memory build, defaults 88/80, 2-second sampling). Result: thresholds, hysteresis and the decoy response behave as designed; the banner and warning still mention a 503 (RVPN-01-124).

```
70 100 -> shedding 0, upgrade -> 101
88 100 -> shedding 1, upgrade -> 404; upgrade response vs decoy (cmp): identical bytes
84 100 -> shedding 1 (hysteresis), upgrade -> 404
80 100 -> shedding 0, upgrade -> 101, handshake completed
```

A20: WebSocket growth after the cap (19:31). Server H; one WebSocket-path connection sent one fragment, waited for the server's FIN at the cap, then streamed 200 MiB of fragments. Result: accepted after the cap (RVPN-01-103).

```
t+0: sent 1 MiB FIN=0 fragment | rss=14.6 MiB
t+20.0s: server sent FIN (cap fired)
after the cap: 200 MiB more FIN=0 fragments accepted | rss=471.8 MiB
server-side socket for this connection: (FIN_WAIT_2)
```

A21: metrics exposure (19:34). Server A (metrics 127.0.0.1:20101) and server G (metrics requested on 0.0.0.0:20115 without `allow_public`). Result: both listeners on loopback only; the clamp warning was logged; the endpoint answers any bytes with the full page.

```
redsocket 78540 127.0.0.1:20101
redsocket 79841 127.0.0.1:20115
refused/timeout on <host-lan-ip>:20101 and <host-lan-ip>:20115
[METRICS] refusing to bind 0.0.0.0:20115 - that is not loopback and metrics.allow_public is false.
    ↪ Serving on 127.0.0.1:20115 instead.
```

A22: shutdown of the first session (19:36). Every recorded PID was stopped by PID (servers A to K, echo, sink, UDP echo, reference, both Redis instances); the client and helper processes had already exited; a transient Redis version probe at 19:04 had been ended by `timeout 3`; no listener remained in 20100 to 20199.

S01: scrub scan of `3012aa6` (19:27). A regular-expression scan for non-documentation addresses, key-like strings, CDN host names, mailboxes and absolute paths. Result: public addresses in `src/http.rs`, `src/proxy.rs`, `src/protection.rs`, `src/messaging.rs` and one document; synthetic CloudFront-style names in `src/protection.rs:2009–2010`; the example keypair; no mailboxes or absolute paths. The scan's IPv6 pattern missed several literals and its key search failed on a shell glob, which S02 corrected.

A23: library tests on `4430a1d` (22:35). `cargo test --offline --lib` on a scratch copy of `4430a1d` (10,211 files, identical to the index), with a stub `git` first on `PATH` so that `build.rs` could not execute git and `CARGO_INCREMENTAL=0`. Result: pass, including every fixture changed by `4430a1d`.

```
test http::real_ip_tests::cdn_headers_win_in_priority_order ... ok
test proxy::tests::assignment_is_balanced_and_survives_losing_one_address ... ok
test proxy::tests::unroutable_classification ... ok
test result: ok. 110 passed; 0 failed; 1 ignored; 0 measured; 0 filtered out; finished in 0.99s
exit=0
```

A24: read deadline before HTTP (22:37). Server P (20171, `proxy_protocol = "auto"`) and server Q (20173, `disabled`); `idle_probe.py` connected and watched for 20 s. Result: no deadline in `auto` mode (RVPN-01-105, -108).

```
Q disabled: sent=b'' first_byte_at=5.003s closed_at=5.003s bytes=b'HTTP/1.1 400 Bad Request\r\nContent-
    ↪ Length: 0\r\n\r\n'
P auto: sent=b'' first_byte_at=never closed_at=still open at 20.0s
P auto: sent=b'PROXY ' first_byte_at=never closed_at=still open at 20.0s
```

A25: silent WebSocket client across the cap (22:38). Server R (20175, 20-second cap build); `ws_silent.py` upgraded with a WebSocket key, then stayed silent without closing for 45 s; the server's sockets were read with `lsof -p 1356` (server R's PID). Result: the socket was held in `FIN_WAIT_2` after the cap until the client closed (RVPN-01-103, -107).

```
t+5s server sockets: 127.0.0.1:20175->127.0.0.1:63080 (ESTABLISHED)
t+20.0s client: server sent FIN (EOF); client keeps its side open
t+25s server sockets: 127.0.0.1:20175->127.0.0.1:63080 (FIN_WAIT_2)
t+40s server sockets: 127.0.0.1:20175->127.0.0.1:63080 (FIN_WAIT_2)
after client close: (released)
```

A26: stalled MUX2 relays after the carrier closes (22:39). Server S (20177, `deny_cidrs = []`), sink on 20179; `wp1tool mux-window --streams 32 --bytes 2097152 --hold-secs 3`; metrics, `lsof` and `ps` sampled before, during and after. Result: relays, sockets and memory outlived the carrier for at least 60 s (RVPN-01-113).

```
before: redsocket_active_relays 0 redsocket_active_tunnels 0 | sockets to the sink: 0 | fds: 15 | rss:
    ↪ 9.9 MiB
carrier open: redsocket_active_relays 32 redsocket_active_tunnels 1 | sockets to the sink: 32 (
    ↪ ESTABLISHED) | fds: 47 | rss: 23.7 MiB
t+2s after close: redsocket_active_relays 32 redsocket_active_tunnels 0 | sockets to the sink: 32 (
    ↪ ESTABLISHED) | fds: 47 | rss: 23.7 MiB
t+60s after close: redsocket_active_relays 32 redsocket_active_tunnels 0 | sockets to the sink: 32 (
    ↪ ESTABLISHED) | fds: 47 | rss: 23.7 MiB
[MUX2] tunnel closed ip=127.0.0.1 cid=0123456789abcdef age=6s bytes_up=18874336 bytes_down=0
```

A27: passive view of a session (22:41). Server T (20180, default policy); `wire_relay.py 20182 20180` recorded each read; `wp1tool v1 --meta ... --target _rs.ping:1 --atyp-domain --payload nonce-01` and `wp1tool mux --hold-secs 2` connected through the relay. Result: fixed 57-byte 101 and 50-byte first records in each direction with cleartext lengths (RVPN-01-114).

```
c2s off=0 len=104 'GET /noise HTTP/1.1\r\nHost: example.com\r\nUpgrade: websocket\r\nConnection: Upgrade\
    ↪ r\n...'
s2c off=0 len=57 'HTTP/1.1 101 Switching Protocols\r\nConnection: Upgrade\r\n\r\n'
c2s off=104 len=50 00 30 56 0f 18 af 80 24 f8 88 58 01 8f f3 25 af ...
s2c off=57 len=50 00 30 ae 48 63 e1 a6 f5 57 85 3b 4b dc 56 90 90 ...
```

```
c2s off=154 len=54 00 34 d5 c2 f8 6c 21 08 87 9b 6d 7c 16 65 fa d8 ...
(MUX2) s2c off=107 len=30 00 1c 30 a3 a8 ad 28 8d 45 b7 9c 34 0b 4a 76 5f ...
```

A28: single-label allow entry (22:43). Server U (20184, `[protection]` with the A16 fixture bundle and `[messaging ]`), plain Redis stand-in on 20183; `wp1tool mux` with header `MUX2|Asia/Tehran|rostam|android|c1d00000000000bb ||blk1,msg1`. Result: the label `example` was accepted as an allow entry and cleared `evil.example` (RVPN-01-115). After the CLEAR verdict the server looked up `evil.example` through the host's resolver (no records); no connection was made.

```
(1) before: stream1 <- verdict BLOCK bits=0x0001 reason="domain"
(2) ctl -> {"rules":{"allow":["example"],"deny":[],"categories_off":0}}
   rules:c1d00000000000bb in the store: allow ["example"] deny [] categories_off 0
   redsocket_messaging_rules_total 1 redsocket_messaging_rules_rejected_total 0
(3) after: stream1 <- verdict CLEAR (0x00)
(4) control: rules reset to allow [] -> stream1 <- verdict BLOCK bits=0x0001 reason="domain"
```

S02: scrub scan of `4430a1d` (22:44). `scrub_scan.py` over every file except `.git/`, `target/` and `vendor/`, classifying IPv4 and IPv6 literals with the Python `ipaddress` module and reporting file, line and class only. Result: 356 raw hits; see RVPN-01-123 for the classification.

S03: lock-file inventory (22:45). Python `tomllib` over `Cargo.lock` and `Cargo.toml`. Result: see Appendix B.

Re-verification session (version 1.1). All tests from A29 onward ran on 2026-10-15 between 00:40 and 01:18 host local time (UTC-7) against `147ab5e`, with the port range, PID handling and redactions described above; the server logs quoted carry UTC timestamps. Servers ran from the A29 release binary or, for A32 and A32d, from the 20-second cap build; the `3012aa6` comparisons used the A05 scratch binaries with the same cap. `RUST_LOG` was `debug` unless stated. Loopback targets were a TCP echo (20194), a TCP sink that never reads (20179), an IPv6 UDP echo on `[::1]:20189`, a plain Redis stand-in (20183) and a TLS Redis stand-in with the test CA (20186). The new helpers `pp_send.py`, `hold_relay.py`, `ws_wrap_relay.py`, `ws_after_fin.py`, `hold_port.py` and `udp6.py` are scratch scripts. No input in this session made the server attempt an off-host connection or a name lookup: every name tested was either refused by policy or blocked by a verdict, and the exemption of names under the default `allow_hosts` suffixes was confirmed statically for that reason. The end-to-end tests of A45 bound ephemeral loopback ports outside the assigned range, as in A02 and A23. Every process was stopped by PID, and no listener remained in 20100 to 20199 afterwards.

A29, A29b: release builds of `147ab5e` (00:40, 00:41). `cargo build --offline --locked --release` of a scratch copy of `147ab5e` (vendor linked read-only, git stub first on `PATH`, target directory in the scratch area), then of the same copy with the 600-second constant set to 20 seconds. Result: success in 44.7 s and 42.2 s.

```
redsocket-http 3.3.0 (unknown 2026-10-15)
   Finished `release` profile [optimized] target(s) in 44.68s
```

A30: HTTP front (00:44). Server F1 (20171, default); `probe.py 20171` with nine A06 shapes, then `idle_probe.py` after an upgrade followed by nothing and by 12 of the 50 bytes of a `msg1`. Result: the front is unchanged (RVPN-01-101, -116); both silent connections were closed at 10.0 s (RVPN-01-108).

```
upgrade-foo, upgrade-h2c, upgrade-ws-no-host, upgrade-ws-bad-key-bad-version,
post-upgrade, gate-two-hosts, proxyline-as-request -> HTTP/1.1 101 Switching Protocols
plain-get -> HTTP/1.1 404 Not Found (closed)
tls-clienthello -> HTTP/1.1 400 Bad Request after 5.001s
upgrade, then nothing: 101 at 0.000s, closed at 10.003s
upgrade, then 12 bytes of msg1: 101 at 0.000s, closed at 10.003s
[HTTP/Noise] 127.0.0.1 tunnel error: opening exchange not completed in time (2 times)
```

A31: PROXY protocol modes (00:46). Server K (20177, `[http]` without `proxy_protocol`), A (20173, `auto`) and R (20175, `required`); `pp_send.py` sent a spoofed v1 `TCP4` line, `PROXY UNKNOWN`, a v2 `LOCAL` header and no header, each followed by an upgrade tagged in the `cid` and `User-Agent`; `idle_probe.py` repeated the A24 shapes. Result: an omitted key means `Disabled`; only `required` records the header's source; `UNKNOWN` and `LOCAL` yield the socket peer; a silent or partial PROXY phase is closed at 5 s (RVPN-01-105, -108, -116).

```
banner K: proxy_protocol: Disabled A: Auto R: Required
K [HTTP] PROXY TCP4 from edge=127.0.0.1 real_ip=127.0.0.1 (line read as the request)
```

```
A [HTTP] GET /?cid=A31A-v1tcp4 from edge=127.0.0.1 real_ip=127.0.0.1
R [HTTP] GET /?cid=A31R-v1tcp4 from edge=203.0.113.7 real_ip=203.0.113.7
R [HTTP] GET /?cid=A31R-v1unknown from edge=127.0.0.1 real_ip=127.0.0.1
R [HTTP] GET /?cid=A31R-v2local from edge=127.0.0.1 real_ip=127.0.0.1
R no header: [HTTP] PROXY parse from 127.0.0.1:59697: PROXY protocol header required but not present
A nothing sent: closed at 5.003s A 'PROXY ' sent: closed at 5.003s R nothing sent: closed at 5.004s
[HTTP] PROXY parse from 127.0.0.1:59698: PROXY header not completed in time (3 times)
```

A32: connection cap (00:48). Server C20 (20190, 20-second cap build of `147ab5e`, `deny_cidrs = []`) and, for comparison, O20 (20192, the 20-second build of `3012aa6`); echo on 20194. A32a: `wp1tool v1` to the echo, `hello`, 30 s idle, then `after-cap`; A32b: `ws_silent.py 20190 40`; A32c: `wp1tool mux --hold-secs 30` through `hold_relay.py` (20182 to C20, 20197 to O20), which kept the server side open and silent for 25 s after the server's close; sockets were read with `lsof -p 43157` (C20) and `lsof -p 43178` (O20). Result: relays and sockets end at the cap on `147ab5e`; the WebSocket socket of A32b was released when its tunnel ended at the 10-second opening deadline (RVPN-01-107).

```
A32a t+8s active_relays 1 active_tunnels 1; t+25s active_relays 0 active_tunnels 0
A32a [07:49:19.809] v1: after-hold send failed: Broken pipe (os error 32)
A32b t+5s ESTABLISHED; t+10.0s client: server sent FIN; t+25s and t+35s: no server socket
A32c relays: t+20.0s server sent FIN; keeping the server-side socket open and silent
A32c t+25s C20: no server socket O20: 127.0.0.1:20192->127.0.0.1:59791 (FIN_WAIT_2)
A32c t+35s C20: no server socket O20: 127.0.0.1:20192->127.0.0.1:59791 (FIN_WAIT_2)
[HTTP] Connection timeout (20s-PATCHED) from 127.0.0.1:59758 (A32a; 59790 for A32c)
```

A32d: WebSocket-path session across the cap (01:17). Servers C20 (PID 57148) and O20 (PID 57170) as in A32; `ws_wrap_relay.py` in complete-frame mode with a 25-second hold (20186 to C20, 20187 to O20) carried `wp1tool mux --hold-secs 30` over the WebSocket path and kept the server side open and silent after the server's close. Result: on `147ab5e` the socket was released at the cap; on `3012aa6` it stayed in `FIN_WAIT_2` (RVPN-01-103, -107).

```
client (both): noise: NK handshake complete (msg1 48 B, msg2 48 B)
C20 [HTTP] WebSocket upgrade from 127.0.0.1 path=ws ua=""
relays (both): t+20.0s server side ended; keeping the server-side socket open and silent for 25s
t+25s C20: no server socket O20: 127.0.0.1:20192->127.0.0.1:60856 (FIN_WAIT_2)
t+35s C20: no server socket O20: 127.0.0.1:20192->127.0.0.1:60856 (FIN_WAIT_2)
```

A33, A33b: descriptor exhaustion and failed binds (00:54). Server U256 (20195) started under `ulimit -n 256`; `fd_hold.py 20195 300 5 46138` (46138 being U256's PID), then one new request after the hold. A33b: `hold_port.py 20199` held a port without `SO_REUSEPORT`, and server X (20198) was started with `extra_binds = ["127.0.0.1:20199"]`. Result: the process survived descriptor exhaustion, and a failed bind stopped start-up (RVPN-01-108).

```
  200 attempted: 101=200 failed=0 server_alive=True
conn 245 failed: [Errno 54] Connection reset by peer (server alive=True)
  300 attempted: 101=290 failed=10 server_alive=True
after the hold and the 10 s deadline, a new request: HTTP/1.1 404 Not Found
  10 [ACCEPT] 127.0.0.1:20195: Too many open files (os error 24); retrying in 1s
A33b [HTTP] Listening on 127.0.0.1:20198
A33b Error: cannot bind 127.0.0.1:20199: Address already in use (os error 48) (process exited)
```

A34: control replies (00:54). Server D1 (20171, default); `wp1tool v1` with the A11a META `cid` value to `_rs.stat:1`, `_rs.config:1` and `_rs.bogus:1`. Result: the `cid` is JSON-encoded, and the counters and the banner are gone; an unknown subcommand still receives an HTTP-style 404 line (RVPN-01-109).

```
_rs.stat v1 <- {"cid":"evilcid\",\"injected\":true,\"x\":\""}
_rs.config v1 <- {"noise_pubkey":"<server public key>"}
_rs.bogus v1 <- HTTP/1.1 404 unknown _rs subcommand: bogus
```

A35, A35b: log injection and identities (00:54, 00:55). A35: the A11g CONNECT, whose domain carried a line feed and a forged tunnel-open record, sent over MUX2 to server D1; then, against server I1 (20173, `RUST_LOG=info`), a v1 control call, a v1 CONNECT to 10.0.0.1, a MUX2 CONNECT to 192.0.2.1 and a MUX2 header with an invalid `cid`. A35b: a META and a MUX2 header whose `tz` carried a line feed and a forged record, against D1.

Result: control characters in names are refused and nothing identifying is logged at `info`; header fields are still logged unescaped at `debug` (RVPN-01-110).

```
A35 stream1 <- stream closed before a reply: early eof
A35 [MUX2] stream error: control character in domain addr (no forged line in the log)
A35 I1 log: 14 lines, all from start-up; the only addresses are the listeners'
A35b D1 log line 23: [2026-10-15T00:00:00.000Z INFO forged] line app=wp1 os=macos cid=c1d0000000000a35
    ↪ ...
A35b D1 log line 36: [2026-10-15T00:00:00.000Z INFO forged] mux line app=wp1 os=macos cid=
    ↪ c1d0000000000a35 ...
```

A36: unsigned and oversize store content (00:57). Server M (20184, `[protection]` with the A16 fixture bundle and `[messaging]`), plain Redis stand-in on 20183; a `msg1` client with `cid` `c1d00000000000cc` held its control stream for 14 s, after A39 had written rules for that `cid`, while a forged `entitlement` broadcast (seq 50), a 70,085-byte broadcast (seq 51) and a notice (seq 52) were written and published. Result: the earlier session's rules were replayed (RVPN-01-102); the oversize entry was skipped and counted and the stream continued; the forged entitlement was delivered (RVPN-01-111).

```
ctl <- frame 2 (136 B): {"seq":1,...,"type":"rules","payload":{"allow":["co.uk"],...},"scope":"cid"}
ctl <- frame 3 (110 B): {"seq":50,...,"type":"entitlement","payload":{"state":"banned","tier":"free"},"
    ↪ scope":"broadcast"}
ctl <- frame 4 (109 B): {"seq":52,...,"type":"notice","payload":{"title":"after the oversize entry"},"
    ↪ scope":"broadcast"}
[messaging] seq 51 of 70085 bytes skipped: over the frame limit
redsocket_messaging_dropped_total 0 -> 1
```

A37: UDP source filter (00:58). Server N (20180, `deny_cidrs = []`); `wp1tool mux-udp --target '[::1]:20189' --payload ping-from-client --hold-secs 8`; the association's socket was read with `lsof -p 48100` (server N's PID), and `udp6.py` sent one datagram from 127.0.0.1 and one from `::1` on another port. Result: address-dependent filtering (RVPN-01-112).

```
server N UDP sockets: *:55361
udpsend: 'stranger-from-127.0.0.1' from ('127.0.0.1', 51876) to ('127.0.0.1', 55361)
udpsend: 'same-address-other-port' from ('::1', 65261) to ('::1', 55361)
udp <- 43 B from ::1:20189: echo:ping-from-client
udp <- 45 B from ::1:65261: same-address-other-port
```

A38: stalled MUX2 streams (00:59). Server N, sink on 20179; `wp1tool mux-window --streams 32 --bytes 2097152` with `--hold-secs 3` (A38a, the A26 shape) and `--hold-secs 75` (A38b); metrics, `lsof` and `ps` sampled. Result: relays end with the carrier, and stalled streams end after the 60-second write timeout while the carrier stays open (RVPN-01-113).

```
A38a carrier open: active_relays 32 active_tunnels 1 | sockets to the sink: 32 | fds: 48 | rss: 23.4 MiB
A38a t+2s after close: active_relays 0 active_tunnels 0 | sockets to the sink: 0 | fds: 15 | rss: 23.5
    ↪ MiB
A38b t+30s carrier open: active_relays 32 active_tunnels 1 | sockets to the sink: 32 | fds: 48 | rss:
    ↪ 27.5 MiB
A38b t+65s carrier open: active_relays 0 active_tunnels 1 | sockets to the sink: 0 | fds: 16 | rss: 27.5
    ↪ MiB
A38b after close: active_relays 0 active_tunnels 0 | sockets to the sink: 0 | fds: 15 | rss: 27.5 MiB
```

A39: user rules (00:57). Server M and the Redis stand-in of A36, header `MUX2|Asia/Tehran|rostam|android|c1d00000000000cc||blk1,msg1`; the A28 sequence with the single label `example`, then an allow rule for the public suffix `co.uk`. Result: the label was refused and the domain stayed blocked; the public suffix was stored (RVPN-01-115).

```
(1) before: stream1 <- verdict BLOCK bits=0x0001 reason="domain"
(2) ctl -> {"rules":{"allow":["example"],"deny":[],"categories_off":0}}
   rules:c1d00000000000cc in the store: (empty)
   redsocket_messaging_rules_total 0 redsocket_messaging_rules_rejected_total 1
(3) after: stream1 <- verdict BLOCK bits=0x0001 reason="domain"
(4) ctl <- {"seq":1,...,"type":"rules","payload":{"allow":["co.uk"],...},"scope":"cid"}
   redsocket_messaging_rules_total 1 redsocket_messaging_rules_rejected_total 1
```

A40: IPv4-embedding forms (00:54). Server D1 (default policy); `wp1tool v1` CONNECTs to `[::7f00:1]:20172`, `[64:ff9b::7f00:1]:20172` and `[::ffff:7f00:1]:20172`. Result: all three tunnels were ended by the server without a connect attempt; the first two, admitted in A10a, were logged as refused (RVPN-01-117).

```
[HTTP/Noise] 127.0.0.1 tunnel error: refused unroutable destination ::7f00:1:20172 (1 address(es), none
    ↪ routable from an exit)
[HTTP/Noise] 127.0.0.1 tunnel error: refused unroutable destination 64:ff9b::7f00:1:20172 (1 address(es),
    ↪  none routable from an exit)
```

A41: key checks (01:02). Seven start attempts on 127.0.0.1:20171: (a) the shipped `config.example.toml` with only the ports changed; the former example private key (b) as `private_key`, (c) through `--private-key` and (d) in `extra_private_keys`; (e) the former example key in upper-case hex; (f) a fresh key through `--private-key`; (g) a `public_key` that does not belong to `private_key`. A first attempt at (d) produced a configuration with a duplicate key and was repeated; the scratch configurations that held the former example key were deleted. Result: placeholders and the former key are refused in all three places; the upper-case form, the command-line key and a mismatching public key are accepted (RVPN-01-119).

```
(a) Error: private_key is still the example placeholder; generate a keypair with --keygen
(b) Error: private_key is the published example key; generate a keypair with --keygen
(c) Error: private_key is the published example key; generate a keypair with --keygen
(d) Error: extra_private_keys[0] is the published example key; generate a keypair with --keygen
(e) running: [CONFIG] public_key mismatch: config=<key> derived=<key> - using derived
(f) running; a 64-hex value appears in the process arguments
(g) running: [CONFIG] public_key mismatch: config=<key> derived=<key> - using derived
```

A42: malformed handshakes (00:54). `noise_fuzz.py 20171 45788 20172` (45788 being server D1's PID), the ten shapes of A08. Result: no crash, and each attempt counted once (RVPN-01-124).

```
msg1 len=0, 1, 47, 48 random, 48 zero, 49, 65535 server_closed=True pid_alive=True
msg1 len=48 with 10 bytes / one byte / nothing server_closed=False within the 3 s window
hs_fail before: 0 after: 10 (delta 10)
```

A43, A43b: WebSocket growth (00:54, 00:58). A43: server D1; `ws_growth.py 20171 45788 declare 8`, `ws_growth.py 20171 45788 stream 4 512` and `ws_after_fin.py 20171 45788 200` (the A20 shape: one fragment, wait for the server's close, then stream). A43b: server N and the `3012aa6` server O (20192, `deny_cidrs = []`); `ws_wrap_relay.py` (20186 to N, 20187 to O) carried a `wp1tool mux-window` session inside one binary WebSocket message whose `FIN` bit was never set, 32 streams of 2 MiB to the sink on N and 4 streams of 64 KiB on O. Result: nothing is reassembled; a tunnel works inside an unfinished message on `147ab5e` and never completes its handshake on `3012aa6` (RVPN-01-103).

```
A43 before rss=11.5 MiB; 8 declared 16 MiB frames: rss=11.8 MiB
A43 stream: the first connection was reset by the server during its first 1 MiB fragment
A43 A20 shape: t+10.0s server closed its side; streaming after the close reset at once; rss 12.2 MiB
A43b N: noise: NK handshake complete (msg1 48 B, msg2 48 B)
A43b N: window: the server accepted 786432 B per stream; carrier open: active_relays 32, rss 24.7 MiB
A43b N: relay: client side ended after 25185106 B in fragments, none with FIN
A43b N: 2 s after the client left: active_relays 0 active_tunnels 0
A43b O: client still waiting for msg2 after 12 s (stopped by PID); 50 B forwarded
```

A44: statistics rows (01:03). Server ST (20187) with `[stats.valkey]` pointed at the TLS Redis stand-in (20186, test CA); (1) a non-upgrade request with `?cid=c1d0000000000a44` and `X-Forwarded-For: 203.0.113.44`; (2) a v1 control call whose META `tz` held 300 characters. The first read-back printed nothing because of how the client's output was parsed, and the run was repeated with `redis-cli --tls --raw`. Result: fields cut to 64 characters; a presence row still created from plain HTTP and attributed to the forged header (RVPN-01-104, -106); the debug log held neither the statistics password nor the private key (RVPN-01-118).

```
(1) response: HTTP/1.1 404 Not Found
stats:cid LLEN=2
row: ip=203.0.113.44 cid=c1d0000000000a44 tz_len=0 app=- os=- kind=tunnel host=<host>
row: ip=127.0.0.1 cid=c1d0000000000a45 tz_len=64 app=wp1 os=macos kind=tunnel host=<host>
debug log lines containing the statistics password: 0; containing the private key: 0
```

A45: library and new end-to-end tests (01:04). `cargo test --offline --locked --release --lib --test mux_stall_e2e --test udp_filter_e2e` on the scratch copy of `147ab5e`, with the git stub first on `PATH` and `CARGO_INCREMENTAL=0`. Result: pass.

```
test result: ok. 127 passed; 0 failed; 1 ignored; 0 measured; 0 filtered out; finished in 5.01s
Running tests/mux_stall_e2e.rs ... test result: ok. 1 passed; 0 failed
Running tests/udp_filter_e2e.rs ... test result: ok. 1 passed; 0 failed
exit=0
```

A46: dependency audit of `147ab5e` (01:06). `cargo audit -n -f Cargo.lock` on the scratch copy, whose `Cargo.lock` is identical to the repository's, with the tool and database of A04. Result: unchanged (RVPN-01-122).

```
Loaded 1329 security advisories; Scanning Cargo.lock for vulnerabilities (263 crate dependencies)
rustls 0.23.42: RUSTSEC-2026-0346, RUSTSEC-2026-0285, RUSTSEC-2026-0348, RUSTSEC-2026-0349
rustls-pemfile 2.2.0: RUSTSEC-2025-0134 (unmaintained)
error: 4 vulnerabilities found! warning: 1 allowed warning found
```

A47: metrics (01:07). Server D1; a first scrape 5.8 s after start, then a v1 CONNECT to 10.0.0.1, a v1 control call and one malformed `msg1`. Result: uptime from process start, failures counted once and only for the handshake, dropped-row counter exported, and a banner that describes the decoy (RVPN-01-124).

```
first scrape: redsocket_uptime_seconds 5 redsocket_hs_fail 0 redsocket_stats_dropped_total 0
afterwards: redsocket_uptime_seconds 7 redsocket_hs_fail 1 redsocket_stats_dropped_total 0
[SHED] ... Clients see the 404 decoy and rotate to another node; established tunnels are never touched.
```

S04: scrub scan of `147ab5e` (01:04). `scrub_scan.py` over the repository with `.git/`, `target/` and `vendor/` skipped and the untracked version 1.0 report excluded. Result: 337 raw hits: 261 crate checksums in `Cargo.lock`; two key constants, the former example private key in `src/config.rs`, kept so that it can be refused, and the former example public key as the `mux_client` default; 21 discarded on review as multicast, reserved, IPv4-embedding or partially matched forms; and 53 remaining hits; see RVPN-01-123.

Appendix B: Dependency Inventory and Advisories

Inventory. `Cargo.lock` (format version 4) resolves 263 packages: the two workspace members `redsocket-http` 3.3.0 and `e2e-harness` 0.1.0, and 261 crates from crates.io, each with a SHA-256 checksum. All 261 are vendored under `vendor/`, each with its `.cargo-checksum.json`, and `.cargo/config.toml` replaces the crates.io source with the vendored directory, so builds run offline and cargo verifies every vendored crate against the lock file. There are no git dependencies and no path dependencies outside the workspace. `io-uring` is compiled only for Linux targets. No software bill of materials, build provenance, signature or CI configuration is present in the tree, and `build.rs` stamps the binary with the output of `git rev-parse`, `git status` and `date`, so builds of one commit are not bit-for-bit reproducible (S03).

Advisories. `cargo audit -n` (cargo-audit 0.22.2; RustSec advisory database commit `edb58f59`, committed 2026-10-14 20:18 UTC; 1,329 advisories loaded; 263 crates scanned), A04:

Advisory	Crate and version	Title	Fixed in	Applicability here
RUSTSEC-2026-0285 (5.3)	rustls 0.23.42	TLS 1.3 handshake messages accepted across encryption-level boundaries	0.23.45	TLS client to pinned Valkey endpoints; needs a hostile endpoint
RUSTSEC-2026-0346 (4.0)	rustls 0.23.42	`Debug` for client session values exposes TLS secrets	0.23.46	Not reachable: session values are never formatted
RUSTSEC-2026-0348 (3.7)	rustls 0.23.42	Brotli certificate decompression allocates before authentication	0.23.46	Not reachable: no Brotli crate in the build

Advisory	Crate and version	Title	Fixed in	Applicability here
RUSTSEC-2026-0349 (7.7)	rustls 0.23.42	Unauthenticated ServerHello downgrades later connections to classical key exchange	0.23.46	Not determined; depends on hybrid key exchange with the ring provider
RUSTSEC-2025-0134 (warning)	rustls-pemfile 2.2.0	Unmaintained	n/a	Replace with `rustls-pki-types` PEM parsing

Yanked-crate status was not assessed: the audit ran without fetching the crates.io index.

Direct dependencies (`Cargo.toml:13–63`) and their resolved versions:

Crate	Requirement	Resolved	Role
snow	0.9	0.9.6	Noise NK
x25519-dalek	2 (`static_secrets`)	2.0.1	Public key derivation
tokio, tokio-util	1, 0.7	1.53.1, 0.7.19	Runtime, codecs, compat
yamux, futures	0.13, 0.3	0.13.10, 0.3.33	MUX2 multiplexing
rustls, tokio-rustls, rustls-pemfile	0.23, 0.26, 2	0.23.42, 0.26.4, 2.2.0	TLS client to Valkey (RVPN-01-122)
ring	0.17	0.17.14	Ed25519 verification, TLS provider
hickory-resolver	0.26.1	0.26.1	DNS resolution with cache
idna, flate2	1, 1	1.1.0, 1.1.10	Name normalisation, bundle decompression
serde, serde_json, toml	1, 1, 0.8	1.0.229, 1.0.151, 0.8.23	Configuration and messages
sha1, sha2, base64, hex	0.10, 0.10, 0.22, 0.4	0.10.7, 0.10.9, 0.22.1, 0.4.3	Accept key, digests, encodings
socket2, libc	0.5, 0.2	0.5.10, 0.2.186	Socket options
mimalloc	0.1	0.1.52	Global allocator
rand	0.8	0.8.7	Randomness (direct)
clap, log, env_logger, anyhow	4, 0.4, 0.11, 1	4.6.3, 0.4.33, 0.11.11, 1.0.104	CLI, logging, errors
memchr, once_cell, chrono	2, 1, 0.4	2.8.3, 1.21.4, 0.4.45	Utilities
io-uring (Linux only)	0.6.4	0.6.4	Opt-in relay backend

Duplicated crates. Eleven crates are present in more than one version: `bitflags` (1.3.2, 2.13.1), `chacha20` (0.9.1, 0.10.2), `cpufeatures` (0.2.17, 0.3.1), `getrandom` (0.2.17, 0.3.4, 0.4.3), `r-efi` (5.3.0, 6.0.0), `rand` (0.8.7, 0.9.5, 0.10.2), `rand_chacha` (0.3.1, 0.9.0), `rand_core` (0.6.4, 0.9.5, 0.10.1), `socket2` (0.5.10, 0.6.5), `syn` (2.0.119, 3.0.2) and `windows-sys` (0.52.0, 0.61.2). None carries an advisory; consolidating `rand` on one major version would shrink the audit surface.

Recommendations. Update rustls and replace rustls-pemfile (RVPN-01-122); generate a CycloneDX 1.7 or SPDX 3.0.1 bill of materials from `Cargo.lock` for each release; run `cargo audit` with a fresh database in CI and on a schedule; publish build provenance (SLSA v1.0) and sign release artifacts; derive the version stamp from source metadata so that builds are reproducible.

Appendix C: OWASP ASVS 5.0.0 Conformance Checklist

The checklist applies OWASP ASVS 5.0.0 (30 May 2025) to the exit server as built from `4430a1d`, updated for version 1.1 where the remediation pass in `147ab5e` changed a result (eleven rows) or a note. Results are Pass, Fail, Partial or Not assessed. Requirements that concern surfaces this component does not have are recorded as Not assessed with that reason; whole chapters without such a surface are listed in one row each at the end.

Requirement	Subject	Result	Note
1.1.1	Decode to canonical form once	Pass	Host names normalised once (IDNA, case, trailing dot)
1.1.2	Encode as the final step	Partial	Control replies and log lines built without encoding
1.2.1	Encoding for HTTP responses	Pass	Fixed responses; accept key is a hash
1.2.2	URL building	Not assessed	No URLs built
1.2.3	JSON output encoding	Pass	`cid` JSON-encoded in `_rs.stat` since `c5037bf` (RVPN-01-109)
1.2.4	Database query injection	Pass	RESP arrays are length-prefixed; keys use validated `cid`
1.2.5	OS command injection	Pass	Only fixed commands (`hostname`, build-time `git`, `date`)
1.2.6 to 1.2.8	LDAP, XPath, LaTeX injection	Not assessed	No such interpreters
1.2.9	Regular-expression escaping	Not assessed	No expressions built from input
1.2.10	CSV and formula injection	Partial	TSV breaks neutralised; formula prefixes kept
1.3.1	HTML sanitisation	Not assessed	No HTML input
1.3.2	No dynamic code execution	Pass	None present
1.3.3	Sanitise and trim for dangerous contexts	Partial	Row fields cut to 64 characters; debug log strings unescaped (RVPN-01-104, -110)
1.3.4, 1.3.5	SVG and template-language content	Not assessed	No such content
1.3.6	SSRF protection	Partial	Deny list holds; NAT64 local-use prefix and own addresses open (RVPN-01-117)
1.3.7 to 1.3.9	Template, JNDI, memcache injection	Not assessed	No such components
1.3.10	Format-string safety	Pass	Rust formatting is compile-time
1.3.11	Mail injection	Not assessed	No mail
1.3.12	Regular-expression denial of service	Pass	No expressions evaluated on network input
1.4.1	Memory-safe strings and buffers	Partial	Safe Rust; io_uring `unsafe` not assessed
1.4.2	Integer overflow	Pass	Lengths are u16-framed and checked; saturating counters
1.4.3	Memory and resources released	Pass	Relays and sockets end with their connections since the remediation pass (RVPN-01-107, -113)
1.5.1	XML parsers	Not assessed	No XML
1.5.2	Safe deserialisation	Pass	serde into typed structures
1.5.3	Consistent parsing across parsers	Partial	Duplicate `Host`; PROXY header no longer trusted in `auto` (RVPN-01-116)
2.1.1	Input validation rules documented	Partial	README documents the wire format only
2.1.2	Combined-data rules documented	Not assessed	No combined business data
2.1.3	Business limits documented	Partial	Shedding documented; per-user limits absent
2.2.1	Positive input validation	Partial	Parsers bounded; public-suffix rule entries accepted (RVPN-01-115)
2.2.2	Validation at a trusted layer	Pass	All validation server-side
2.2.3	Related items consistent	Partial	PROXY v1 family and addresses not cross-checked

Requirement	Subject	Result	Note
2.3.1	Steps processed in order	Pass	Upgrade, handshake, header, request enforced in sequence
2.3.2	Business limits implemented	Fail	No per-user or per-peer limits (RVPN-01-113)
2.3.3	Transactions	Partial	Rules write and mailbox append are separate commands
2.3.4, 2.3.5	Locking of scarce resources; multi-user approval	Not assessed	Not applicable
2.4.1	Anti-automation	Fail	Presence rows at line rate; no connection ceilings (RVPN-01-104, -108)
2.4.2	Realistic human timing	Not assessed	Machine protocol
4.1.1	Content-Type with charset	Partial	`text/plain` without charset
4.1.2	HTTP-to-HTTPS redirects	Not assessed	Port-80 service by design
4.1.3	Intermediary headers not user-controlled	Fail	CDN headers trusted from any peer (RVPN-01-106); PROXY fixed (RVPN-01-105)
4.1.4	Only supported methods	Fail	Any method is upgraded (RVPN-01-101)
4.1.5	Per-message signatures	Partial	Bundle signed; mailbox content unsigned (RVPN-01-111)
4.2.1	Message boundaries	Partial	No bodies parsed; duplicate `Host` accepted
4.2.2	Generated Content-Length consistent	Pass	Fixed responses are consistent
4.2.3 to 4.2.5	HTTP/2 and HTTP/3; outbound request size	Not assessed	HTTP/1.1 server only; no outbound HTTP
4.3.1, 4.3.2	GraphQL	Not assessed	No GraphQL
4.4.1	WebSocket over TLS	Partial	Plain `ws` on port 80; payload is Noise-encrypted
4.4.2 to 4.4.4	Origin check and WebSocket session tokens	Not assessed	No browser session to protect
8.1.1	Authorization rules documented	Fail	`cid` ownership model undocumented
8.1.2	Field-level rules documented	Fail	Not documented
8.1.3, 8.1.4	Contextual attributes documented	Not assessed	Level 3; not documented in the tree
8.2.1	Function-level access	Partial	Control calls open to every key holder, now per-session only (RVPN-01-109)
8.2.2	Data-specific access (IDOR)	Fail	Mailbox and rules keyed on asserted `cid` (RVPN-01-102)
8.2.3	Field-level access	Fail	Any client writes another user's rules (RVPN-01-102)
8.2.4	Adaptive controls	Not assessed	Level 3
8.3.1	Authorization at a trusted layer	Fail	Decision rests on a client-asserted identifier
8.3.2	Changes applied immediately	Partial	Rules pushed live; early streams judged without them
8.3.3	Originating subject's permissions	Not assessed	Level 3
8.4.1	Cross-tenant controls	Fail	One user's session affects another's state (RVPN-01-102)
8.4.2	Administrative interfaces	Pass	Only the loopback metrics endpoint
11.1.1	Key management policy	Not assessed	Not in the tree; rotation via `extra_private_keys`

Requirement	Subject	Result	Note
11.1.2	Cryptographic inventory	Partial	Algorithms named in README and code only
11.1.3	Cryptographic discovery	Not assessed	Level 3
11.1.4	Post-quantum migration plan	Fail	Classical X25519 only; no plan
11.2.1	Industry-validated implementations	Pass	snow, x25519-dalek, ring, rustls
11.2.2	Crypto agility	Partial	Noise pattern fixed in code; keys rotatable
11.2.3	At least 128-bit security	Pass	X25519, ChaCha20-Poly1305, BLAKE2s, Ed25519, SHA-256
11.2.4	Constant-time operations	Pass	Delegated to the libraries
11.2.5	Cryptographic modules fail securely	Pass	Decryption failures end the session
11.3.1	No ECB or weak padding	Pass	AEAD only
11.3.2	Approved ciphers and modes	Pass	ChaCha20-Poly1305; TLS via rustls defaults
11.3.3	Authenticated encryption	Pass	AEAD throughout
11.3.4	Nonce uniqueness	Pass	snow transport counters
11.3.5	Encrypt-then-MAC	Pass	AEAD
11.4.1	Approved hash functions	Pass	SHA-1 only for the RFC 6455 accept value
11.4.2	Password storage	Not assessed	No passwords stored
11.4.3	Collision-resistant integrity hashes	Pass	SHA-256 per bundle file; Ed25519
11.4.4	Password-based key derivation	Not assessed	Not used
11.5.1	CSPRNG for secrets	Pass	Keys and ephemerals from the system RNG
11.5.2	RNG under heavy demand	Not assessed	Level 3
11.6.1	Approved key generation and signatures	Pass	X25519 keygen; Ed25519 verification
11.6.2	Approved key exchange	Pass	X25519 in Noise NK
11.7.1	Memory encryption	Not assessed	Level 3; deployment
11.7.2	Minimal exposure of data in use	Partial	Key material never zeroised (RVPN-01-119)
12.1.1	Current TLS versions only	Pass	TLS 1.3 preferred, 1.2 enabled
12.1.2	Recommended cipher suites	Pass	rustls defaults
12.1.3	mTLS client certificates validated	Not assessed	Server accepts no TLS
12.1.4	Revocation checking	Partial	Private CA pinned; no revocation
12.1.5	Encrypted Client Hello	Not assessed	No outbound HTTPS
12.2.1	TLS for external HTTP services	Partial	Plain HTTP on port 80 by design; payload in Noise
12.2.2	Publicly trusted certificates	Not assessed	No TLS service
12.3.1	Encrypted protocols to back ends	Partial	Stats always TLS; messaging may set `tls = false`
12.3.2	TLS clients validate certificates	Pass	rustls with pinned roots and server names
12.3.3	TLS between internal HTTP services	Not assessed	None
12.3.4	Internal CAs pinned	Pass	Only the configured CA is trusted
12.3.5	Strong service authentication	Partial	Optional mTLS for messaging; password for stats
13.1.1	Communication needs documented	Partial	README lists the Valkey connections

Requirement	Subject	Result	Note
13.1.2	Connection limits documented	Fail	None documented or enforced
13.1.3	Resource strategies and timeouts documented	Fail	Cap semantics undocumented (RVPN-01-107)
13.1.4	Secrets and rotation documented	Partial	Key rotation documented only
13.2.1	Back-end authentication	Partial	Static Valkey passwords; mTLS optional
13.2.2	Least-privilege back-end accounts	Not assessed	Valkey ACLs not in the tree
13.2.3	No default credentials	Pass	Placeholders shipped; former example key refused (RVPN-01-119)
13.2.4, 13.2.5	Outbound allowlist	Partial	Exit by design; deny-list gaps (RVPN-01-117)
13.2.6	Documented connection configuration	Not assessed	Level 3
13.3.1	Secrets management, none in source	Partial	Former example private key kept only as a refusal constant; key accepted on the command line (RVPN-01-119)
13.3.2	Least-privilege access to secrets	Not assessed	Deployment
13.3.3	Isolated security module	Not assessed	Level 3
13.3.4	Secret expiry and rotation	Partial	Server key rotation supported
13.4.1	No source-control metadata exposed	Pass	No files served
13.4.2	Debug modes disabled	Pass	Destination logging off by default; level `warn`
13.4.3	No directory listings	Pass	No files served
13.4.4	No TRACE	Pass	TRACE without `Upgrade` receives the decoy
13.4.5	Monitoring endpoints not exposed	Pass	Metrics clamped; `_rs.stat` reduced to the `cid` (RVPN-01-109)
13.4.6	No detailed version information	Pass	`_rs.config` no longer names the product (RVPN-01-109)
13.4.7	Served file extensions restricted	Not assessed	No files served
14.1.1	Sensitive data classified	Fail	No classification of identifiers or addresses
14.1.2	Protection requirements documented	Fail	None
14.2.1	No sensitive data in URLs	Fail	`?cid=` in the legacy request line (RVPN-01-110)
14.2.2	No caching in server components	Not assessed	No caches of user data
14.2.3	No data to untrusted parties	Pass	Records go to the operator's store only
14.2.4	Controls per protection level	Partial	Identities no longer logged at the default level; records unchanged (RVPN-01-110)
14.2.5	Cache-deception protection	Not assessed	No web cache
14.2.6	Minimum sensitive data returned	Fail	Other users' entitlement (RVPN-01-102); node figures removed (RVPN-01-109)
14.2.7	Retention	Partial	Mailbox TTLs set; stats retention not in the tree
14.2.8	File metadata	Not assessed	No files
14.3.1 to 14.3.3	Client-side data protection	Not assessed	No browser client

Requirement	Subject	Result	Note
15.1.1	Remediation time frames documented	Fail	No security policy
15.1.2	SBOM maintained	Fail	None produced
15.1.3	Resource-demanding functionality documented	Partial	Shedding only
15.1.4, 15.1.5	Risky components and dangerous functionality documented	Partial	io_uring marked experimental in a comment
15.2.1	Components within remediation windows	Fail	rustls advisories (RVPN-01-122)
15.2.2	Availability defences	Partial	RVPN-01-103 and -108 fixed; RVPN-01-113 in part
15.2.3	No test code in production	Partial	`protection::fixture` compiled into the library
15.2.4	Dependencies from the expected repository	Pass	Vendored, checksummed, crates.io only
15.2.5	Isolation of dangerous parts	Not assessed	Deployment
15.3.1	Only required fields returned	Partial	`_rs.stat` returns node-wide figures
15.3.2	Redirects not followed	Pass	No outbound HTTP client
15.3.3	Mass assignment	Pass	Typed structures with explicit fields
15.3.4	Original client address from trusted fields	Fail	CDN headers from any peer (RVPN-01-106); PROXY fixed (RVPN-01-105)
15.3.5	Strict typing	Pass	Rust type system
15.3.6	Prototype pollution	Not assessed	No JavaScript
15.3.7	Parameter pollution	Partial	Duplicate `Host`: first value wins
15.4.1	Thread-safe shared state	Pass	Atomics, `Mutex` and `RwLock`; no lock held across `await`
15.4.2	No time-of-check to time-of-use	Pass	Judge and dialer share one resolution
15.4.3	Consistent locking	Pass	Short critical sections
15.4.4	No thread starvation	Not assessed	Level 3
16.1.1	Logging inventory	Fail	None
16.2.1	Log metadata	Partial	UTC timestamps; identity is the asserted `cid`
16.2.2	Synchronised UTC timestamps	Pass	UTC with millisecond precision
16.2.3	Logs only to documented sinks	Partial	Journal and stats store; no inventory
16.2.4	Common log format	Partial	Free text with key-value fragments
16.2.5	Sensitive data in logs	Partial	Identities at debug only, two rate-limited warn lines (RVPN-01-110); RVPN-01-118 fixed
16.3.1	Authentication events logged	Partial	Handshake failures counted once (RVPN-01-124), logged at debug only
16.3.2	Failed authorization logged	Partial	Invalid `cid` and rejected rules counted
16.3.3	Security events and bypass attempts	Partial	Refusals counted; probing invisible at default level
16.3.4	Unexpected errors and TLS failures	Pass	Valkey and TLS failures logged at warn
16.4.1	Log-injection encoding	Partial	Control characters refused in names; header fields unescaped at debug (RVPN-01-110)
16.4.2, 16.4.3	Log protection and forwarding	Not assessed	Deployment
16.5.1	Generic error messages	Pass	Fixed 400 and 404 bodies
16.5.2	Secure operation when back ends fail	Partial	Circuit breaker; protection fails open (RVPN-01-120)

Requirement	Subject	Result	Note
16.5.3	No fail-open	Fail	RVPN-01-120
16.5.4	Errors do not end the process	Pass	Accept errors waited out since `7594504` (RVPN-01-108)
V3 (all)	Web frontend security	Not assessed	No browser-facing content
V5 (all)	File handling	Not assessed	No file upload or download
V6 (all)	Authentication	Not assessed	No user authentication; missing client proof assessed under V8
V7 (all)	Session management	Not assessed	No sessions beyond the Noise transport
V9 (all)	Self-contained tokens	Not assessed	No tokens
V10 (all)	OAuth and OIDC	Not assessed	Not used
V17 (all)	WebRTC	Not assessed	Not used

Appendix D: Open Questions for the Maintainers

1. Is the 600-second wrapper meant to end tunnels, or only the phases before the relay? The v1 relay outlives it and MUX2 carriers do not (RVPN-01-107). Answered by the remediation pass: it now ends tunnels on every carrier (A32).
2. Does any shipped client use RFC 6455 framing? If none does, can `ws.rs` and the WebSocket branch of the io_uring backend be removed (RVPN-01-103)? The remediation pass kept the path and made it stream (`41078d3`); which clients use it is still open.
3. Do production clients reach exits directly on port 80, through a CDN, or both, and in what proportion? The answer decides how much of RVPN-01-114 is exposed today.
4. What are the descriptor limit (`LimitNOFILE`) and the memory limit of the service in production, and is the process restarted automatically after an exit (RVPN-01-108, RVPN-01-113)?
5. Is `[messaging]` enabled on any public exit today, and how do clients obtain their `cid`: randomly generated, device-derived or issued by the cashier (RVPN-01-102)?
6. Is a client-authentication step, a per-`cid` secret or a client static key, on the roadmap, and which capability flag would carry it?
7. Which systems consume `stats:cid` and `stats:tunnel_closed`, which fields do they need, and how long are rows kept (RVPN-01-104, RVPN-01-110)?
8. Which exits sit behind a PROXY-emitting load balancer or a CDN, and which configuration keys do their `[http]` tables carry (RVPN-01-105, RVPN-01-106)?
9. Which host names does the app's control plane actually use under the CDN and cloud suffixes in the default `allow_hosts` (RVPN-01-115)?
10. Does any exit network provide a NAT64 translator, or bind management services to its public addresses (RVPN-01-117)?
11. Were any of the address literals replaced by `4fb070c`, `4430a1d`, `60cf688` and `f202f2d`, or any of those listed in RVPN-01-123, including the default key in `src/bin/mux_client.rs`, live fleet or client values, and if so have they been rotated?
12. Should the `extra_binds` listeners share the primary listener's fate on error, or is independent failure intended (RVPN-01-108)? Answered by `7594504`: a failed bind now stops start-up (A33b).
13. Is the product placed on the EU market, and if so, which process and contact handle the vulnerability and incident reporting that Regulation (EU) 2024/2847 requires from 11 September 2026?
14. When will the HTTP front be withdrawn from public exits in favour of the Proteus transport, and which client versions still need it (RVPN-01-101, RVPN-01-114, RVPN-01-116)?